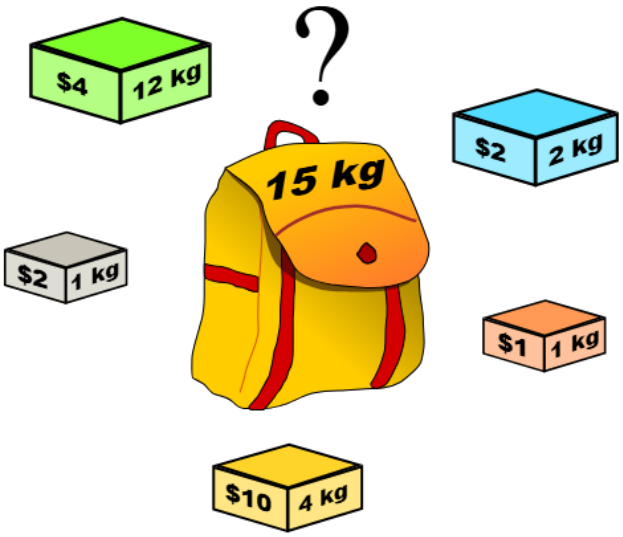
Tugas Besar 1

Penerapan Algoritma *Local Search* untuk Optimasi Pemilihan dan Penataan Paket dalam Peti Kemas

IF3170 Intelegensi Artifisial

Semester 1 Tahun 2026/2027



Disusun oleh

Three Agents

Muhammad Aufar Rizqi Kusuma	(13524061)
Arina Azka	(13524049)
Michael James Liman	(13524106)

Laboratorium Intelegensi Buatan
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **3**
1.1 *Backstory* . . . 3
1.2 *Overview* Permasalahan . . . 3
2 Landasan Teori **5**
2.1 Algoritma Hill Climbing . . . 5
2.1.1 *Steepest Ascent Hill Climbing* . . . 5
2.1.2 *Hill Climbing with Sideways Move* . . . 6
2.1.3 *Stochastic Hill Climbing* . . . 6
2.1.4 *Random Restart Hill Climbing* . . . 7
2.2 Algoritma Simulated Annealing . . . 7
2.3 Genetic Algorithm . . . 9
3 Implementasi Algoritma **11**
3.1 Representasi State . . . 11
3.2 *Constraint* Permasalahan . . . 12
3.3 *Successor Function* . . . 14
3.4 *Objective Function* . . . 14
3.5 Formulasi *Initial State* . . . 15
3.6 Implementasi Hill Climbing . . . 16
3.6.1 Pembangkitan *Neighbor* dan Heuristik . . . 16
3.6.2 Parameter dan Kondisi Berhenti . . . 16
3.6.3 Metrik yang Dicatat . . . 17
3.6.4 *Source Code* . . . 17
3.6.5 Pelaksana Eksperimen dan Visualisasi . . . 22
3.7 Implementasi Simulated Annealing . . . 22
3.7.1 Parameter dan Validasi Masukan . . . 22
3.7.2 Skema Pendinginan dan Probabilitas Penerimaan . . . 22
3.7.3 Loop Pencarian . . . 23
3.7.4 Deteksi *Stuck* pada Optimum Lokal . . . 23
3.7.5 Metrik dan Hasil . . . 24
3.7.6 Pelaksana Eksperimen dan Visualisasi . . . 24
3.8 Implementasi Genetic Algorithm . . . 24
3.8.1 Konfigurasi dan Generator Acak . . . 24
3.8.2 Inisialisasi Populasi . . . 25
3.8.3 Evaluasi *Fitness* dan Penyimpanan Individu Terbaik . . . 25
3.8.4 Seleksi Induk dengan *Roulette Wheel* . . . 25
3.8.5 Crossover Satu Titik dan Perbaikan Kelayakan . . . 26
3.8.6 Mutasi Anak . . . 26
3.8.7 Pembentukan Generasi Berikutnya dan Terminasi . . . 27
3.9 Implementasi Spesifikasi Bonus . . . 28
3.9.1 *Extra* Objective Function . . . 28
3.9.2 All-variant Hill Climbing Algorithm . . . 29
3.9.3 Multi-Truck . . . 31
3.9.4 *Visualizer* Berbasis *Web Server* . . . 35
4 Hasil Eksperimen dan Analisis **38**
4.1 Metodologi Eksperimen . . . 38

4.1.1 Kumpulan Instance dan Kasus Uji . . . 38
4.1.2 Protokol Perbandingan . . . 38
4.1.3 Metrik dan Analisis . . . 39
4.1.4 Studi Ablasi . . . 39
4.1.5 Dataset dan Konfigurasi Eksperimen . . . 39
4.2 Hasil Eksperimen dan Analisis tiap Algoritma . . . 41
4.2.1 Hill Climbing . . . 41
4.2.2 Simulated Annealing . . . 50
4.2.3 Genetic Algorithm . . . 56
4.3 Analisis Keseluruhan . . . 60
4.3.1 Perbandingan Antar-Algoritma pada Seluruh Kategori . . . 60
4.3.2 Ketahanan pada Kasus Batas dan Masukan Tidak Valid . . . 62
4.3.3 Pengaruh Variasi *Objective Function* . . . 63
4.3.4 Keterbatasan . . . 63
4.4 Analisis Implementasi Spesifikasi Bonus . . . 63
4.4.1 *Extra* Objective Function . . . 63
4.4.2 All-variant Hill Climbing Algorithm . . . 64
4.4.3 Studi Ablasi Operator GA . . . 64
4.4.4 Multi-Truck . . . 65
5 Kesimpulan dan Saran **71**
5.1 Kesimpulan . . . 71
5.2 Saran . . . 72
6 Lampiran **74**
6.1 Pembagian Tugas . . . 74
6.2 Repositori GitHub . . . 74

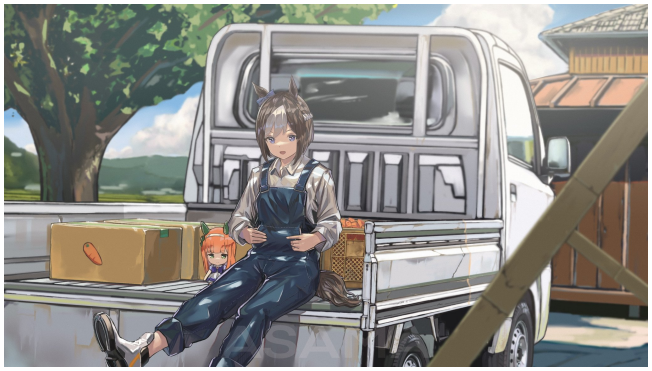
Bab 1

Pendahuluan

1.1 *Backstory*

Keluarga Mejiro sedang berencana mengekspansi bisnisnya ke bidang logistik darat dan ingin melakukan uji coba berskala kecil terlebih dahulu untuk torasen. Sebagai teman yang baik, Mejiro McQueen menawarkan posisi driver truk uji coba kepada Special Week, karena sudah sering menyetir truk di kampung halamannya di Hokkaido. Usaha baru ini bernama Mejiro Express dengan slogan "Lebih kilat dari Umamusume".

Karena Special Week sibuk berlatih untuk memenangkan lomba-lombanya, dia tidak bisa membuang waktu berlama-lama untuk menentukan pengaturan pengiriman yang optimal. Oleh karena itu, dia meminta kalian, toreena-toreena yang pintar dalam keinformatikawan, untuk membuat suatu program yang cerdas untuk menolongnya.



Gambar 1.1: Special Week, pengemudi truk andal. Ilustrasi oleh Manatsu no Gin Hourui Uma C108 Doyoubi Higashi Chiku Ho-10a di X

1.2 *Overview* Permasalahan

Tugas Besar 1 mata kuliah IF3170 Inteligensi Artifisial menugaskan mahasiswa untuk mencari konfigurasi pemuatan paket yang optimal pada sebuah truk peti kemas. Secara umum, paket memiliki ukuran, berat, dan nilai yang beragam, sedangkan ruang serta kapasitas angkut truk terbatas. Tujuan optimasi adalah memilih dan menata paket agar nilai total paket yang berhasil dimuat menjadi sebesar mungkin.

Permasalahan ini mencakup pemilihan paket bernilai tinggi sekaligus penataannya di dalam ruang tiga dimensi. Susunan paket harus tetap layak untuk diangkut dengan memperhatikan keterbatasan ruang, kestabilan tumpukan, kondisi barang pecah-belah, dan kapasitas truk. Banyaknya kombinasi paket dan posisi membuat pencarian konfigurasi terbaik sulit dilakukan secara menyeluruh.

Untuk mencari solusi, tugas ini meminta implementasi tiga algoritma pencarian, yaitu satu varian Hill Climbing, Simulated Annealing, dan Genetic Algorithm. Algoritma-algoritma tersebut mengeksplorasi konfigurasi secara

bertahap dari solusi awal yang dibentuk secara acak, lalu berusaha menemukan konfigurasi dengan nilai objektif yang lebih baik. Kinerja pencarian kemudian dibandingkan melalui kualitas solusi, perubahan nilai objektif selama proses, durasi, dan konsistensi hasil eksperimen.

Bab 2

Landasan Teori

2.1 Algoritma Hill Climbing

Hill climbing adalah algoritma *local search* yang bekerja pada satu *state* saja. Algoritma ini mulai dari sebuah *state* awal, membangkitkan *neighbor* dari *state* saat ini, lalu berpindah ke *neighbor* yang nilai *objective function*-nya lebih baik. Pencarian berhenti ketika tidak ada lagi *neighbor* yang lebih baik. Karena hanya menyimpan satu *state* dan tidak menyimpan jalur pencarian, kebutuhan memorinya sangat kecil dan setiap iterasinya murah. Pada permasalahan ini *objective function* dimaksimalkan sehingga algoritma bergerak naik menuju nilai *value* total paket yang lebih besar.

Kelemahan utama *hill climbing* adalah tidak adanya mekanisme untuk keluar dari daerah yang buruk. Ada tiga kondisi yang menyebabkan pencarian terhenti sebelum mencapai *global optimum*.

- *Local maximum* adalah *state* yang lebih baik daripada semua *neighbor*-nya, tetapi bukan yang terbaik secara global.
- *Plateau* adalah daerah datar tempat semua *neighbor* bernilai sama sehingga tidak ada arah yang jelas untuk naik.
- *Shoulder* adalah *plateau* yang masih memiliki jalan menaik di ujungnya dan dapat terlewat jika pencarian berhenti terlalu cepat.

Keempat varian berikut dibuat untuk mengurangi kelemahan tersebut dengan cara yang berbeda.

2.1.1 *Steepest Ascent Hill Climbing*

Pada setiap iterasi, algoritma mengevaluasi sekumpulan *neighbor* dan memilih yang bernilai paling tinggi. Algoritma berpindah ke kandidat itu hanya jika nilainya lebih besar daripada nilai *state* saat ini. Jika tidak, pencarian berhenti pada *local optimum*. Pemilihan terbaik bersifat deterministik untuk sampel yang dievaluasi. Namun, sampel *neighbor* dibangkitkan secara acak sehingga hasil dapat berbeda antarpercobaan. Varian ini masih dapat terjebak pada *local maximum* atau *plateau*.

Algorithm 1 Steepest-Ascent Hill Climbing

Data Solusi awal s_0, fungsi tetangga N, fungsi objektif f, jumlah sampel m, batas iterasi $I_{\max}$
Result Optimum lokal hasil pencarian s
  $s \leftarrow s_0$, $i \leftarrow 0$
  while $i < I_{\max}$ **do**
    Bangkitkan m *neighbor* valid dari $N(s)$
    $s' \leftarrow \arg\max_{u \in N(s)} f(u)$ dari sampel tersebut
    if $f(s') \leq f(s)$ **then**
      return s

end if
$s \leftarrow s'$, $i \leftarrow i + 1$
end while
return s

2.1.2 *Hill Climbing with Sideways Move*

Varian ini sama dengan *steepest ascent*, tetapi juga mengizinkan perpindahan ke *neighbor* yang nilainya sama dengan *state* saat ini (*sideways move*). Dengan cara ini algoritma dapat menyeberangi *plateau* dan *shoulder*. Agar tidak berputar tanpa akhir pada *plateau* yang tidak memiliki jalan keluar, jumlah *sideways move* dibatasi oleh parameter *maximum sideways move*. Jika tidak ada peningkatan dan batas perpindahan setara telah tercapai, pencarian berhenti.

Algorithm 2 Hill Climbing with Sideways Move

Data Solusi awal s_0, fungsi tetangga N, fungsi objektif f, jumlah sampel m, batas iterasi $I_{\max}$, batas *sideways* L
Result Solusi akhir s
$s \leftarrow s_0$, $i \leftarrow 0$, $q \leftarrow 0$
while $i < I_{\max}$ **do**
Bangkitkan m *neighbor* valid dari $N(s)$
$s' \leftarrow \arg\max_{u \in N(s)} f(u)$ dari sampel tersebut
if $f(s') > f(s)$ **then**
$s \leftarrow s'$
else
if $f(s') = f(s)$ **and** $q < L$ **then**
$s \leftarrow s'$, $q \leftarrow q + 1$
else
return s
end if
end if
$i \leftarrow i + 1$
end while
return s

2.1.3 *Stochastic Hill Climbing*

Algoritma memilih satu *neighbor* secara acak pada setiap iterasi dan berpindah ke *neighbor* tersebut hanya jika nilainya lebih baik. Varian ini tidak perlu mengevaluasi seluruh *neighbor* dalam satu iterasi sehingga setiap iterasinya lebih murah, dan sifat acaknya membuat jalur pencarian lebih bervariasi. Karena tidak ada kondisi "tidak ada *neighbor* yang lebih baik" yang dapat dipastikan dari satu sampel, pencarian dihentikan setelah jumlah iterasi maksimum tercapai.

Algorithm 3 Stochastic Hill Climbing

Data Solusi awal s_0, fungsi tetangga acak N, fungsi objektif f, batas iterasi $I_{\max}$
Result Solusi akhir s
$s \leftarrow s_0$, $i \leftarrow 0$
while $i < I_{\max}$ **do**
Pilih satu *neighbor* valid s' secara acak dari $N(s)$
if $f(s') > f(s)$ **then**
$s \leftarrow s'$
end if
$i \leftarrow i + 1$

end while
return s

2.1.4 *Random Restart Hill Climbing*

Algoritma menjalankan *steepest ascent hill climbing* dari *state* awal yang diberikan, kemudian mengulanginya dari *state* awal acak yang baru dan menyimpan hasil terbaik dari seluruh pendakian. Peluang menemukan *global optimum* meningkat seiring bertambahnya jumlah *restart*, karena setiap *restart* mengeksplorasi daerah ruang pencarian yang berbeda. Parameter *maximum restart* membatasi pengulangan setelah pendakian pertama sehingga jumlah total pendakian adalah satu ditambah jumlah *restart*. Kelemahannya adalah biaya komputasi yang bertambah sebanding dengan jumlah pendakian.

Algorithm 4 Random-Restart Hill Climbing

Data Solusi awal s_0, pembangkit solusi acak R, fungsi tetangga N, fungsi objektif f, jumlah sampel m, batas iterasi per pendakian $I_{\max}$, batas *restart* K
Result Solusi terbaik yang ditemukan s_{best}
$s_{\text{best}} \leftarrow s_0$, $f_{\text{best}} \leftarrow -\infty$
for $r \leftarrow 0$ sampai K **do**
  if $r = 0$ **then**
    $s \leftarrow s_0$
  else
    $s \leftarrow R()$
  end if
  Jalankan *steepest-ascent* dari s hingga berhenti atau mencapai $I_{\max}$. Simpan hasilnya sebagai s_r
  if $f(s_r) > f_{\text{best}}$ **then**
    $s_{\text{best}} \leftarrow s_r$, $f_{\text{best}} \leftarrow f(s_r)$
  end if
end for
return s_{best}

2.2 Algoritma Simulated Annealing

Simulated Annealing adalah algoritma pencarian lokal stokastik yang meniru proses pendinginan material untuk mencari solusi bernilai objektif tinggi. Berbeda dari Hill Climbing yang hanya menerima perpindahan ke solusi lebih baik, Simulated Annealing juga dapat menerima solusi yang lebih buruk dengan peluang tertentu. Mekanisme ini membantu pencarian keluar dari optimum lokal dan mengeksplorasi bagian lain dari ruang solusi [1, 2].

Pada setiap iterasi, algoritma memilih satu solusi tetangga dan menghitung perubahan nilai objektifnya. Untuk persoalan maksimisasi, perubahan tersebut didefinisikan sebagai $\Delta E = f(s') - f(s)$, dengan s sebagai solusi saat ini dan s' sebagai kandidat tetangga. Kandidat yang meningkatkan atau mempertahankan nilai objektif selalu diterima. Kandidat yang menurunkan nilai objektif dapat diterima dengan peluang yang bergantung pada temperatur T. Peluang ini dirumuskan pada Persamaan 3.6 [2].

$$P(\text{menerima } s') = \begin{cases} 1, & \Delta E \geq 0, \\ \exp\left(\dfrac{\Delta E}{T}\right), & \Delta E < 0. \end{cases} \tag{2.1}$$

Temperatur mengendalikan tingkat eksplorasi algoritma. Pada temperatur tinggi, penurunan nilai objektif masih mungkin diterima sehingga pencarian dapat menjelajahi ruang solusi dengan lebih luas. Ketika temperatur diturunkan, peluang menerima perpindahan yang lebih buruk ikut mengecil dan pencarian semakin berfokus pada perbaikan. Salah satu jadwal pendinginan yang umum digunakan adalah pendinginan geometris, dengan

$0 < \alpha < 1$ sebagai faktor pendinginan [1].

Sebagai contoh, jika kandidat menurunkan nilai objektif sebesar 10, peluang penerimaannya pada $T = 10$ adalah $e^{-1} \approx 0{,}37$. Pada $T = 5$, peluang itu menjadi $e^{-2} \approx 0{,}14$. Implementasi membandingkan peluang tersebut dengan bilangan acak antara 0 dan 1. Kandidat yang lebih buruk diterima jika bilangan acaknya lebih kecil daripada peluang itu. Penurunan besar atau temperatur rendah membuat langkah mundur semakin jarang diterima.

$$T_{k+1} = \alpha T_k, \qquad 0 < \alpha < 1. \tag{2.2}$$

Rumus ini berarti setiap iterasi mengalikan temperatur saat ini dengan faktor yang sama, bukan menguranginya dengan nilai tetap. Misalnya, jika $T_0 = 100$ dan $\alpha = 0{,}995$, temperatur turun sekitar 0,5% per iterasi dan setelah 100 iterasi menjadi sekitar 60,6. Penurunan bertahap ini membuat langkah eksplorasi yang awalnya cukup longgar menjadi semakin selektif.

Simulated Annealing dapat melewati optimum lokal dengan menerima perpindahan yang sesaat menurunkan nilai objektif tanpa menyimpan banyak kandidat solusi. Kualitas hasil dan waktu pencarian bergantung pada temperatur awal, faktor pendinginan, temperatur minimum, serta cara membentuk tetangga. Jika temperatur turun terlalu cepat, pencarian masih dapat terjebak pada optimum lokal. Frekuensi kondisi ini menjadi salah satu metrik evaluasi. Algoritma ini tidak menjamin solusi global terbaik dalam waktu terbatas, dan hasilnya dapat berbeda antarpercobaan [1, 2].

Simulated Annealing cocok untuk ruang solusi besar yang memiliki banyak optimum lokal, terutama jika evaluasi solusi murah dan solusi mendekati optimal tetap berguna. Pada persoalan pemuatan paket, pencarian dimulai dari konfigurasi acak. Setiap iterasi membentuk satu tetangga dengan menukar, memindahkan, atau memutar paket. Kandidat yang meningkatkan nilai objektif diterima. Kandidat yang menurunkan nilai dapat diterima sesuai peluang pada Persamaan 3.6. Setelah evaluasi, temperatur diturunkan mengikuti jadwal pendinginan. Nilai $\exp(\Delta E/T)$ mencatat peluang penerimaan kandidat yang lebih buruk dan dianalisis terhadap jumlah iterasi sesuai spesifikasi.

Pseudocode berikut menggunakan pendinginan geometris dan berhenti ketika temperatur mencapai nilai minimum. Variabel s_{best} menyimpan konfigurasi bernilai tertinggi yang ditemukan, meskipun proses menerima perpindahan sementara ke konfigurasi yang lebih buruk.

Algorithm 5 Simulated Annealing

Data Solusi awal s_0, fungsi tetangga N, fungsi objektif f, parameter $T_0 > T_{\min} > 0$ dan $0 < \alpha < 1$
Result Solusi terbaik yang ditemukan s_{best}
$s \leftarrow s_0$, $s_{\text{best}} \leftarrow s_0$, $T \leftarrow T_0$
while $T > T_{\min}$ **do**
  if $N(s) = \emptyset$ **then**
    return s_{best}
  end if
  Pilih s' secara acak dari $N(s)$
  $\Delta E \leftarrow f(s') - f(s)$
  if $\Delta E \geq 0$ **then**
    $s \leftarrow s'$
  else
    Bangkitkan u secara acak dari $[0, 1]$
    if $u < \exp(\Delta E/T)$ **then**
      $s \leftarrow s'$
    end if
  end if
  if $f(s) > f(s_{\text{best}})$ **then**
    $s_{\text{best}} \leftarrow s$
  end if
  $T \leftarrow \alpha T$
end while
return s_{best}

2.3 Genetic Algorithm

Genetic Algorithm adalah metode pencarian berbasis populasi yang memperbarui kumpulan kandidat solusi melalui mekanisme yang terinspirasi oleh seleksi alam. Setiap kandidat disebut individu, sedangkan nilai objektifnya berperan sebagai ukuran fitness. Pada tiap generasi, algoritma memilih individu sebagai induk, menggabungkan bagian solusi melalui crossover, lalu menerapkan mutasi secara acak untuk membentuk populasi baru [1, 3].

Karakteristik Genetic Algorithm adalah pencarian atas banyak kandidat secara bersamaan dan penggunaan operator stokastik untuk menghasilkan generasi berikutnya. Seleksi roulette memberi peluang lebih besar kepada individu dengan fitness lebih tinggi. Jika fitness individu ke-i adalah f_i dan bobot seleksinya w_i, peluang memilih individu tersebut dirumuskan sebagai

$$P(i) = \frac{w_i}{\sum_{j=1}^{n} w_j}. \tag{2.3}$$

Pembilang adalah bobot individu yang sedang dihitung, sedangkan penyebut menjumlahkan bobot seluruh individu agar semua peluang berjumlah satu. Jika dua individu memiliki bobot 10 dan 20, peluang terpilihnya masing-masing adalah 1/3 dan 2/3. Dengan pemilihan berbobot, individu dengan bobot 20 kira-kira dua kali lebih sering terpilih daripada individu dengan bobot 10. Individu lain tetap memiliki peluang untuk terpilih.

Untuk persoalan pemuatan paket dengan nilai objektif nonnegatif, bobot seleksi dapat menggunakan nilai objektif. Jika seluruh bobot bernilai nol, seleksi dilakukan secara seragam. Crossover satu titik menggabungkan gen dari dua induk, sementara mutasi mengubah satu gen dengan peluang tertentu. Kedua operator harus menghasilkan individu yang memenuhi batasan pemuatan paket [3].

Keunggulan Genetic Algorithm adalah kemampuannya mengeksplorasi beberapa wilayah ruang solusi secara paralel dan mengurangi ketergantungan pencarian pada satu konfigurasi awal. Keragaman populasi dapat membantu pencarian menghindari optimum lokal. Namun, hasil dan waktu pencarian bergantung pada ukuran populasi, jumlah generasi, operator, serta parameter seperti laju mutasi. Algoritma ini tidak menjamin solusi global terbaik dalam jumlah generasi terbatas, dan setiap generasi memerlukan evaluasi banyak kandidat [1, 3].

Genetic Algorithm sesuai untuk ruang solusi besar dengan representasi kandidat yang dapat digabungkan dan dimutasi, khususnya ketika solusi layak dapat dipulihkan setelah operator diterapkan. Pada persoalan pemuatan paket, satu individu merepresentasikan penempatan seluruh paket. Populasi awal memuat konfigurasi awal yang valid dan konfigurasi valid lain yang dibentuk secara acak. Di setiap generasi, individu dievaluasi dengan fungsi objektif, induk dipilih menggunakan roulette, kemudian crossover satu titik dan mutasi dilakukan untuk membentuk populasi berikutnya. Individu terbaik disimpan terpisah sepanjang pencarian agar solusi terbaik tetap tersedia meskipun tidak diwariskan ke generasi baru.

Pseudocode berikut menggunakan seleksi roulette, crossover satu titik dengan perbaikan kelayakan, dan mutasi satu gen dengan peluang μ. Pencarian berhenti setelah jumlah generasi yang ditentukan dan mengembalikan individu terbaik yang pernah dievaluasi.

Algorithm 6 Genetic Algorithm

Data Solusi awal valid s_0, fungsi objektif f, ukuran populasi n, jumlah generasi G, laju mutasi μ
Result Individu terbaik yang ditemukan s_{best}
  $P \leftarrow$ populasi valid berukuran n yang memuat s_0
  $s_{\text{best}} \leftarrow s_0$, $f_{\text{best}} \leftarrow -\infty$
  for $g \leftarrow 0$ sampai G **do**
    Evaluasi $f(x)$ untuk setiap $x \in P$
    $x_{\text{best}} \leftarrow \arg\max_{x \in P} f(x)$
    if $f(x_{\text{best}}) > f_{\text{best}}$ **then**
      $s_{\text{best}} \leftarrow x_{\text{best}}$, $f_{\text{best}} \leftarrow f(x_{\text{best}})$
    end if
    if $g < G$ **then**

$P_{\text{baru}} \leftarrow \emptyset$
while $|P_{\text{baru}}| < n$ **do**
Pilih induk p_1 dan p_2 dari P dengan seleksi roulette
$c \leftarrow$ crossover satu titik (p_1, p_2) dan perbaiki agar valid
Dengan peluang μ, mutasikan satu gen pada c dan pertahankan validitasnya
Tambahkan c ke P_{baru}
end while
$P \leftarrow P_{\text{baru}}$
end if
end for
return s_{best}

Bab 3

Implementasi Algoritma

3.1 Representasi State

State merepresentasikan satu konfigurasi pemuatan paket pada sebuah truk. Implementasi menyimpan data paket dan truk secara terpisah, lalu menghubungkan setiap paket dengan posisinya melalui pemetaan dari ID paket ke objek penempatan. Nilai penempatan `None` menandakan paket berada di luar truk sehingga state tetap mencatat seluruh paket, baik yang dimuat maupun tidak.

Objek `Package` menyimpan ID, dimensi awal, nilai, berat, status mudah pecah, dan ETA. Dimensi disimpan sebagai tiga bilangan bulat, sedangkan orientasi adalah permutasi indeks dimensi yang menentukan urutan ukuran setelah rotasi. Ada enam kemungkinan orientasi, sesuai seluruh permutasi indeks tiga dimensi. Objek `Truck` menyimpan dimensi ruang muat serta kapasitas berat maksimum. Objek `Placement` menggabungkan koordinat sudut paket, dimensi setelah orientasi diterapkan, dan orientasinya. Kode 3.1 memperlihatkan struktur data inti dari `src/model.py`. Pemeriksaan validasi dihilangkan dari cuplikan agar fokus pada field representasinya.

Kode 3.1: Struktur inti model state

```python
from dataclasses import dataclass
from itertools import permutations
from typing import Mapping, TypeAlias
Dimensions: TypeAlias = tuple[int, int, int]
Coordinate: TypeAlias = tuple[int, int, int]
Orientation: TypeAlias = tuple[int, int, int]
ORIENTATIONS = tuple(permutations((0, 1, 2)))
@dataclass(frozen=True, slots=True)
class Package:
    id: str
    dimensions: Dimensions
    value: int
    weight: int
    fragile: bool
    eta: int
@dataclass(frozen=True, slots=True)
class Truck:
    dimensions: Dimensions
    max_capacity: int
@dataclass(frozen=True, slots=True)
class Placement:
    coordinate: Coordinate
    oriented_dimensions: Dimensions
    orientation: Orientation = (0, 1, 2)
@dataclass(frozen=True, slots=True)
class State:
    packages: tuple[Package, ...]
    truck: Truck
    placements: Mapping[str, Placement | None]
```

Secara matematis, satu state dituliskan sebagai $S = (P, T, \phi)$, dengan $P = (p_1, \ldots, p_n)$ sebagai kumpulan paket, $T = (\mathbf{L}, C)$ sebagai truk berdimensi $\mathbf{L}$ dan berkapasitas maksimum C, serta ϕ sebagai pemetaan penempatan.

Jika paket p_i memiliki ID id_i dan dimensi $\mathbf{d}_i$, maka representasinya adalah

$$\begin{aligned} I &= \{\mathrm{id}_1, \ldots, \mathrm{id}_n\}, \\ \phi : I &\longrightarrow \{\perp\} \cup \left\{ (\mathbf{x}_i, \mathbf{d}_i^{\pi_i}, \pi_i) \,\middle|\, \mathbf{x}_i \in \mathbb{Z}^3_{\geq 0}, \quad \pi_i \in S_3 \right\}, \\ \mathbf{d}_i^{\pi_i} &= (d_{i,\pi_i(1)}, d_{i,\pi_i(2)}, d_{i,\pi_i(3)}). \end{aligned} \tag{3.1}$$

Di sini $\perp$ menyatakan paket tidak dimuat, $\mathbf{x}_i$ adalah koordinat sudut paket di dalam truk, dan S_3 adalah himpunan enam permutasi sumbu yang merepresentasikan orientasi. Domain I memastikan setiap paket memiliki tepat satu entri penempatan. Persamaan 3.1 menjelaskan bentuk representasi. Batas ruang, kapasitas, dan batasan fisik lainnya menentukan apakah state tersebut layak.

Pemetaan `placements` harus memiliki tepat satu entri untuk setiap ID paket. Konstruktor `State` memeriksa keunikan ID, kelengkapan kunci pemetaan, dan kecocokan dimensi hasil orientasi dengan dimensi paket. Fungsi objektif state menjumlahkan nilai paket yang memiliki penempatan sehingga paket di luar truk tidak berkontribusi terhadap nilai solusi.

Contoh berikut menunjukkan sebuah paket di dalam truk dan satu paket yang tidak dimuat. Koordinat menggunakan urutan sumbu x, y, dan z, sedangkan koordinat $(0, 0, 0)$ menandakan sudut bawah paket berada di titik asal ruang muat.

Kode 3.2: Contoh pembentukan state pemuatan

```
from src.model import Package, Placement, State, Truck

package = Package("P001", (3, 4, 1), 60, 5, False, 3)
outside_package = Package("P002", (1, 1, 1), 10, 2, False, 1)
truck = Truck((10, 20, 5), 500)

state = State(
    packages=(package, outside_package),
    truck=truck,
    placements={
        "P001": Placement((0, 0, 0), (3, 4, 1)),
        "P002": None,
    },
)
```

3.2 *Constraint* Permasalahan

State layak apabila seluruh syarat berikut terpenuhi. Misalkan $I^+ = \{i \in I \mid \phi(i) \neq \perp\}$ adalah himpunan ID paket yang dimuat. Untuk setiap $i \in I^+$, penempatannya ditulis sebagai $\phi(i) = (\mathbf{x}_i, \mathbf{d}_i, \pi_i)$, dengan $\mathbf{x}_i = (x_i, y_i, z_i)$ sebagai koordinat dan $\mathbf{d}_i = (w_i, d_i, h_i)$ sebagai dimensi setelah rotasi. Syarat kelayakan dituliskan dalam enumerasi berikut.

(1) Setiap paket harus seluruhnya berada di dalam ruang muat truk.

$$\mathbf{0} \leq \mathbf{x}_i, \qquad \mathbf{x}_i + \mathbf{d}_i \leq \mathbf{L}, \qquad \forall i \in I^+.$$

(2) Setiap pasangan paket harus terpisah pada setidaknya satu sumbu sehingga keduanya tidak saling bertumpang tindih. Di sini $d_{i,k}$ adalah ukuran paket i pada sumbu k.

$$\exists k \in \{x, y, z\} : \quad x_{i,k} + d_{i,k} \leq x_{j,k} \;\vee\; x_{j,k} + d_{j,k} \leq x_{i,k}, \qquad \forall i, j \in I^+,\ i \neq j.$$

(3) Jumlah berat semua paket yang dimuat tidak boleh melebihi kapasitas truk C. Simbol w_i^{paket} menyatakan berat paket, bukan ukuran pada sumbu x.

$$\sum_{i \in I^+} w_i^{\text{paket}} \leq C.$$

(4) Paket yang berada di atas lantai truk harus ditopang oleh setidaknya satu paket lain yang tidak rapuh. Bagian atas paket penopang harus sejajar dengan dasar paket yang ditopang, dan proyeksi alas keduanya harus beririsan.

$$z_i > 0 \Rightarrow \exists j \in I^+,\ j \neq i :$$
$$z_j + h_j = z_i, \quad \neg\,\text{rapuh}(j), \quad \text{overlap}_{xy}(i,j), \qquad \forall i \in I^+.$$

Predikat overlap_{xy} bernilai benar ketika proyeksi alas dua paket beririsan dengan luas positif. Karena itu, implementasi tidak menganggap kontak yang hanya terjadi pada tepi atau satu titik sebagai dukungan, meskipun spesifikasi menyebut dukungan pada setidaknya satu titik alas. Paket di lantai truk ($z_i = 0$) tidak memerlukan paket penopang. Kode 3.3 menunjukkan pemeriksaan yang dipakai oleh `validate_state`. Fungsi bantu irisan satu dimensi dan alas juga disertakan.

Kode 3.3: Validasi batas ruang, tabrakan, kapasitas, dan dukungan

```
from itertools import combinations

from src.model import Package, Placement, State

def _overlap_1d(start_a, size_a, start_b, size_b):
    return start_a < start_b + size_b and start_b < start_a + size_a

def _overlap_xy(a, b):
    return (
        _overlap_1d(a.coordinate[0], a.oriented_dimensions[0],
                    b.coordinate[0], b.oriented_dimensions[0])
        and _overlap_1d(a.coordinate[1], a.oriented_dimensions[1],
                        b.coordinate[1], b.oriented_dimensions[1])
    )

def _overlap_3d(a, b):
    return _overlap_xy(a, b) and _overlap_1d(
        a.coordinate[2], a.oriented_dimensions[2],
        b.coordinate[2], b.oriented_dimensions[2]
    )

def validate_state(state):
    errors = []
    placed = [
        (package, placement)
        for package in state.packages
        if (placement := state.placements[package.id]) is not None
    ]

    for package, placement in placed:
        if any(
            position + size > limit
            for position, size, limit in zip(
                placement.coordinate, placement.oriented_dimensions,
                state.truck.dimensions
            )
        ):
            errors.append(f"package {package.id!r} is outside the truck")

    for (package_a, placement_a), (package_b, placement_b) in combinations(placed, 2):
        if _overlap_3d(placement_a, placement_b):
            errors.append(f"packages {package_a.id!r} and {package_b.id!r} overlap")

    total_weight = sum(package.weight for package, _ in placed)
    if total_weight > state.truck.max_capacity:
        errors.append("truck capacity is exceeded")

    for package, placement in placed:
        if placement.coordinate[2] == 0:
            continue
        supported = any(
            not support_package.fragile
            and support_placement.coordinate[2]
                + support_placement.oriented_dimensions[2]
                == placement.coordinate[2]
            and _overlap_xy(placement, support_placement)
```

```
            for support_package, support_placement in placed
            if support_package.id != package.id
        )
        if not supported:
            errors.append(f"package {package.id!r} is unsupported")

    return errors
```

3.3 *Successor Function*

Fungsi suksesor membangkitkan kandidat dengan menerapkan salah satu operator berikut secara acak.

(1) *Move* memilih satu paket. Jika paket sudah dimuat, orientasinya dipertahankan. Jika masih di luar truk, orientasi dipilih secara acak. Paket kemudian ditempatkan pada koordinat baru yang masih berada dalam rentang truk.

(2) *Rotate* memilih satu paket yang sedang dimuat, lalu menukar dua sumbu orientasinya (rotasi 90 derajat) tanpa mengubah koordinat paket.

(3) *Swap* memilih dua paket dan menukar status pemuatan atau posisi keduanya. Dimensi penempatan disesuaikan dengan ukuran dan orientasi masing-masing paket.

Operator *swap* hanya tersedia jika state memiliki setidaknya dua paket. Misalkan $\mathcal{O}(S)$ adalah himpunan operator yang tersedia pada state S. Himpunan suksesor layak didefinisikan sebagai

$$N(S) = \{S' \mid S' = o(S),\ o \in \mathcal{O}(S),\ S' \neq S,\ \text{validate}(S') = \emptyset\}. \tag{3.2}$$

Pada operator *move*, ketinggian dipilih dari lantai truk atau permukaan atas paket lain. Setiap kandidat harus berbeda dari state asal dan lolos validasi. Implementasi mencoba paling banyak 100 kandidat, lalu mengembalikan kandidat layak pertama. Jika tidak ada paket atau tidak ditemukan kandidat yang layak, fungsi mengembalikan state asal. Cuplikan 3.4 memperlihatkan pemilihan operator dan penyaringan tersebut.

Kode 3.4: Pemilihan kandidat suksesor yang valid

```
from random import Random

def random_neighbor(state, rng=None, max_attempts=100):
    if max_attempts < 1 or not state.packages:
        return state
    rng = rng or Random()
    operations = [_move_candidate, _rotate_candidate]
    if len(state.packages) > 1:
        operations.append(_swap_candidate)

    for _ in range(max_attempts):
        candidate = rng.choice(operations)(state, rng)
        if candidate is not state and _valid(candidate):
            return candidate
    return state
```

3.4 *Objective Function*

Fungsi objektif memberi nilai pada state berdasarkan total nilai paket yang berhasil dimuat. Untuk state $S = (P, T, \phi)$, definisikan $I^+(S) = \{i \in P \mid \phi(i) \neq \perp\}$ sebagai paket yang berada di dalam truk. Nilai state dan tujuan optimasinya adalah

$$f(S) = \sum_{i \in I^+(S)} v_i, \tag{3.3}$$

$$S^* \in \arg\max_{S \in \mathcal{F}} f(S), \tag{3.4}$$

dengan v_i adalah nilai paket i dan $\mathcal{F}$ adalah himpunan state yang memenuhi seluruh constraint pada bagian sebelumnya. Dengan demikian, paket yang berada di luar truk tidak menyumbang nilai, sementara paket dengan nilai lebih tinggi meningkatkan nilai objektif jika dapat dimuat tanpa melanggar constraint. Fungsi ini tidak memberi bonus langsung untuk ruang kosong atau penalti berat. Berat hanya membatasi state melalui constraint kapasitas. Implementasi menghitung jumlah nilai paket yang memiliki penempatan, seperti ditunjukkan pada Kode 3.5.

Kode 3.5: Perhitungan nilai objektif sebuah state

```
def objective(self):
    return sum(
        package.value
        for package in self.packages
        if self.placements[package.id] is not None
    )
```

3.5 Formulasi *Initial State*

Program membentuk state awal secara konstruktif dan acak, bukan dengan mengambil sampel seragam dari seluruh state layak. Mula-mula semua paket dianggap berada di luar truk, kemudian paket diproses dalam urutan acak. Proses untuk paket ke-k dapat diringkas sebagai

(1) Atur penempatan awal setiap paket menjadi $\perp$, yang menandakan paket berada di luar truk.

(2) Acak urutan paket yang akan dicoba.

(3) Untuk setiap paket, coba hingga 50 kali secara default. Pada tiap percobaan, pilih orientasi secara acak, lalu pilih koordinat yang muat di dalam truk.

(4) Terima penempatan pertama yang menghasilkan state valid. Jika semua percobaan gagal, biarkan paket tersebut di luar truk dan lanjutkan ke paket berikutnya.

Secara formal, jika σ adalah urutan acak paket, penempatan dimulai dengan $\phi_0(i) = \perp$ untuk setiap $i \in P$. Pada langkah k, kandidat ϕ'_k mengganti penempatan paket σ_k. Kandidat diterima hanya jika $\text{validate}(P, T, \phi'_k) = \emptyset$. Setelah seluruh paket diproses, pemetaan yang terbentuk menjadi state awal $S_0 = (P, T, \phi)$. Koordinat horizontal dipilih acak dari rentang integer yang muat, sedangkan koordinat z dipilih dari lantai truk atau ketinggian permukaan atas paket yang sudah ditempatkan. Kode 3.6 memperlihatkan inti proses tersebut.

Kode 3.6: Pembentukan state awal acak yang valid

```
def random_initial_state(packages, truck, rng=None, attempts_per_package=50):
    rng = rng or Random()
    placements = {package.id: None for package in packages}
    order = list(packages)
    rng.shuffle(order)

    for package in order:
        for _ in range(attempts_per_package):
            orientation = rng.choice(ORIENTATIONS)
            dimensions = package.oriented_dimensions(orientation)
            if any(size > limit for size, limit in zip(dimensions, truck.dimensions)):
                continue
            candidate_placements = placements.copy()
            candidate_placements[package.id] = _candidate_position(
                package, orientation, truck, placements, rng
            )
            candidate = _make_state(packages, truck, candidate_placements)
            if _valid(candidate):
                placements = candidate_placements
                break

    return _make_state(packages, truck, placements)
```

Karena paket diproses satu per satu dengan jumlah percobaan terbatas, hasil state awal bergantung pada urutan dan bilangan acak yang digunakan. Paket yang gagal ditempatkan tidak menghalangi paket berikutnya untuk dicoba. Strategi ini selalu mempertahankan state valid, tetapi tidak menjamin nilai awal maksimum atau bahwa setiap paket yang sebenarnya dapat dimuat akan berhasil ditempatkan.

3.6 Implementasi Hill Climbing

Seluruh varian *hill climbing* diimplementasikan pada berkas `src/algorithms/hill_climbing/algorithm.py`. Algoritma ditulis generik terhadap tipe *state*. Seluruh operasi yang bergantung pada domain (nilai *objective*, validasi, pembangkitan *state* acak, dan pembangkitan *neighbor*) diterima melalui objek `SearchProblem` pada `src/contracts.py`. Fungsi `search_problem_for_state` pada `src/search_problems.py` memasangkan objek tersebut sesuai tipe *state*. Untuk `State` (satu truk) dipakai `validate_state`, `random_initial_state`, dan `random_neighbor`, sedangkan untuk `FleetState` (multi-truk) dipakai padanan versi armada. Dengan demikian satu implementasi *hill climbing* dapat dipakai pada kedua jenis *state*, dan semua algoritma menerima masukan serta mengembalikan keluaran yang seragam, yaitu objek `SearchResult` yang memuat *state* awal, *state* akhir, nilai *objective function* akhir, riwayat nilai per iterasi, durasi pencarian, dan metrik khusus algoritma.

3.6.1 Pembangkitan *Neighbor* dan Heuristik

Ruang *neighbor* pada permasalahan ini sangat besar karena koordinat paket berupa bilangan bulat sehingga seluruh *neighbor* tidak dapat dienumerasi pada setiap iterasi. Sebagai gantinya, setiap *neighbor* dibangkitkan lewat `problem.neighbor`. Pada satu truk fungsi ini adalah `random_neighbor`, yang secara acak memilih salah satu dari tiga operasi (menukar dua paket, memindahkan paket ke koordinat lain, atau merotasi paket 90°) dan hanya mengembalikan *state* yang lolos validasi *constraint*. Bila tidak ada kandidat valid dalam batas percobaan, fungsi mengembalikan *state* asal. Sebelum dievaluasi, setiap kandidat diperiksa ulang dengan `problem.is_valid`, dan `ValueError` dilempar jika operator *neighbor* menghasilkan *state* yang tidak valid.

Heuristik yang kami gunakan adalah *steepest* berbasis sampel. Pada setiap iterasi dibangkitkan `neighbors_per_iteration` (bawaan 30) *neighbor* valid, kemudian *neighbor* dengan nilai terbaik dianggap sebagai *neighbor* terbaik. Konsekuensinya, varian *steepest* dan *sideways* tidak menjamin menemukan *neighbor* terbaik yang sesungguhnya, tetapi peluangnya naik seiring bertambahnya ukuran sampel. Varian *stochastic* memakai tepat satu *neighbor* per iterasi.

3.6.2 Parameter dan Kondisi Berhenti

Parameter	Bawaan	Keterangan
`variant`	`steepest`	`steepest`, `sideways`, `stochastic`, atau `random_restart`
`neighbors_per_iteration`	30	Jumlah *neighbor* yang disampel per iterasi (tidak dipakai *stochastic*)
`max_iterations`	1000	Batas iterasi. Menjadi kondisi berhenti pada *stochastic* dan batas pengaman pada varian lain
`max_sideways`	100	Batas *sideways move* (varian *sideways*)
`max_restarts`	10	Jumlah *restart* setelah percobaan pertama (varian *random restart*)
`seed`	—	Seed pembangkit acak agar eksperimen dapat diulang

Tabel 3.1: Parameter algoritma *hill climbing*

- *Steepest*. Berhenti ketika *neighbor* terbaik dari sampel tidak lebih baik daripada *state* saat ini.
- *Sideways*. Sama seperti *steepest*, tetapi *neighbor* bernilai sama diterima selama jumlah *sideways move* belum mencapai `max_sideways`.
- *Stochastic*. Berhenti setelah `max_iterations` iterasi. Iterasi yang *neighbor*-nya ditolak tetap dihitung.

- *Random restart.* Menjalankan *steepest* dari *state* awal yang diberikan, lalu dari `max_restarts` *state* acak baru sehingga total terdapat `max_restarts` + 1 pendakian. Hasil terbaik dari seluruh pendakian dikembalikan.

3.6.3 Metrik yang Dicatat

Setiap *run* mengembalikan `HillClimbingMetrics` yang berisi varian yang dipakai, banyak iterasi hingga berhenti, jumlah dan batas *sideways move*, jumlah dan batas *restart*, serta banyak iterasi pada setiap *restart*. Riwayat nilai *objective function* dicatat pada setiap iterasi (elemen pertama adalah nilai *state* awal) dan digunakan untuk membuat plot nilai *objective function* terhadap banyak iterasi. Durasi pencarian diukur dengan `perf_counter`.

3.6.4 *Source Code*

Berkas `algorithm.py` terdiri dari dua fungsi bantu validasi (`_check_int` dan `_score`), fungsi pendakian `_climb`, fungsi utama `hill_climbing`, dan empat fungsi pembungkus. Bagian ini membahas kodenya potongan demi potongan; seluruh listing disalin dari `algorithm.py`.

Impor dan Konstanta

Kode 3.7: Impor dan konstanta pada `algorithm.py`

```
from math import isfinite
from random import Random
from time import perf_counter
from typing import TypeVar

from src.contracts import ObjectiveFunction, SearchProblem
from src.results import HillClimbingMetrics, ObjectiveValue, SearchResult

StateT = TypeVar("StateT")
VARIANTS = ("steepest", "sideways", "stochastic", "random_restart")
```

`StateT` adalah variabel tipe sehingga seluruh fungsi berlaku untuk tipe *state* apa pun, baik `State` satu truk maupun `FleetState`. `SearchProblem` dan `ObjectiveFunction` berasal dari kontrak bersama, sedangkan `HillClimbingMetrics` dan `SearchResult` adalah format keluaran yang sama untuk semua algoritma. Tuple `VARIANTS` adalah daftar nama varian yang sah. Daftar ini dipakai untuk memvalidasi parameter `variant`, pilihan `-variants` pada baris perintah, dan pilihan varian pada *visualizer*, sehingga nama varian hanya didefinisikan di satu tempat.

Fungsi Bantu Validasi

Kode 3.8: Fungsi `_check_int` dan `_score`

```
def _check_int(value, name: str, minimum: int) -> None:
    if type(value) is not int or value < minimum:
        raise ValueError(f"{name} must be an integer >= {minimum}")


def _score(objective: ObjectiveFunction[StateT], state: StateT) -> ObjectiveValue:
    score = objective(state)
    if isinstance(score, bool) or not isinstance(score, (int, float)) or not isfinite(score):
        raise ValueError("objective_fn harus mengembalikan finite integer atau float values")
    return score
```

`_check_int` memastikan parameter berupa bilangan bulat dengan batas bawah tertentu. Pemeriksaan memakai `type(value) is not int`, bukan `isinstance`, sehingga nilai `bool` (subkelas `int` pada Python) ikut ditolak. `_score` membungkus setiap pemanggilan *objective function* dan menolak hasil yang bukan bilangan, berupa `bool`, atau tidak berhingga (`NaN` dan tak hingga). Dengan begitu perbandingan nilai pada `_climb` selalu terjadi antar-bilangan yang terdefinisi, termasuk untuk *objective* alternatif yang dibuat pengguna.

Fungsi `_climb`: Persiapan

Kode 3.9: Tanda tangan dan inisialisasi `_climb`

```
def _climb(
    start: StateT,
    problem: SearchProblem[StateT],
    objective: ObjectiveFunction[StateT],
    rng: Random,
    *,
    variant: str,
    neighbors_per_iteration: int,
    max_iterations: int,
    max_sideways: int,
) -> tuple[StateT, ObjectiveValue, list[ObjectiveValue], int, int]:

    current, current_score = start, _score(objective, start)
    history = [current_score]
    iterations = sideways = 0
```

`_climb` menjalankan satu pendakian lengkap dari `start` dan mengembalikan lima nilai secara berurutan: *state* akhir, nilai *objective* akhir, riwayat nilai, jumlah iterasi, dan jumlah *sideways move*. Seluruh parameter pengaturan bersifat *keyword-only*. `current` dan `current_score` menyimpan posisi pendakian saat ini. `history` dimulai dengan nilai *state* awal, sehingga panjangnya selalu satu lebih banyak daripada jumlah iterasi. `iterations` dan `sideways` dimulai dari nol.

Fungsi `_climb`: Pembangkitan dan Evaluasi Kandidat

Kode 3.10: Pembangkitan dan evaluasi kandidat pada `_climb`

```
    while iterations < max_iterations:
        if variant == "stochastic":
            candidates = [problem.neighbor(current, rng)]
        else:
            candidates = [problem.neighbor(current, rng) for _ in range(neighbors_per_iteration)]
        if any(not problem.is_valid(candidate) for candidate in candidates):
            raise ValueError("neighbor␣operator␣returned␣invalid␣state")
        scored = [(_score(objective, c), c) for c in candidates]
        best_score, best = max(scored, key=lambda pair: pair[0])
```

Perulangan berjalan selama `iterations` belum mencapai `max_iterations`. Pada varian *stochastic*, hanya satu *neighbor* dibangkitkan per iterasi. Pada varian lain, `neighbors_per_iteration` *neighbor* dibangkitkan lewat `problem.neighbor` dari *state* saat ini. Setiap kandidat diperiksa dengan `problem.is_valid`; bila ada yang tidak valid, `ValueError` dilempar karena itu berarti operator *neighbor* melanggar kontrak. Kandidat lalu dinilai satu per satu dengan `_score`, dan `max` memilih pasangan (nilai, *state*) bernilai tertinggi. Bila beberapa kandidat sama nilainya, `max` mengembalikan yang dibangkitkan paling awal. Untuk *stochastic*, daftar kandidat hanya berisi satu elemen, sehingga elemen itu otomatis menjadi `best`.

Fungsi `_climb`: Keputusan Pindah atau Berhenti

Kode 3.11: Percabangan keputusan pada `_climb`

```
        if best_score > current_score:
            current, current_score = best, best_score
        elif variant == "sideways" and best_score == current_score and sideways < max_sideways:
            current = best
            sideways += 1
        elif variant == "stochastic":
            iterations += 1
            history.append(current_score)
            continue
        else:
            break
        iterations += 1
```

```
        history.append(current_score)

    return current, current_score, history, iterations, sideways
```

Cabang pertama berlaku untuk semua varian: bila kandidat terbaik lebih baik daripada *state* saat ini, pendakian pindah ke kandidat tersebut. Cabang kedua hanya untuk *sideways*: bila kandidat terbaik sama nilainya dan jatah belum habis, pendakian tetap pindah dan `sideways` bertambah. Cabang ketiga hanya untuk *stochastic*: kandidat yang tidak lebih baik ditolak, tetapi iterasi tetap dihitung dan nilai saat ini dicatat ulang, lalu `continue` melewati penambahan di bawahnya agar tidak terhitung dua kali. Cabang terakhir menghentikan pendakian dengan `break`. Ini terjadi pada *steepest* yang tidak menemukan perbaikan (*local optimum* menurut sampel) dan pada *sideways* yang kandidat terbaiknya lebih buruk atau jatahnya habis. Karena `break` terjadi sebelum `iterations += 1`, iterasi terakhir yang gagal tidak dihitung, sehingga untuk *steepest* `iterations` sama dengan jumlah perpindahan yang diterima. Hal ini menjelaskan mengapa rata-rata iterasi *steepest* pada Tabel 4.22 hanya 0,33.

Fungsi `hill_climbing`: Validasi dan Persiapan

Kode 3.12: Validasi parameter dan persiapan pada `hill_climbing`

```
def hill_climbing(
    initial_state: StateT,
    *,
    seed: int,
    objective_fn: ObjectiveFunction[StateT] | None = None,
    search_problem: SearchProblem[StateT] | None = None,
    variant: str = "steepest",
    neighbors_per_iteration: int = 30,
    max_iterations: int = 1000,
    max_sideways: int = 100,
    max_restarts: int = 10,
    objective_name: str | None = None,
    run_index: int = 1,
) -> SearchResult[StateT]:
    if variant not in VARIANTS:
        raise ValueError(f"variant harus salah satu dari {VARIANTS}")
    if type(seed) is not int:
        raise ValueError("seed harus berupa bilangan bulat")
    _check_int(neighbors_per_iteration, "neighbors_per_iteration", 1)
    _check_int(max_iterations, "max_iterations", 0)
    _check_int(max_sideways, "max_sideways", 0)
    _check_int(max_restarts, "max_restarts", 0)
    _check_int(run_index, "run_index", 1)

    started_at = perf_counter()
    rng = Random(seed)
    from src.search_problems import search_problem_for_state

    problem = search_problem or search_problem_for_state(initial_state, objective_fn)
    objective = objective_fn if objective_fn is not None else problem.objective_fn
    if not problem.is_valid(initial_state):
        raise ValueError("initial_state is invalid")
    climb_kwargs = dict(
        variant="steepest" if variant == "random_restart" else variant,
        neighbors_per_iteration=neighbors_per_iteration,
        max_iterations=max_iterations,
        max_sideways=max_sideways,
    )
```

Fungsi ini adalah pintu masuk yang sesuai kontrak algoritma: menerima *state* awal, `seed`, dan `objective_fn` opsional, serta mengembalikan `SearchResult`. Parameter `search_problem` opsional dipakai bila pemanggil ingin memakai operasi domain sendiri. Sebelum mencari, fungsi memvalidasi nama varian, tipe `seed` (harus `int`), dan batas bawah parameter bilangan bulat: `neighbors_per_iteration` minimal 1, `max_iterations`, `max_sideways`, dan `max_restarts` minimal 0, serta `run_index` minimal 1. Dengan `max_iterations` 0, pencarian langsung mengembalikan *state* awal.

Setelah itu waktu mulai dicatat dengan `perf_counter` dan satu pembangkit acak `Random(seed)` dibuat untuk

seluruh *run*, sehingga hasil dapat diulang dengan *seed* yang sama. `problem` diambil dari `search_problem` bila diberikan, atau dipasangkan otomatis oleh `search_problem_for_state` sesuai tipe *state*. *Objective* yang dipakai adalah `objective_fn` bila diberikan, atau *objective* bawaan `problem`. *State* awal diperiksa kevalidannya. Terakhir, `climb_kwargs` mengumpulkan parameter yang diteruskan ke `_climb`; varian *random restart* dipetakan ke `"steepest"` karena tiap pendakiannya memakai aturan *steepest*.

Fungsi `hill_climbing`: Cabang *Random Restart*

Kode 3.13: Cabang *random restart* pada `hill_climbing`

```
    if variant == "random_restart":
        best_state, best_score = initial_state, None
        history: list[ObjectiveValue] = []
        per_restart: list[int] = []
        for restart in range(max_restarts + 1):
            start = (
                initial_state
                if restart == 0
                else problem.random_state(rng)
            )
            if not problem.is_valid(start):
                raise ValueError("initial-state␣operator␣returned␣invalid␣state")
            state, score, hist, iters, _ = _climb(
                start, problem, objective, rng, **climb_kwargs
            )
            history.extend(hist)
            per_restart.append(iters)
            if best_score is None or score > best_score:
                best_state, best_score = state, score
        final_state, final_score = best_state, best_score
        iterations, sideways = sum(per_restart), 0
        restarts = max_restarts
```

Perulangan menjalankan `max_restarts` + 1 pendakian. Pendakian ke-0 dimulai dari `initial_state` yang diberikan, sedangkan pendakian berikutnya dimulai dari *state* acak baru dari `problem.random_state(rng)`, yang diperiksa kevalidannya sebelum dipakai. Semua pendakian memakai pembangkit acak `rng` yang sama, sehingga seluruh *run* tetap ditentukan oleh satu *seed*. Setiap hasil pendakian ditambahkan ke `history` (`extend`), dan jumlah iterasinya dicatat pada `per_restart`. Hasil terbaik dipertahankan dengan perbandingan `score > best_score` yang ketat, sehingga bila dua pendakian sama baiknya, yang lebih awal dipertahankan. Setelah perulangan, `iterations` adalah jumlah iterasi seluruh pendakian, `sideways` bernilai 0 karena tidak ada *sideways move* pada pendakian *steepest*, dan `restarts` bernilai `max_restarts`. Akibat `extend`, kurva riwayat *objective* turun pada titik awal tiap *restart* dan tidak monoton.

Fungsi `hill_climbing`: Varian Lain

Kode 3.14: Cabang varian selain *random restart*

```
    else:
        final_state, final_score, history, iterations, sideways = _climb(
            initial_state, problem, objective, rng, **climb_kwargs
        )
        per_restart, restarts = [], 0
```

Untuk *steepest*, *sideways*, dan *stochastic*, `_climb` dipanggil sekali dari *state* awal. Karena tidak ada *restart*, `per_restart` berupa daftar kosong dan `restarts` bernilai 0.

Fungsi `hill_climbing`: Penyusunan Hasil

Kode 3.15: Penyusunan parameter dan `SearchResult`

```
    parameters: dict[str, int | float | str | bool] = {
        "variant": variant,
```

```
        "max_iterations": max_iterations,
    }
    if variant != "stochastic":
        parameters["neighbors_per_iteration"] = neighbors_per_iteration
    if variant == "sideways":
        parameters["max_sideways"] = max_sideways
    if variant == "random_restart":
        parameters["max_restarts"] = max_restarts

    return SearchResult(
        algorithm=f"hill_climbing_{variant}",
        objective_name=objective_name or (
            "custom" if objective_fn is not None else problem.objective_name
        ),
        initial_state=initial_state,
        final_state=final_state,
        objective_value=final_score,
        objective_history=tuple(history),
        duration_seconds=perf_counter() - started_at,
        metrics=HillClimbingMetrics(
            variant=variant,
            iterations=iterations,
            sideways_moves=sideways,
            maximum_sideways_moves=max_sideways if variant == "sideways" else None,
            restarts=restarts,
            maximum_restarts=max_restarts if variant == "random_restart" else None,
            iterations_per_restart=tuple(per_restart),
        ),
        seed=seed,
        run_index=run_index,
        parameters=parameters,
    )
```

Kamus `parameters` hanya memuat parameter yang relevan bagi varian terpilih: `neighbors_per_iteration` tidak dicatat untuk *stochastic*, `max_sideways` hanya untuk *sideways*, dan `max_restarts` hanya untuk *random restart*. `SearchResult` memuat nama algoritma (misalnya `hill_climbing_sideways`), nama *objective* (diambil dari `objective_name`, atau `"custom"` bila `objective_fn` diberikan tanpa nama, atau nama bawaan `problem`), *state* awal dan akhir, nilai akhir, riwayat *objective* (diubah menjadi *tuple*), durasi sejak `started_at`, `seed`, `run_index`, dan `parameters`. `HillClimbingMetrics` mencatat varian, jumlah iterasi, jumlah *sideways move* beserta batasnya (hanya untuk *sideways*), jumlah *restart* beserta batasnya (hanya untuk *random restart*), dan iterasi tiap pendakian. Durasi mencakup validasi dan seluruh pendakian, termasuk semua *restart*.

Fungsi Pembungkus Varian

Kode 3.16: Fungsi pembungkus tiap varian

```
def steepest_ascent_hill_climbing(initial_state: StateT, **kw) -> SearchResult[StateT]:
    return hill_climbing(initial_state, variant="steepest", **kw)


def sideways_hill_climbing(initial_state: StateT, **kw) -> SearchResult[StateT]:
    return hill_climbing(initial_state, variant="sideways", **kw)


def stochastic_hill_climbing(initial_state: StateT, **kw) -> SearchResult[StateT]:
    return hill_climbing(initial_state, variant="stochastic", **kw)


def random_restart_hill_climbing(initial_state: StateT, **kw) -> SearchResult[StateT]:
    return hill_climbing(initial_state, variant="random_restart", **kw)
```

Keempat fungsi ini hanya memanggil `hill_climbing` dengan `variant` yang sudah ditetapkan dan meneruskan parameter lain lewat `**kw`. Pemanggil dapat memilih varian lewat nama fungsi tanpa menulis argumen `variant` (menyertakan `variant` lagi pada `**kw` akan menimbulkan `TypeError`).

3.6.5 Pelaksana Eksperimen dan Visualisasi

Pelaksana eksperimen *hill climbing* berada pada `src/experiments/hc/experiments.py`, dengan antarmuka baris perintah pada `src/experiments/hc/run.py`. `src/experiments/run_hill_climbing.py` hanya meneruskan ke *entry point* tersebut. Fungsi `run_hc_experiments` menjalankan setiap varian sebanyak tiga kali dengan tiga *seed* berbeda. *State* awal setiap *run* dibangkitkan secara acak dari *seed* yang dikenai XOR dengan konstanta tetap (`seed ^ 0x5EED`). XOR adalah operasi bit deterministik yang membentuk seed turunan untuk inisialisasi. Pasangan seed dan konstanta yang sama selalu menghasilkan seed awal yang sama. Karena seed turunan ini dipakai bersama, seluruh varian dan seluruh *objective function* yang dibandingkan memulai dari *state* awal yang sama. Untuk setiap *run*, program menyimpan berkas `result.json`, plot nilai *objective function* terhadap iterasi, dan visualisasi tiga dimensi *state* awal dan akhir. Ringkasan lintas *run* (rata-rata, simpangan baku, dan nilai terbaik *objective*, serta rata-rata durasi dan iterasi) disimpan pada `summary.json` dan `summary.png`. Program dijalankan melalui `python -m src.experiments.run_hill_climbing`.

3.7 Implementasi Simulated Annealing

Implementasi *Simulated Annealing* (SA) berada pada berkas `src/algorithms/simulated_annealing/algorithm.py`. Algoritma ini memakai representasi `State`, validator *constraint*, dan fungsi `random_neighbor` yang sama dengan *hill climbing* dan *genetic algorithm*, serta mengembalikan objek `SearchResult` yang seragam. Seluruh fungsi pada algoritma ini terdapat dalam satu fungsi `simulated_annealing`. Uraian berikut mengikuti urutan eksekusinya.

3.7.1 Parameter dan Validasi Masukan

Pemanggil menyediakan *state* awal, *seed*, jumlah iterasi maksimum, temperatur awal, dan laju pendinginan. Fungsi objektif dapat berupa `State.objective` atau fungsi lain yang diberikan pemanggil. Fungsi memeriksa bahwa *seed* berupa bilangan bulat, `max_iterations` tidak negatif, `initial_temperature` bernilai hingga dan positif, serta `cooling_rate` berada pada rentang $(0, 1]$. Pelanggaran memunculkan `ValueError` sebelum pencarian dimulai.

Parameter	Bawaan CLI	Keterangan
`max_iterations`	2000	Jumlah iterasi. Satu-satunya kondisi berhenti
`initial_temperature`	100,0	Temperatur awal T_0
`cooling_rate`	0,995	Faktor pengali temperatur setiap iterasi (α)
`stuck_patience`	100	Jumlah iterasi tanpa perbaikan terbaik hingga dihitung sebagai *stuck*
`seed`	—	*Seed* pembangkit acak agar eksperimen dapat diulang

Tabel 3.2: Parameter algoritma *simulated annealing*

3.7.2 Skema Pendinginan dan Probabilitas Penerimaan

Temperatur diturunkan secara geometris. Pada iterasi ke-t, temperatur adalah

$$T_t = T_0 \, \alpha^t, \tag{3.5}$$

dengan T_0 sebagai temperatur awal dan α sebagai `cooling_rate`. Karena persoalan ini adalah maksimisasi, selisih nilai kandidat terhadap *state* saat ini ditulis $\Delta E = f(S') - f(S)$. Kandidat yang tidak lebih buruk ($\Delta E \geq 0$) selalu diterima, sedangkan kandidat yang lebih buruk diterima dengan peluang

$$P(\text{terima}) = \begin{cases} 1 & \Delta E \geq 0, \\ e^{\Delta E / T_t} & \Delta E < 0. \end{cases} \tag{3.6}$$

Dalam kode, rumus pendinginan berarti nilai temperatur cukup dikalikan dengan `cooling_rate` pada setiap iterasi. Bentuk berpangkat merangkum pengulangan perkalian tersebut sebanyak t kali. Semakin rendah

temperatur atau semakin besar penurunan nilai, semakin kecil peluang menerima langkah yang memburuk. Peluang itu menjadi ambang untuk bilangan acak. Kandidat diterima jika bilangan acak kurang dari $e^{\Delta E/T_t}$. Kode 3.17 menunjukkan fungsi bantu yang menghitung ambang ini.

Kode 3.17: Peluang penerimaan kandidat

```
def _acceptance_probability(delta, temperature):
    if delta >= 0:
        return 1.0
    if temperature <= 0:
        return 0.0
    return exp(delta / temperature)
```

3.7.3 Loop Pencarian

Pencarian dimulai dari *state* awal yang diberikan. Pada setiap iterasi, satu *neighbor* valid dibangkitkan oleh `random_neighbor`, nilainya dievaluasi, lalu peluang pada Persamaan 3.6 dicatat. Kandidat diterima jika $\Delta E \geq 0$ atau jika bilangan acak seragam pada $[0, 1)$ lebih kecil daripada peluang tersebut. Setelah itu, temperatur dikalikan dengan `cooling_rate`. Selain *state* saat ini, algoritma menyimpan *state* terbaik yang pernah dijumpai sehingga langkah yang memburuk tidak menghilangkan solusi terbaik.

Kode 3.18: Loop utama *simulated annealing*

```
rng = Random(seed)
objective = resolve_objective(initial_state, objective_fn)
current = best = initial_state
current_score = best_score = _validate_objective(objective(current))
temperature = float(initial_temperature)

for _ in range(max_iterations):
    candidate = random_neighbor(current, rng)
    candidate_score = _validate_objective(objective(candidate))
    delta = candidate_score - current_score
    acceptance_probability = _acceptance_probability(delta, temperature)
    acceptance_history.append(acceptance_probability)
    temperature_history.append(temperature)

    if delta >= 0 or rng.random() < acceptance_probability:
        current, current_score = candidate, candidate_score

    if current_score > best_score:
        best, best_score = current, current_score
        stalled_iterations = 0
        plateau_counted = False
    else:
        stalled_iterations += 1
        if stalled_iterations >= stuck_patience and not plateau_counted:
            stuck_count += 1
            plateau_counted = True
    objective_history.append(best_score)
    current_history.append(current_score)
    temperature *= cooling_rate
```

Pengujian `delta >= 0` sebelum pemanggilan `rng.random()` berarti kandidat yang tidak lebih buruk tidak mengonsumsi bilangan acak sehingga urutan keputusan acak tetap deterministik untuk *seed* yang sama. Karena `random_neighbor` mengembalikan *state* asal ketika tidak menemukan kandidat valid, iterasi tersebut menghasilkan $\Delta E = 0$ dan *state* tidak berubah.

3.7.4 Deteksi *Stuck* pada Optimum Lokal

Spesifikasi meminta frekuensi algoritma terjebak pada optimum lokal. Implementasi mengukurnya melalui `stuck_count`, yaitu banyaknya plateau terpisah tempat nilai terbaik tidak membaik selama `stuck_patience` iterasi berturut-turut. Penghitung `stalled_iterations` direset setiap kali nilai terbaik meningkat, dan penanda `plateau_counted` mencegah satu plateau panjang dihitung berulang kali. Dengan demikian, satu plateau yang berlangsung ratusan iterasi hanya menambah `stuck_count` sebanyak satu, dan plateau baru hanya dihitung setelah ada perbaikan terbaik yang baru.

3.7.5 Metrik dan Hasil

Setelah iterasi selesai, algoritma mengemas *state* terbaik dan metrik ke dalam `SearchResult`. `SimulatedAnnealingMetrics` memuat jumlah iterasi, `stuck_count` beserta ambangnya, riwayat nilai *state* saat ini, riwayat temperatur, dan riwayat $e^{\Delta E/T}$ (peluang penerimaan) pada setiap iterasi. Riwayat objektif pada `objective_history` menyimpan nilai terbaik sejauh ini, dengan elemen pertama berupa nilai *state* awal, sedangkan riwayat nilai *state* saat ini disimpan terpisah agar dapat memperlihatkan langkah yang memburuk. Karena `final_state` adalah *state* terbaik yang pernah dijumpai, nilainya tidak pernah lebih rendah daripada nilai *state* awal.

Kode 3.19: Mengemas solusi dan metrik hasil pencarian SA

```
return SearchResult(
    algorithm="simulated_annealing",
    objective_name=objective_name or ("total_value" if objective_fn is None else "custom"),
    initial_state=initial_state,
    final_state=best,
    objective_value=best_score,
    objective_history=tuple(objective_history),
    duration_seconds=perf_counter() - started_at,
    metrics=SimulatedAnnealingMetrics(
        iterations=max_iterations,
        exp_delta_e_over_temperature=tuple(acceptance_history),
        stuck_count=stuck_count,
        stuck_patience=stuck_patience,
        current_objective_history=tuple(current_history),
        temperature_history=tuple(temperature_history),
    ),
    seed=seed,
    run_index=run_index,
    parameters={
        "max_iterations": max_iterations,
        "initial_temperature": initial_temperature,
        "cooling_rate": cooling_rate,
        "stuck_patience": stuck_patience,
    },
)
```

3.7.6 Pelaksana Eksperimen dan Visualisasi

Berkas `src/experiments/simulated_annealing.py` menjalankan SA satu kali untuk setiap *seed* sehingga tiga *seed* menghasilkan tiga *run* per konfigurasi. *State* awal setiap *run* dibangkitkan dengan `random_initial_state` dari *seed* yang diturunkan melalui operasi XOR dengan `0x5EED`, turunan yang sama dengan pelaksana GA sehingga kedua algoritma dapat dimulai dari *state* yang identik. Untuk setiap *run*, program menyimpan `result.json`, plot riwayat nilai objektif, plot $e^{\Delta E/T}$ terhadap iterasi, dan visualisasi tiga dimensi *state* awal dan akhir. Ringkasan lintas *run* (rata-rata dan simpangan baku nilai akhir, durasi, dan `stuck_count`) disimpan pada `summary.json` dan `summary-runs.png`. Fungsi `compare_objectives` dapat mengulang eksperimen untuk beberapa fungsi objektif dengan *seed* dan parameter yang sama, lalu menulis perbandingannya. Program dijalankan melalui `src/experiments/run_sa.py`.

3.8 Implementasi Genetic Algorithm

Implementasi Genetic Algorithm pada proyek ini menyimpan tiap individu sebagai objek `State`. Karena itu, satu gen berkorespondensi dengan penempatan satu paket, termasuk status paket di luar truk. Alur algoritma dipisah ke beberapa modul agar pembentukan populasi, seleksi, crossover, mutasi, dan loop generasi dapat dijelaskan serta diuji secara terpisah. Uraian berikut mengikuti urutan eksekusi `genetic_algorithm`.

3.8.1 Konfigurasi dan Generator Acak

Pemanggil menyediakan state awal, seed, ukuran populasi, dan jumlah generasi. Laju mutasi secara default adalah 0,05, seleksi default menggunakan *roulette wheel*, dan crossover diaktifkan. Seed dipakai untuk menginisialisasi satu generator acak yang digunakan bersama oleh seluruh tahap sehingga run dengan masukan dan seed yang sama dapat direproduksi. Fungsi objektif dapat menggunakan metode `State.objective` atau fungsi objektif lain yang diberikan pemanggil.

Kode 3.20: Menyiapkan pencarian GA

```
rng = Random(seed)
objective = resolve_objective(initial_state, objective_fn)
population = initialize_population(initial_state, population_size, rng)
```

3.8.2 Inisialisasi Populasi

Populasi pertama harus berisi state valid dan selalu menyertakan `initial_state` sebagai individu pertama. Anggota lainnya dibentuk menggunakan generator state awal acak yang dijelaskan sebelumnya. Dengan cara ini, algoritma memulai pencarian dari solusi yang diberikan sekaligus mempertimbangkan konfigurasi alternatif. Fungsi `initialize_population` memastikan ukuran populasi positif dan menolak state awal yang melanggar constraint.

Kode 3.21: Membangun populasi awal

```
def initialize_population(initial_state, population_size, rng):
    if type(population_size) is not int or population_size < 1:
        raise ValueError("population_size must be a positive integer")
    violations = validate_state(initial_state)
    if violations:
        raise ValueError(f"initial_state is invalid: {'; '.join(violations)}")

    return (
        initial_state,
        *(
            random_initial_state(initial_state.packages, initial_state.truck, rng)
            for _ in range(population_size - 1)
        ),
    )
```

3.8.3 Evaluasi *Fitness* dan Penyimpanan Individu Terbaik

Pada awal setiap generasi, nilai objektif setiap individu dihitung sebagai *fitness*. Implementasi juga memastikan setiap nilai berupa bilangan hingga. Individu dengan nilai tertinggi pada generasi tersebut dicatat, begitu pula rata-rata fitness populasi. Di luar populasi yang sedang berjalan, `best_state` dan `best_score` menyimpan solusi terbaik yang pernah ditemukan sehingga solusi tersebut tetap dapat dikembalikan walaupun tidak terpilih menjadi induk atau tidak muncul pada generasi berikutnya.

Kode 3.22: Evaluasi populasi dan pembaruan solusi terbaik

```
fitnesses = _evaluate_population(population, objective)
generation_best_index = max(
    range(len(population)), key=fitnesses.__getitem__
)
generation_best_score = fitnesses[generation_best_index]
best_history.append(generation_best_score)
average_history.append(sum(fitnesses) / len(fitnesses))

if best_score is None or generation_best_score > best_score:
    best_state = population[generation_best_index]
    best_score = generation_best_score
```

3.8.4 Seleksi Induk dengan *Roulette Wheel*

Untuk menghasilkan anak, program memilih dua induk dari populasi saat ini. Pada konfigurasi standar, seleksi menggunakan *roulette wheel*. Individu dengan fitness lebih tinggi mendapat peluang lebih besar untuk dipilih, tetapi individu lain tetap berpeluang terpilih. Karena fungsi objektif pemuatan paket bernilai nonnegatif, nilainya langsung digunakan sebagai bobot. Implementasi juga menangani fitness negatif dengan menggeser bobot, dan memilih secara seragam apabila semua bobot bernilai nol. Mode seleksi seragam tersedia sebagai opsi eksperimen.

Kode 3.23: Pemilihan induk secara roulette

```
minimum = min(fitnesses)
weights = [fitness - minimum if minimum < 0 else fitness
           for fitness in fitnesses]
if sum(weights) == 0:
    return rng.choice(population)
return rng.choices(population, weights=weights, k=1)[0]
```

3.8.5 Crossover Satu Titik dan Perbaikan Kelayakan

Setelah dua induk dipilih, *single-point crossover* mengambil gen sebelum titik potong dari induk pertama dan gen setelahnya dari induk kedua. Urutan gen mengikuti urutan paket dalam state. Anak mula-mula dibuat dengan semua paket di luar truk, lalu paket-paket yang diwarisi dalam keadaan termuat dicoba satu per satu dari ketinggian terendah. Jika penempatan warisan membuat anak melanggar constraint, program mencoba mencari penempatan valid baru untuk paket itu. Jika pencarian perbaikan gagal, paket tetap di luar truk. Dengan demikian, anak yang dihasilkan tetap layak.

Kode 3.24: Mengambil gen dari dua induk pada satu titik potong

```
cut = rng.randrange(1, len(packages)) if len(packages) > 1 else 1
inherited = {
    package.id: (
        parent_a.placements[package.id]
        if index < cut
        else parent_b.placements[package.id]
    )
    for index, package in enumerate(packages)
}
```

Cuplikan tersebut menentukan pewarisan gen. Tahap berikutnya membangun state anak dan memvalidasi setiap penempatan yang diwarisi. Kode di bawah menunjukkan penanganan gen yang tidak dapat langsung dipasang secara valid.

Kode 3.25: Mempertahankan validitas anak hasil crossover

```
candidate = State(packages, parent_a.truck, placements)
if not validate_state(candidate):
    child = candidate
else:
    repaired = random_valid_placement(child, package, rng)
    if repaired is not None:
        placements = dict(child.placements)
        placements[package.id] = repaired
        child = State(packages, parent_a.truck, placements)
```

3.8.6 Mutasi Anak

Setiap anak kemudian mengalami mutasi dengan peluang `mutation_rate`, yang secara default bernilai 0,05. Artinya, peluang tersebut diterapkan pada setiap anak yang dibentuk, bukan secara terpisah pada setiap paket. Jika mutasi terpilih, satu paket dipilih secara acak. Paket yang belum dimuat dicoba untuk dimasukkan. Paket yang sudah dimuat dapat dipindahkan, dirotasi, atau dikeluarkan. Kandidat mutasi hanya diterima jika berbeda dari penempatan lama dan seluruh state tetap valid. Fungsi mencoba paling banyak 50 kandidat. Jika tidak menemukan mutasi valid, individu tidak berubah.

Kode 3.26: Peluang mutasi pada setiap anak

```
if rng.random() < mutation_rate:
    child = mutate_individual(child, rng)
next_population.append(child)
```

Kode 3.27: Memilih satu paket dan jenis mutasinya

```
package = rng.choice(individual.packages)
old = individual.placements[package.id]
operations = ("insert",) if old is None else ("move", "rotate", "remove")
```

```
for _ in range(attempts):
    operation = rng.choice(operations)
    if operation == "remove":
        placement = None
    elif operation == "rotate":
        assert old is not None
        placement = random_valid_placement(
            individual, package, rng,
            orientation=rng.choice(tuple(o for o in ORIENTATIONS if o != old.orientation)),
            coordinate=old.coordinate, attempts=1,
        )
    else:
        placement = random_valid_placement(
            individual, package, rng,
            orientation=old.orientation if old is not None else None,
            attempts=1,
        )
    if placement == old:
        continue
    placements = dict(individual.placements)
    placements[package.id] = placement
    candidate = State(individual.packages, individual.truck, placements)
    if not validate_state(candidate):
        return candidate
return individual
```

3.8.7 Pembentukan Generasi Berikutnya dan Terminasi

Anak-anak dibuat berulang kali sampai jumlahnya sama dengan `population_size`. Populasi baru kemudian menggantikan populasi lama secara generasional. Individu terbaik tidak otomatis disalin ke generasi berikutnya, tetapi solusi terbaik global tetap tersimpan dalam `best_state`. Loop mengevaluasi generasi awal bernomor nol hingga generasi ke-G sehingga menghasilkan $G+1$ catatan fitness. Setelah generasi terakhir dievaluasi, pencarian berhenti dan hasil akhirnya memuat state dengan fitness terbaik yang pernah dijumpai, riwayat fitness terbaik dan rata-rata per generasi, durasi, seed, serta parameter pencarian.

Kode 3.28: Loop generasi dan pembentukan populasi baru

```
best_history = []
average_history = []
best_state = initial_state
best_score = None

for generation in range(generations + 1):
    fitnesses = _evaluate_population(population, objective)
    best_index = max(range(len(population)), key=fitnesses.__getitem__)
    generation_best = fitnesses[best_index]
    best_history.append(generation_best)
    average_history.append(sum(fitnesses) / len(fitnesses))
    if best_score is None or generation_best > best_score:
        best_state = population[best_index]
        best_score = generation_best
    if generation == generations:
        break

    next_population = []
    while len(next_population) < population_size:
        if selection_mode == "uniform":
            parent_a = rng.choice(population)
            parent_b = rng.choice(population)
        else:
            parent_a = roulette_wheel_select(population, fitnesses, rng)
            parent_b = roulette_wheel_select(population, fitnesses, rng)
        child = (single_point_crossover(parent_a, parent_b, rng)
                 if crossover_enabled else parent_a)
        if rng.random() < mutation_rate:
            child = mutate_individual(child, rng)
        next_population.append(child)
    population = tuple(next_population)
```

Setelah generasi terakhir dievaluasi, algoritma mengemas solusi terbaik dan metrik pencarian ke dalam objek `SearchResult`. Seed, indeks run, dan parameter pencarian turut dicatat agar hasil eksperimen dapat ditelusuri.

Kode 3.29: Mengemas solusi dan metrik hasil pencarian

```
return SearchResult(
    algorithm="genetic_algorithm",
    objective_name=objective_name or ("total_value" if objective_fn is None else "custom"),
    initial_state=initial_state,
    final_state=best_state,
    objective_value=best_score,
    objective_history=tuple(best_history),
    duration_seconds=perf_counter() - started_at,
    metrics=GeneticAlgorithmMetrics(
        generations=generations,
        population_size=population_size,
        best_objective_history=tuple(best_history),
        average_objective_history=tuple(average_history),
    ),
    seed=seed,
    run_index=run_index,
    parameters={
        "population_size": population_size,
        "generations": generations,
        "mutation_rate": mutation_rate,
        "selection_mode": selection_mode,
        "crossover_enabled": crossover_enabled,
    },
)
```

Seed acak diberikan pada generator `Random` sehingga konfigurasi seed yang sama dapat mereproduksi urutan keputusan acak yang sama. Sebagai catatan implementasi, riwayat fitness terbaik menyimpan nilai terbaik pada masing-masing generasi, sedangkan `final_state` menyimpan individu terbaik sepanjang seluruh generasi. Karena tidak ada elitisme, nilai terbaik per generasi dapat turun meskipun solusi terbaik global yang dikembalikan tetap dipertahankan.

3.9 Implementasi Spesifikasi Bonus

3.9.1 *Extra* Objective Function

Dua *objective function* tambahan diimplementasikan pada `src/objectives.py`. Keduanya hanya memakai variabel masalah yang sudah ada (nilai, ETA, penempatan, dan ukuran truk), tidak menambah data baru, dan didaftarkan bersama `total_value` pada kamus `OBJECTIVES` sehingga dapat dipilih dengan nama dari runner eksperimen.

- `eta_weighted_value` $= \sum_{i \in \text{truk}} \dfrac{v_i}{e_i + 1}$, dengan v_i nilai dan e_i ETA paket i. Paket yang mendesak (e_i kecil) memberi kontribusi lebih besar. Paket dengan ETA 0 bernilai penuh, sedangkan paket dengan ETA 3 hanya seperempatnya.
- `compact_value` $=$ total_value $+ 0{,}99 \cdot \left(1 - \dfrac{\sum_i (z_i + h_i)\, V_i}{H \sum_i V_i}\right)$, dengan z_i koordinat vertikal, h_i tinggi terorientasi, V_i volume paket, dan H tinggi truk. Rasio di dalam kurung adalah rata-rata tinggi puncak paket relatif terhadap tinggi truk, dengan paket bervolume besar memberi bobot lebih besar. Jadi, semakin rendah tumpukan, semakin kecil rasio tersebut dan semakin besar bonus kekompakannya. Suku kedua selalu kurang dari satu satuan nilai sehingga total nilai paket tetap menjadi penentu utama. Bonus hanya memecah seri ketika total nilai sama dan membuat perubahan posisi atau rotasi yang mempertahankan paket tetap terlihat oleh algoritma.

Kode 3.30: Fungsi *objective* tambahan pada `objectives.py`

```
_COMPACTNESS_SCALE = 0.99
```

```python
def eta_weighted_value(state: State) -> float:
    return sum(
        package.value / (package.eta + 1)
        for package in state.packages
        if state.placements[package.id] is not None
    )


def compact_value(state: State) -> float:
    return state.objective() + _COMPACTNESS_SCALE * compactness(state)


OBJECTIVES = {
    "total_value": total_value,
    "eta_weighted_value": eta_weighted_value,
    "compact_value": compact_value,
}
```

Fungsi `compactness` menghitung satu dikurangi rata-rata tinggi puncak paket yang dibobot volume dan dinormalisasi terhadap tinggi truk sehingga bernilai dalam $[0, 1]$ dan bernilai 0 untuk truk kosong. Setiap algoritma menerima *objective* melalui parameter `objective_fn` pada kontrak yang sama, dan `src/experiments/common/objective_comparison.py` menyediakan fungsi bersama untuk memilih *objective* berdasarkan nama serta menilai ulang setiap *state* akhir dengan `total_value` agar perbandingan antar-*objective* adil.

3.9.2 All-variant Hill Climbing Algorithm

Spesifikasi bonus meminta seluruh varian *hill climbing*. Implementasi mencakup empat varian, yaitu *steepest ascent*, *sideways move*, *stochastic*, dan *random restart*. Keempatnya berbagi satu fungsi `hill_climbing` yang memilih perilaku melalui parameter `variant`, dan satu fungsi pendakian `_climb`. Dengan desain ini, perbedaan hasil antar-varian hanya berasal dari aturan penerimaan *neighbor*, kondisi berhenti, dan pengulangan pendakian, bukan dari perbedaan operator *neighbor*, *objective*, atau cara pencatatan. Kode lengkap ada pada Kode 3.7 sampai Kode 3.16; subbagian ini merinci bagian yang membedakan varian.

Aturan Tiap Varian pada `_climb`

Pada setiap iterasi, `_climb` membangkitkan kandidat, menghitung nilainya, lalu memilih kandidat terbaik (`best`). Kandidat dibangkitkan sebanyak `neighbors_per_iteration` untuk *steepest*, *sideways*, dan *random restart*, tetapi hanya satu untuk *stochastic*. Keputusan berikutnya ditentukan oleh variannya.

Varian	Kandidat per iterasi	Aturan penerimaan	Kondisi berhenti
Steepest	`neighbors_per_iteration`	Pindah jika `best_score` > nilai saat ini	Kandidat terbaik tidak lebih baik, atau `max_iterations` tercapai
Sideways	`neighbors_per_iteration`	Seperti *steepest*; jika sama, pindah selama `sideways` < `max_sideways`	Kandidat terbaik lebih buruk, atau jatah *sideways* habis, atau `max_iterations` tercapai
Stochastic	1	Pindah hanya jika kandidat lebih baik; jika tidak, *state* tetap	Selalu menjalankan `max_iterations` iterasi
Random restart	`neighbors_per_iteration`	Pendakian *steepest*	`max_restarts` + 1 pendakian selesai

Tabel 3.3: Aturan penerimaan dan kondisi berhenti tiap varian *hill climbing*

Aturan tersebut diwujudkan oleh satu rantai kondisi pada akhir tiap iterasi `_climb`. Potongan berikut adalah inti yang membedakan keempat varian (kode lengkapnya pada Kode 3.11).

Kode 3.31: Inti perbedaan antar-varian pada `_climb`

```python
if best_score > current_score:
```

```
    current, current_score = best, best_score
elif variant == "sideways" and best_score == current_score and sideways < max_sideways:
    current = best
    sideways += 1
elif variant == "stochastic":
    iterations += 1
    history.append(current_score)
    continue
else:
    break
iterations += 1
history.append(current_score)
```

Cabang pertama berlaku untuk semua varian (perbaikan selalu diterima). Cabang kedua hanya aktif untuk *sideways*: kandidat bernilai sama diterima dan dihitung pada `sideways`. Cabang ketiga khusus *stochastic*: kandidat yang tidak lebih baik ditolak, tetapi iterasi tetap dihitung dan nilai saat ini dicatat ulang pada riwayat, sehingga riwayat selalu berpanjang `max_iterations` + 1. Cabang terakhir menghentikan pendakian, yaitu pada *local optimum* untuk *steepest* dan *random restart*, atau saat jatah *sideways* habis untuk *sideways*.

Penanganan *Random Restart*

Random restart tidak mempunyai aturan penerimaan sendiri. `hill_climbing` memanggil `_climb` dengan `variant="steepest"` sebanyak `max_restarts` + 1 kali: pendakian pertama dimulai dari *state* awal yang diberikan, dan pendakian berikutnya dari *state* acak baru yang dibangkitkan `problem.random_state`. Hasil terbaik dari seluruh pendakian dikembalikan.

Kodenya ada pada Kode 3.13.

Ada tiga konsekuensi pencatatan. Pertama, `objective_history` adalah gabungan riwayat semua pendakian (`history.extend`), sehingga kurvanya turun pada titik awal tiap *restart* dan tidak monoton. Kedua, `iterations` adalah jumlah iterasi seluruh pendakian, sedangkan iterasi per pendakian disimpan pada `iterations_per_resta` Ketiga, `sideways_moves` selalu 0 karena pendakian memakai aturan *steepest*. Nilai akhir yang dikembalikan adalah nilai terbaik dari seluruh pendakian, sehingga sama dengan nilai maksimum pada `objective_history` (diperiksa pada `test_random_restart_metrics`).

Pemilihan Varian dan Pengaturan Pelaksanaan

Varian dapat dipilih lewat parameter `variant`, atau lewat fungsi pembungkus `steepest_ascent_hill_climbing`, `sideways_hill_climbing`, `stochastic_hill_climbing`, dan `random_restart_hill_climbing` (Kode 3.16). Parameter yang dicatat pada `SearchResult.parameters` hanya yang relevan bagi varian tersebut: `neighbors_per_iteratio` tidak dicatat untuk *stochastic*, `max_sideways` hanya untuk *sideways*, dan `max_restarts` hanya untuk *random restart*. Pelaksana eksperimen memanggil seluruh varian secara berurutan dengan *seed* dan *state* awal yang sama:

Kode 3.32: Perulangan varian dan *seed* pada `run_hc_experiments`

```
for variant in variants:
    for run_index, seed in enumerate(seeds, start=1):
        initial_seed = seed ^ 0x5EED
        initial_state = random_initial_state(packages, truck, Random(initial_seed))
        result = hill_climbing(
            initial_state,
            seed=seed,
            variant=variant,
            neighbors_per_iteration=neighbors_per_iteration,
            max_iterations=max_iterations,
            max_sideways=max_sideways,
            max_restarts=max_restarts,
            objective_fn=objective_fn,
            objective_name=objective_name,
            run_index=run_index,
        )
```

Pada baris perintah, `-seeds` menerima tepat tiga nilai, `-variants` memilih himpunan varian (bawaan: keempatnya) dan `-objectives` memilih satu atau beberapa *objective function*. Jika lebih dari satu *objective* diberikan, `compare_hc_objectives` menjalankan `run_hc_experiments` untuk tiap *objective* dengan *seed* dan parameter yang sama, lalu menulis perbandingan lintas *objective*. Contoh:

Kode 3.33: Menjalankan seluruh varian pada ketiga *objective*

```
python -m src.experiments.run_hill_climbing \
    --input tests/fixtures/base.json --seeds 11 12 13 \
    --variants steepest sideways stochastic random_restart \
    --objectives total_value eta_weighted_value compact_value
```

Hill climbing juga dapat dijalankan pada armada multi-truk melalui `src/experiments/fleet/run_hc.py`, yang menerima `-variants` dan `-objectives` yang sama. Tidak ada perubahan pada `_climb` untuk keperluan ini karena `search_problem_for_state` menyediakan operasi armada melalui `SearchProblem`.

Catatan Implementasi

- ***Neighbor* yang tidak mengubah *state*.** `random_neighbor` mengembalikan *state* asal bila tidak menemukan kandidat valid dalam `max_attempts` (bawaan 100) percobaan. Kandidat seperti ini bernilai sama dengan *state* saat ini, sehingga pada varian *sideways* ia dapat terpilih dan dihitung sebagai *sideways move* walaupun tidak menjelajahi *state* baru. Akibatnya jatah `max_sideways` dapat terpakai tanpa kemajuan nyata.
- **Pemilihan kandidat terbaik.** `max` mengembalikan kandidat pertama yang bernilai maksimum, sehingga bila beberapa kandidat sama baiknya, yang dibangkitkan lebih awal dipilih.
- **Sampel *neighbor*.** *Steepest*, *sideways*, dan *random restart* hanya mengevaluasi sampel acak (bawaan 30) dari ruang *neighbor*, bukan seluruhnya. Berhenti pada *steepest* berarti tidak ada perbaikan di dalam sampel, bukan bukti bahwa *state* adalah *local optimum* sejati.

Pengujian

Pengujian pada `tests/algorithms/test_hill_climbing.py` memeriksa bahwa setiap varian menghasilkan *state* valid yang tidak lebih buruk daripada *state* awal, bahwa *seed* yang sama memberi riwayat *objective* yang sama, bahwa riwayat tidak menurun dalam satu pendakian (*steepest*, *sideways*, *stochastic*), bahwa `sideways_moves` tidak melebihi `max_sideways`, bahwa metrik *random restart* konsisten, dan bahwa *stochastic* memakai seluruh `max_iterations`. Tes lain memeriksa penolakan argumen tidak valid, penerimaan `SearchProblem` armada, penolakan operator *neighbor* yang menghasilkan *state* tidak valid, dan pembuatan berkas keluaran oleh pelaksana eksperimen.

3.9.3 Multi-Truck

Representasi *State* Armada

Representasi satu truk tidak cukup untuk memodelkan penugasan paket ke beberapa truk. Karena itu, implementasi menggunakan `FleetState`, yang menyimpan kumpulan paket dan truk bersama satu peta penempatan untuk setiap truk. Untuk n paket dan m truk, misalkan $I = \{1, \ldots, n\}$ adalah indeks paket dan $J = \{1, \ldots, m\}$ adalah indeks truk. State armada ditulis sebagai

$$S_F = (P, (T_j)_{j \in J}, (\phi_j)_{j \in J}), \qquad \phi_j : I \longrightarrow \{\perp\} \cup \mathcal{P}_j, \tag{3.7}$$

Di sini, P adalah kumpulan paket, T_j adalah truk ke-j, dan $\mathcal{P}_j$ adalah himpunan penempatan yang mungkin pada truk tersebut. Nilai $\phi_j(i) = \perp$ berarti paket i tidak ditempatkan pada truk j. Jika nilainya bukan $\perp$, nilai tersebut memuat koordinat, dimensi setelah rotasi, dan orientasi paket. Implementasi menyimpan ϕ_j sebagai kamus ID paket ke objek `Placement | None`. Kumpulan kamus ini berada pada `FleetState.placements` dan berpasangan dengan tuple `trucks`.

Setiap paket boleh tidak dimuat atau ditempatkan pada tepat satu truk. Dengan fungsi indikator $\mathbf{1}[\cdot]$, batasan keunikan assignment dirumuskan sebagai

$$\sum_{j \in J} \mathbf{1}[\phi_j(i) \neq \perp] \leq 1, \qquad \forall i \in I. \tag{3.8}$$

Dengan representasi ini, paket yang tidak memperoleh penempatan valid tetap tercatat pada kumpulan P, tetapi seluruh entri penempatannya bernilai $\perp$. Cuplikan 3.34 menunjukkan struktur data yang digunakan.

Kode 3.34: Representasi state armada pada `model.py`

```
@dataclass(frozen=True, slots=True)
class FleetState:
    packages: tuple[Package, ...]
    trucks: tuple[Truck, ...]
    placements: tuple[Mapping[str, Placement | None], ...]

    def __post_init__(self):
        if not self.trucks or len(self.placements) != len(self.trucks):
            raise ValueError("fleet needs one placement map per truck")
        for truck, placements in zip(self.trucks, self.placements):
            State(self.packages, truck, placements)
```

Konstruktor memeriksa bahwa armada tidak kosong, banyak peta penempatan sama dengan banyak truk, dan setiap peta memiliki struktur yang sesuai dengan paket. Pemeriksaan kelayakan geometris dilakukan terpisah oleh validator agar aturan state satu truk tetap dapat digunakan kembali.

Kelayakan Armada dan Fungsi Objektif

Sebuah state armada layak apabila setiap truk memenuhi constraint satu truk dan tidak ada paket yang muncul pada lebih dari satu peta penempatan. Jika $\mathcal{F}_j$ adalah himpunan state yang layak untuk truk j, himpunan state armada layak adalah

$$\mathcal{F}_F = \left\{ S_F \;\middle|\; (P, T_j, \phi_j) \in \mathcal{F}_j \;\forall j \in J, \quad \sum_{j \in J} \mathbf{1}[\phi_j(i) \neq \perp] \leq 1 \;\forall i \in I \right\}. \tag{3.9}$$

Batas dimensi, tabrakan, kapasitas berat, dan dukungan paket diperiksa pada tiap truk. Keunikan penugasan diperiksa pada tingkat armada. Implementasi menggunakan kembali `validate_state` untuk memeriksa truk, lalu menandai ID paket yang sudah ditugaskan untuk mendeteksi duplikasi.

Kode 3.35: Validasi lokal dan keunikan paket pada armada

```
for truck_index, (truck, placements) in enumerate(
    zip(state.trucks, state.placements)
):
    errors.extend(
        validate_state(State(state.packages, truck, placements))
    )
    for package_id, placement in placements.items():
        if placement is None:
            continue
        if package_id in loaded_by:
            errors.append("package assigned to more than one truck")
        else:
            loaded_by[package_id] = truck_index
```

Fungsi objektif memaksimalkan total nilai paket yang dimuat di seluruh armada. Persamaan 3.8 membatasi setiap paket pada satu penempatan sehingga nilai tiap paket hanya dijumlahkan satu kali.

$$f_F(S_F) = \sum_{i \in I} v_i \, \mathbf{1}[\exists j \in J : \phi_j(i) \neq \perp], \qquad S_F^* \in \arg\max_{S_F \in \mathcal{F}_F} f_F(S_F). \tag{3.10}$$

Implementasi `FleetState.objective()` menghitung persamaan tersebut. Objective alternatif ETA dan kekompakan juga menerima `FleetState`. Objective ETA menghitung kontribusi setiap paket yang ditempatkan pada armada, sedangkan kekompakan mengagregasi penempatan di seluruh truk.

Pembentukan *Initial State*

State awal armada dibentuk secara konstruktif dan stokastik. Mula-mula semua paket dianggap belum dimuat pada semua truk. Paket kemudian diproses menurut urutan masukan. Untuk setiap paket, urutan truk diacak. Pada setiap truk, implementasi mencoba orientasi, koordinat, dan ketinggian kandidat secara terbatas. Penempatan diterima jika state pada truk tersebut valid. Paket lalu tidak dicoba pada truk lain. Jika tidak ada penempatan yang ditemukan pada seluruh truk, paket tetap berada di luar armada.

Secara formal, untuk paket i yang sedang diproses, implementasi mengambil permutasi acak π dari J dan memilih truk pertama yang menyediakan kandidat $p \in \mathcal{P}_j$ sehingga penggantian $\phi_j(i) \leftarrow p$ mempertahankan kelayakan lokal. Pencarian kandidat penempatan dibatasi hingga 50 percobaan per truk secara bawaan. Dengan demikian, kegagalan menemukan penempatan tidak membuktikan bahwa paket tersebut mustahil dimuat. Cuplikan 3.36 merangkum proses ini.

Kode 3.36: Inisialisasi state armada secara acak

```
placements = tuple(
    {package.id: None for package in packages} for _ in trucks
)
fleet = FleetState(packages, trucks, placements)
for package in packages:
    truck_order = list(range(len(trucks)))
    rng.shuffle(truck_order)
    for truck_index in truck_order:
        placement = random_valid_placement(
            _truck_state(fleet, truck_index), package, rng
        )
        if placement is None:
            continue
        changed = list(fleet.placements)
        changed[truck_index] = dict(changed[truck_index])
        changed[truck_index][package.id] = placement
        fleet = FleetState(packages, trucks, tuple(changed))
        break
```

Strategi ini menggunakan generator acak ber-seed sehingga dapat direproduksi, dan menjaga agar setiap paket yang berhasil ditempatkan hanya masuk ke satu truk. Namun, strategi ini bukan pencarian menyeluruh dan tidak menjamin state awal memuat sebanyak mungkin paket.

Fungsi *Neighbor* untuk HC dan SA

Pada HC dan SA, satu langkah pencarian armada mengubah assignment paling banyak satu paket. Untuk S_F, himpunan kandidat konseptual dapat ditulis sebagai

$$N_F(S_F) = \{S'_F \in \mathcal{F}_F \mid |\{i \in I : \gamma_i(S_F) \neq \gamma_i(S'_F)\}| \leq 1\}, \tag{3.11}$$

Di sini, $\gamma_i(S_F)$ menyatakan penugasan paket i. Nilainya $\perp$ jika paket tidak dimuat, atau pasangan truk dan penempatannya jika paket dimuat. Implementasi tidak mengenumerasi seluruh $N_F(S_F)$. Program memilih satu paket secara acak, menentukan operasi yang sesuai, lalu menyampel kandidat valid. Jika tidak menemukan kandidat dalam batas percobaan, operator mengembalikan state asal.

Operasi bergantung pada status paket yang dipilih. Paket yang belum dimuat hanya dapat dicoba untuk dimasukkan ke salah satu truk. Paket yang telah dimuat dapat dipindahkan di truk yang sama, diputar pada koordinat yang sama, dipindahkan ke truk lain, atau dikeluarkan dari armada. Kandidat move, rotate, dan reassign diterima hanya jika valid pada truk tujuan. Validator armada memastikan batasan assignment global tetap terjaga. Operator mencoba paling banyak 50 kandidat dan mengembalikan state asal apabila semua percobaan gagal.

Kode 3.37: Pemilihan operasi mutasi/neighbor armada

```
operations = ("insert",) if old is None else (
    "move", "rotate", "reassign", "remove"
)
operation = rng.choice(operations)
if operation == "remove":
    return base

truck_order = list(range(len(base.trucks)))
if operation == "move" and old_truck is not None:
    truck_order = [old_truck]
elif operation == "reassign" and old_truck is not None:
    truck_order = [i for i in truck_order if i != old_truck]
rng.shuffle(truck_order)
```

Cuplikan tersebut diambil dari awal proses mutasi. Untuk operasi selain `remove`, implementasi mencoba penempatan pada truk dalam `truck_order`, membangun `FleetState` kandidat, lalu mengembalikannya hanya apabila berbeda dari state asal dan `validate_fleet_state` tidak menemukan pelanggaran.

Fungsi `_fleet_neighbor` pada `search_problems.py` menggunakan operator ini sebagai fungsi neighbor satu langkah. Dengan demikian, loop HC dan SA generik tetap sama. Perbedaan untuk armada berada pada adapter state dan operasi neighbor yang dipasangkan kepadanya. HC menerima kandidat menurut varian yang dipilih, sedangkan SA dapat menerima kandidat yang lebih buruk dengan probabilitas sesuai temperatur.

Operator *Genetic Algorithm* Armada

Pada GA, satu gen merepresentasikan assignment satu paket, termasuk pilihan truk dan penempatannya. Crossover satu titik memilih gen dari induk pertama sebelum titik potong dan dari induk kedua sesudahnya. Anak dimulai sebagai armada kosong. Gen yang diwarisi diproses menurut indeks truk dan ketinggian penempatan. Implementasi mula-mula mencoba mempertahankan truk, koordinat, dan orientasi yang diwarisi. Jika penempatan tersebut berbenturan dengan gen yang sudah dipasang, operator mencoba memperbaikinya pada truk asal, lalu mencoba truk lain. Gen yang tidak dapat ditempatkan secara valid dibiarkan di luar armada.

Mutasi memilih satu paket. Untuk paket yang belum dimuat, mutasi mencoba insert. Untuk paket yang sudah dimuat, mutasi dapat memilih move, rotate, reassign, atau remove. Perubahan assignment dibangun melalui state sementara dengan paket tersebut dikeluarkan terlebih dahulu, lalu kandidat penempatan diperiksa terhadap constraint truk tujuan. Desain ini membuat mutasi satu-paket tidak perlu memodifikasi gen lain secara langsung, sekaligus menjaga keunikan assignment.

Kode 3.38: Pewarisan assignment paket pada crossover fleet

```
cut = rng.randrange(1, len(packages)) if len(packages) > 1 else 1
for package_index, package in enumerate(packages):
    parent = parent_a if package_index < cut else parent_b
    inherited[package.id] = next(
        (
            (truck_index, placements[package.id])
            for truck_index, placements in enumerate(parent.placements)
            if placements[package.id] is not None
        ),
        None,
    )
```

Setelah pewarisan gen, implementasi membangun anak dari peta penempatan kosong. Untuk setiap gen termuat, urutan percobaan dimulai dari truk yang diwarisi. Jika penempatan persisnya tidak dapat dipertahankan, operator mencari posisi valid lain pada truk itu, lalu mencoba truk lain. Jika seluruh percobaan gagal, paket tetap tidak dimuat.

Loop GA sendiri memakai implementasi generik `genetic_algorithm`. Adapter armada menyediakan pembangkit populasi, crossover, mutasi, objective, dan validator. State awal yang diberikan tetap menjadi individu pertama populasi. Karena populasi generasi berikutnya dibentuk tanpa elitisme, individu terbaik tidak harus muncul pada generasi selanjutnya, tetapi loop generik menyimpan individu terbaik sepanjang pencarian untuk hasil akhir.

Kontrak Pencarian, Masukan, dan Keluaran

Pemilihan operasi berdasarkan tipe state dilakukan oleh `search_problem_for_state`. Untuk `FleetState`, adapter memasangkan objective dan validator armada, pembangkit state awal armada, neighbor berbasis mutasi, serta operator crossover dan mutasi GA. Akibatnya, algoritma HC, SA, dan GA dapat menggunakan tipe state yang sama tanpa menyalin loop pencarian masing-masing.

Masukan armada dapat memakai field JSON `trucks` berupa daftar truk dengan dimensi dan kapasitas yang dapat berbeda. Untuk kompatibilitas dengan format lama, field `truck` tunggal dapat direplikasi melalui argumen CLI `-truck-count`. Konfigurasi armada heterogen ditunjukkan pada contoh berikut.

Kode 3.39: Contoh konfigurasi truk heterogen dalam JSON

```
"trucks": [
    {"dimensions": [10, 20, 5], "max_capacity": 500},
    {"dimensions": [6, 8, 4], "max_capacity": 180}
]
```

Runner fleet berada di `src/experiments/fleet/`. Berkas JSON mencatat nilai armada, konfigurasi dan muatan truk, paket yang tidak termuat, seed, parameter, riwayat objective, serta durasi. Runner juga menyimpan visualisasi state awal dan akhir. SA turut mencatat riwayat probabilitas penerimaan. Tes pada `tests/fleet/` mencakup CLI, serialisasi, visualisasi, seed jamak, objective alternatif, dan konfigurasi truk heterogen.

Ekstensi multi-truk mempertahankan constraint pemuatan untuk setiap truk dan menambahkan constraint penugasan pada tingkat armada. Inisialisasi dan operator pencarian memakai percobaan acak yang terbatas. Karena itu, hasilnya bersifat heuristik dan tidak menjamin optimum global atau pemuatan semua paket yang sebenarnya dapat dimuat.

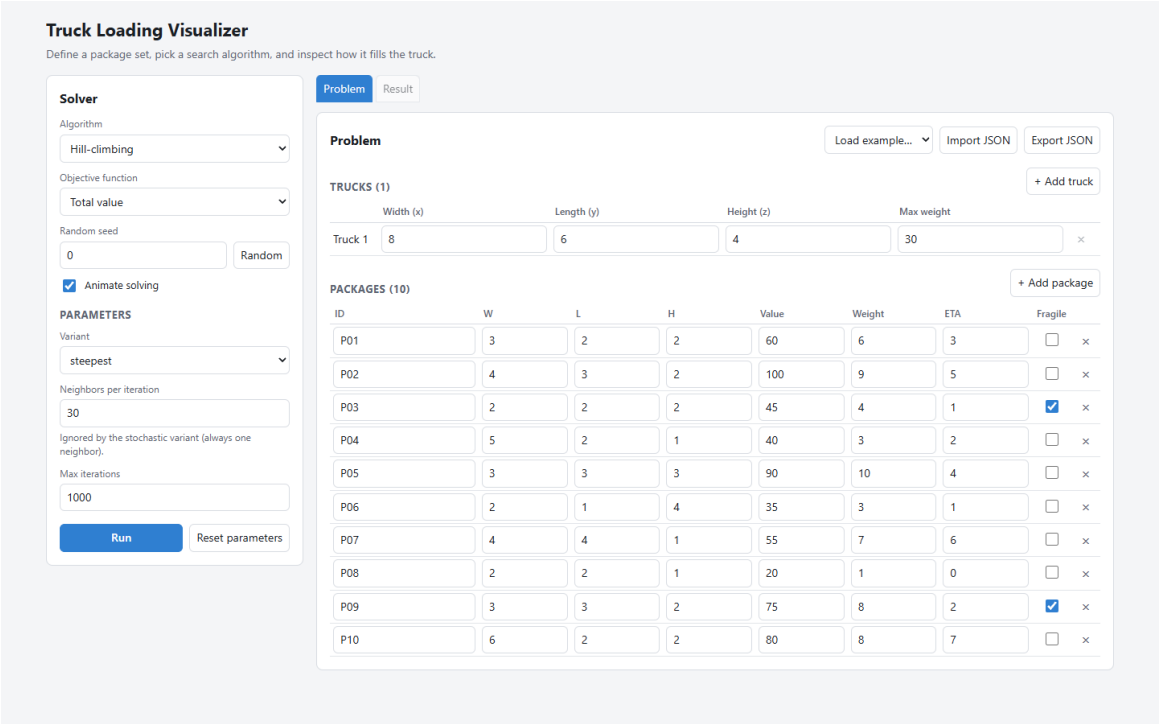
3.9.4 *Visualizer* Berbasis *Web Server*

Selain plot statis dari runner eksperimen, program menyediakan *visualizer* interaktif pada `src/visualizer/`. *Backend* berupa aplikasi FastAPI yang dijalankan dengan `python -m src.visualizer` (bawaan `http://127.0.0.1:8000`), sedangkan *frontend* dibangun dengan React, TypeScript, dan three.js di `src/visualizer/web/`. Hasil *build frontend* disajikan langsung oleh aplikasi Python sehingga cukup satu proses untuk menjalankan keduanya. Dependensinya dipisahkan pada `requirements-visualizer.txt` agar eksperimen tetap dapat dijalankan tanpa memasang *library* web.

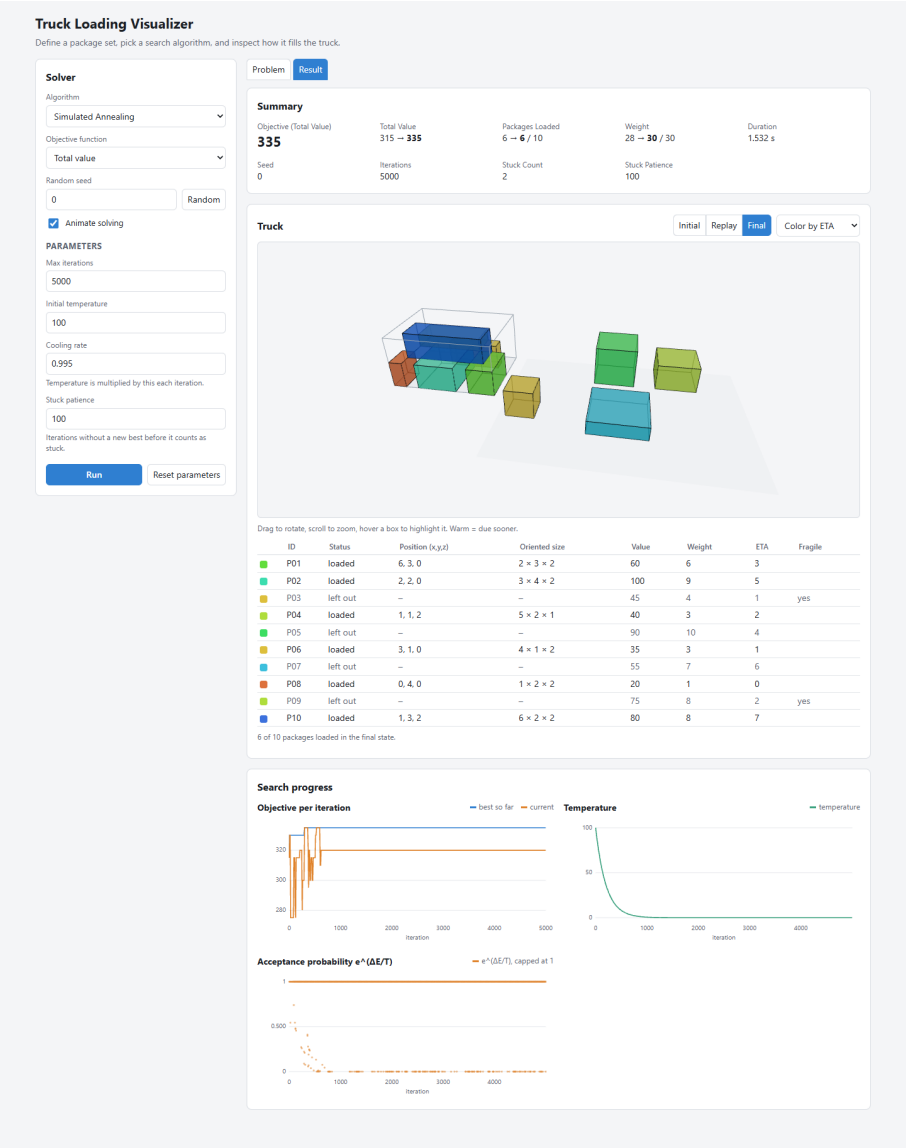
Backend menyediakan tiga *endpoint*. `GET /api/catalogue` mengembalikan daftar algoritma beserta spesifikasi parameternya, `GET /api/examples` mengembalikan *fixture* pada `tests/fixtures` yang lolos *loader* sebagai contoh masalah (ditambah varian armada `base-2-trucks` dan `hard-3-trucks`), dan `POST /api/run` menjalankan pencarian. Permintaan *run* divalidasi oleh model Pydantic pada `schemas.py`, lalu *server* membangkitkan *initial state* dari *seed* yang diberikan dan memanggil fungsi algoritma yang sama dengan yang dipakai pada eksperimen. Jika masalah memiliki satu truk, pencarian berjalan pada `State`; jika lebih, pada `FleetState` dengan operator armada. Parameter setiap algoritma didefinisikan sekali pada `registry.py`, dan formulir pada antarmuka dibangun dari definisi tersebut sehingga algoritma atau parameter baru cukup ditambahkan di satu tempat.

Antarmuka terdiri atas tiga bagian. Panel *Problem* memuat contoh atau JSON masalah serta mengubah daftar truk dan paket. Panel *Solver* memilih algoritma (keempat varian HC, SA, atau GA), *objective function*, *seed*, dan parameternya. Panel hasil menampilkan ringkasan (nilai *objective*, durasi, dan metrik algoritma), tampilan 3D truk beserta paket di dalamnya, dan grafik kemajuan pencarian: *objective* per iterasi untuk HC dan SA, suhu dan probabilitas penerimaan $e^{\Delta E/T}$ untuk SA, serta *fitness* per generasi untuk GA.

Gambar 3.1 dan Gambar 3.2 menunjukkan contoh penggunaan pada *fixture* `base` dengan SA, *seed* 0, dan parameter bawaan. Pada *run* tersebut, nilai total naik dari 315 pada *initial state* menjadi 335 pada *final state* dengan 6 dari 10 paket dimuat.



Gambar 3.1: Tab *Problem* dan panel *Solver* pada *visualizer* dengan contoh `base`



Gambar 3.2: Tab *Result*: ringkasan, tampilan 3D *final state*, dan grafik kemajuan SA

Animasi proses pencarian direkam oleh `FrameRecorder` pada `recording.py` tanpa mengubah kode algoritma. Karena setiap algoritma menilai kandidat melalui `objective_fn`, *recorder* membungkus fungsi tersebut dan mencatat setiap *state* yang memperbaiki nilai terbaik sejauh ini serta sampel merata dari kandidat lain. Masing-masing dibatasi 40 *frame*, dan *frame* terakhir selalu *state* akhir yang dikembalikan algoritma. Antarmuka memutar ulang rekaman tersebut dengan kontrol *play*/*pause*, *scrubber*, dan pilihan kecepatan; pada armada, paket yang berpindah truk dianimasikan keluar dari satu truk dan masuk ke truk lain.

Bab 4

Hasil Eksperimen dan Analisis

4.1 Metodologi Eksperimen

Metodologi eksperimen dirancang untuk membandingkan kualitas solusi dan biaya pencarian secara adil, sekaligus memeriksa ketahanan algoritma pada kasus sulit dan masukan batas. Seluruh algoritma diuji pada instance paket dan truk yang sama, menggunakan fungsi objektif serta validator constraint yang sama. Rincian konfigurasi khusus tiap algoritma dibahas terpisah dan tidak diulang pada metodologi umum ini.

4.1.1 Kumpulan Instance dan Kasus Uji

Kumpulan uji mencakup instance dasar sintetis karena spesifikasi tugas tidak menyediakan data paket. Instance terkontrol diturunkan dari kasus dasar untuk menguji karakteristik tertentu. Ukuran dan tingkat kesulitan dibuat bervariasi agar hasil mewakili beberapa konfigurasi pemuatan. Pengujian mencakup kategori berikut.

(1) **Kasus dasar.** Instance dengan paket yang bervariasi dalam dimensi, nilai, dan berat, serta beberapa kombinasi pemuatan yang layak. Kasus ini digunakan untuk membandingkan kinerja umum.

(2) **Kasus kendala ketat.** Ruang truk diperkecil atau kapasitas berat dikurangi agar lebih banyak kandidat penempatan ditolak. Instance ini menguji dampak keterbatasan ruang dan kapasitas.

(3) **Kasus tanpa pemuatan nonkosong yang layak.** Semua paket dibuat tidak dapat dimuat, misalnya setiap paket lebih besar daripada ruang truk pada semua orientasi atau beratnya melebihi kapasitas. Karena representasi tetap mengizinkan semua paket berada di luar truk, state kosong merupakan state valid dengan nilai objektif nol. Kasus ini menguji apakah algoritma dapat mengembalikan state valid tanpa paket termuat.

(4) **Kasus batas dan struktur khusus.** Pengujian meliputi satu paket, paket yang hanya muat setelah rotasi, paket yang tepat menyentuh dinding truk, berat total yang tepat sama dengan kapasitas, paket rapuh yang memengaruhi dukungan, serta paket bernilai atau berbobot nol. Kasus dengan fitness sama juga digunakan untuk memeriksa perilaku ketika seleksi tidak memiliki preferensi nilai.

(5) **Masukan tidak valid.** Data dengan dimensi atau kapasitas yang melanggar format, ID paket duplikat, atau daftar paket kosong diuji pada batas validasi data. Kasus ini dipisahkan dari benchmark pencarian karena bukan instance masalah yang sah menurut loader.

Instance sintetis untuk uji batas dibuat dengan mengubah satu karakteristik pada satu waktu dari instance dasar, sementara atribut lain dipertahankan. Dengan demikian, perbedaan hasil dapat dikaitkan dengan kendala yang sedang diuji. Kasus yang dirancang untuk memastikan tidak ada paket yang dapat dimuat dibedakan secara eksplisit dari kasus sulit yang masih memiliki solusi layak.

4.1.2 Protokol Perbandingan

Untuk setiap pasangan instance dan seed, state awal yang sama digunakan oleh semua algoritma. Daftar seed ditetapkan sebelum eksperimen. Setiap kombinasi dijalankan sedikitnya tiga kali dengan seed berbeda

sesuai kebutuhan evaluasi tugas. Jika anggaran komputasi memungkinkan, jumlah pengulangan ditambah agar estimasi variasi hasil lebih stabil. Penggunaan seed dan state awal berpasangan mengurangi pengaruh variasi inisialisasi saat membandingkan metode.

Selain jenis instance, faktor lain seperti data paket, truk, fungsi objektif, dan implementasi validasi dijaga tetap sama. Batas penghentian dan konfigurasi spesifik mengikuti definisi masing-masing algoritma. Karena satu iterasi pada algoritma berbeda belum tentu setara, jumlah iterasi tidak digunakan sendirian sebagai ukuran biaya lintas algoritma. Durasi pencarian dan jumlah evaluasi fungsi objektif dicatat sebagai ukuran biaya yang lebih langsung. Pengukuran waktu dilakukan pada lingkungan komputasi yang sama, dengan waktu persiapan data dan pembuatan visualisasi dipisahkan dari waktu pencarian.

Setiap hasil akhir diperiksa kembali menggunakan validator. Run dinyatakan berhasil jika algoritma mengembalikan state valid. Pada instance tanpa pemuatan nonkosong yang layak, state akhir harus valid dan tidak memuat paket. State awal dan akhir, seed, nilai objektif, jumlah paket termuat, riwayat pencarian, durasi, serta konfigurasi run disimpan agar hasil dapat direproduksi dan ditelusuri.

4.1.3 Metrik dan Analisis

Kualitas hasil dinilai dari nilai objektif akhir dan jumlah paket yang termuat. Ketahanan terhadap constraint dinilai dari proporsi run yang menghasilkan state akhir valid. Konsistensi diukur dengan merangkum nilai objektif dan durasi antarrun menggunakan rata-rata serta simpangan baku. Hasil tiap run tetap disajikan agar variasi tidak tersembunyi oleh agregat. Riwayat nilai objektif juga ditampilkan untuk memperlihatkan perkembangan pencarian. Untuk perbandingan lintas algoritma, kurva dapat disejajarkan terhadap jumlah evaluasi objektif atau waktu berjalan, selain menampilkan sumbu iterasi atau generasi yang diwajibkan untuk masing-masing algoritma.

Pada kasus tanpa pemuatan nonkosong yang layak, nilai objektif nol saja tidak cukup untuk menyimpulkan perilaku algoritma karena paket bernilai nol dapat saja termuat. Oleh sebab itu, jumlah paket termuat dan validitas state diperiksa bersamaan dengan nilai objektif. Hasil kasus ini dilaporkan sebagai uji ketahanan, terpisah dari perbandingan kualitas solusi pada instance yang dapat dimuat.

4.1.4 Studi Ablasi

Studi ablasi digunakan untuk mengetahui kontribusi pilihan desain terhadap hasil pencarian. Untuk setiap studi, satu konfigurasi acuan dibandingkan dengan beberapa varian konfigurasi. Pada tiap perbandingan, hanya faktor yang sedang diuji yang diubah. Instance, state awal, seed, fungsi objektif, constraint, dan anggaran pencarian lainnya dijaga tetap sama. Jika dua faktor diduga saling memengaruhi, interaksinya diuji pada eksperimen terpisah, bukan diubah sekaligus tanpa kontrol.

Setiap varian dijalankan dengan seed dan state awal yang sama seperti konfigurasi acuan. Analisis membandingkan nilai objektif akhir dan durasi serta menyajikan rata-rata, simpangan baku, dan riwayat pencarian. Dengan demikian, studi ablasi menunjukkan apakah perubahan konfigurasi memberi hasil yang konsisten pada seed yang diuji.

4.1.5 Dataset dan Konfigurasi Eksperimen

Seluruh eksperimen dijalankan dengan representasi *state*, validator *constraint*, dan fungsi *neighbor* yang sama untuk ketiga algoritma. Spesifikasi tugas tidak menyediakan data paket sehingga seluruh *instance* bersifat sintetis dan disimpan sebagai *fixture* pada `tests/fixtures/`. Setiap kategori pada subbab pertama diwujudkan oleh satu atau lebih berkas, dan sifat tiap berkas diperiksa otomatis oleh `tests/experiments/test_fixtures.py` (misalnya bahwa `tight-space` hanya berbeda dari `base` pada dimensi truk, atau bahwa tidak ada paket pada `infeasible` yang dapat dimuat sendirian).

Kategori	Berkas	Paket	Truk (kapasitas)	Karakteristik
Dasar	`base.json`	10	$8 \times 6 \times 4$ (30)	Dimensi, nilai, berat, ETA, dan kerapuhan bervariasi. Total berat 59 melebihi kapasitas sehingga harus dipilih himpunan bagian
Kendala ketat	`tight-space.json`	10	$6 \times 4 \times 3$ (30)	Paket dan kapasitas sama dengan `base`. Hanya dimensi truk diperkecil
	`tight-capacity.json`	10	$8 \times 6 \times 4$ (15)	Paket dan dimensi sama dengan `base`. Hanya kapasitas dikurangi
Tanpa pemuatan layak	`infeasible.json`	10	$8 \times 6 \times 4$ (30)	Perubahan dari `base`. Paket P01–P05 diberi satu sisi 9, melebihi sisi terpanjang truk. Paket P06–P10 diberi berat 31, melebihi kapasitas
Batas dan struktur khusus	`boundary/single-package`	1	$4 \times 4 \times 4$ (10)	Satu paket yang muat
	`boundary/rotation-only`	2	$5 \times 2 \times 2$ (10)	Paket $2 \times 2 \times 5$ hanya muat setelah rotasi
	`boundary/wall-touching`	4	$4 \times 4 \times 2$ (8)	Empat paket $2 \times 2 \times 2$ mengisi truk tepat sampai dinding
	`boundary/exact-capacity`	4	$10 \times 10 \times 10$ (10)	Optimum (berat $4+3+3$) tepat sama dengan kapasitas
	`boundary/fragile-support`	3	$2 \times 2 \times 2$ (10)	Paket rapuh boleh berada di atas paket biasa, tetapi tidak boleh menopang
	`boundary/zero-value-weight`	4	$4 \times 4 \times 4$ (5)	Paket bernilai nol, berbobot nol, dan keduanya nol
	`boundary/equal-fitness`	6	$2 \times 2 \times 1$ (100)	Enam paket identik. Seleksi tidak punya preferensi
Masukan tidak valid	`invalid/` (8 berkas)	—	—	Dimensi paket tidak positif, terlalu panjang, atau bukan bilangan bulat. Dimensi truk tidak positif. Kapasitas negatif. ID ganda. Daftar paket kosong. Atribut hilang

Tabel 4.1: *Fixture* untuk tiap kategori kasus uji

Eksperimen memakai tiga fungsi objektif. `total_value` adalah fungsi wajib. `eta_weighted_value` membagi nilai paket dengan ETA + 1 sehingga paket yang mendesak mendapat bobot lebih besar. `compact_value` menambahkan bonus kurang dari satu satuan untuk penataan yang lebih rendah. Pengujian dibagi menjadi lima rangkaian.

(1) ***Benchmark* lintas kategori** (`src/experiments/run_benchmark.py`). Seluruh algoritma diuji pada 11 *fixture* valid dengan `total_value`, lima *seed* (11–15), dan satu konfigurasi parameter tetap sehingga 11 instance $\times$ 5 *seed* $\times$ 6 algoritma = 330 *run*. Setiap *run* mencatat validitas *state* akhir, jumlah paket termuat, jumlah evaluasi objektif, dan durasi pencarian. Runner yang sama memberikan delapan *fixture* tidak valid ke *loader* data.

(2) **Perbandingan fungsi objektif per algoritma** pada `base` dengan tiga fungsi objektif dan tiga *seed* (11–13). Rangkaian ini mencakup empat varian *hill climbing*, tiga *run* SA per fungsi objektif, serta dua *sweep* GA untuk jumlah generasi dan ukuran populasi. Perbandingan fungsi objektif juga diulang pada `tight-capacity` dengan konfigurasi yang sama. Runner yang digunakan adalah `src/experiments/run_hill_climbing.py`, `src/experiments/run_sa.py`, dan `src/experiments/run_ga.py`.

(3) **Analisis SA lintas fixture dan parameter** dengan `total_value` (`src/experiments/sa/results_analysis.py`). Baseline dijalankan pada 11 *fixture* valid. Tiga *sweep* satu faktor dilakukan pada `base` dengan lima *seed* (11–15). Baseline memakai parameter SA pada Tabel 4.2. Parameter yang disapu adalah suhu awal $T_0 \in \{10, 100, 1000\}$, laju pendinginan 0,99, 0,995, dan 0,999, serta jumlah iterasi 1000, 3000, dan 6000.

(4) **Analisis GA lintas fixture dan parameter** dengan `total_value` (`src/experiments/ga/results_analysis.py`). Baseline dijalankan pada 11 *fixture* valid. Tiga *sweep* satu faktor dilakukan pada `base` dengan lima *seed* (11–15). Baseline memakai populasi 20, 40 generasi, dan laju mutasi 0,05. Parameter yang disapu adalah ukuran populasi 10, 20, dan 40, jumlah generasi 20, 40, dan 80, serta laju mutasi 0, 0,05, dan 0,20.

(5) **Studi ablasi GA** pada `base` dengan `total_value` dan lima *seed* (11–15) (`src/experiments/run_ga_ablation.py`). Empat varian yang dibandingkan adalah baseline, tanpa mutasi, tanpa *crossover*, dan seleksi *uniform*.

Algoritma	Parameter
Hill climbing	4 varian dengan `neighbors_per_iteration` = 30, `max_iterations` = 1000, `max_sideways` = 100, dan `max_restarts` = 10
Simulated annealing	Baseline memakai `max_iterations` = 3000, T_0 = 100, `cooling_rate` = 0,995, dan `stuck_patience` = 100. *Sweep* terpisah menguji $T_0 \in \{10, 100, 1000\}$, `cooling_rate` {0,99, 0,995, 0,999}, dan `max_iterations` $\{1000, 3000, 6000\}$, dengan parameter lain tetap pada baseline
Genetic algorithm	Baseline memakai populasi 20, 40 generasi, dan *mutation rate* 0,05. *Sweep* terpisah menguji populasi $\{10, 20, 40\}$, generasi $\{20, 40, 80\}$, dan *mutation rate* {0, 0,05, 0,20}. Parameter lain tetap pada baseline dan seleksi memakai *roulette-wheel*. Ablasi memakai populasi 20 dan 50 generasi

Tabel 4.2: Konfigurasi parameter eksperimen

State awal setiap *run* dibangkitkan secara acak dari *seed* yang sama (*seed* dikenai XOR dengan konstanta tetap) sehingga ketiga algoritma dan ketiga *objective* memulai dari *state* yang identik. Pada `base`, nilai total *state* awal untuk *seed* 11 sampai 15 berturut-turut adalah 330, 285, 320, 275, dan 275. Pada `tight-capacity` untuk *seed* 11 sampai 13 adalah 155, 155, dan 160. Pada GA, *state* awal menjadi anggota pertama populasi. Karena hanya ada tiga atau lima *seed*, simpangan baku (simpangan baku sampel, dihitung ulang dari nilai tiap *run*) harus dibaca sebagai gambaran sebaran kasar, bukan hasil uji statistik.

Nilai optimum sebagai acuan. Relaksasi yang hanya memperhatikan kapasitas berat (masalah *knapsack*, dihitung dengan enumerasi) memberi batas atas 335 untuk `base` dan 175 untuk `tight-capacity`. Kedua batas ini tercapai oleh *run* yang valid sehingga 335 dan 175 adalah optimum sebenarnya. Pada `tight-space` batas atas yang sama (335) tidak tercapai oleh *run* mana pun. Nilai terbaik yang ditemukan adalah 300, dan apakah itu optimum tidak diketahui. Pada tabel berikutnya, "terbaik" berarti nilai tertinggi yang ditemukan oleh seluruh *run* pada *instance* tersebut.

4.2 Hasil Eksperimen dan Analisis tiap Algoritma

Hasil *hill climbing* dan perbandingan *objective* SA pada bagian ini menggunakan `base` dengan tiga *seed* (11–13), kecuali bila disebut lain. Analisis lintas *fixture* dan parameter SA serta seluruh analisis GA dijelaskan dan dilaporkan tersendiri karena mencakup seluruh kategori fixture serta lima seed (11–15).

4.2.1 Hill Climbing

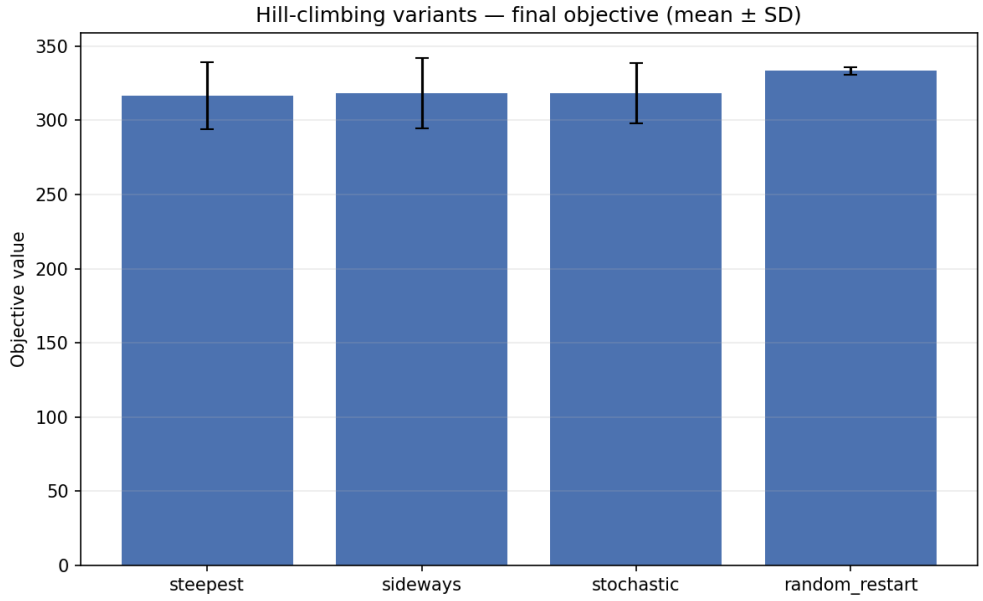
Fungsi objektif `total_value`

Varian	Rata-rata	Simpangan baku	Terbaik	Iterasi rata-rata	Durasi rata-rata (s)
Steepest	316,67	27,54	335	0,33	0,0053
Sideways	318,33	28,87	335	100,67	0,4646
Stochastic	318,33	24,66	335	1000	0,1703
Random restart	333,33	2,89	335	1,67	0,1535

Tabel 4.3: Hasil *hill climbing* dengan `total_value` (`base`, 3 *seed*)

Nilai akhir per *seed* 11, 12, dan 13 untuk *steepest* adalah 330, 285, dan 335. Nilai untuk *sideways* adalah 335, 285, dan 335, untuk *stochastic* adalah 330, 290, dan 335, sedangkan *random restart* menghasilkan 335, 330, dan 335. *Seed* 12 menjadi kasus tersulit. Nilai awalnya 285 dengan lima paket termuat dan merupakan *local optimum* bagi *steepest* serta *sideways*. Varian pertama berhenti pada iterasi ke-0, sedangkan varian kedua memakai 100 *sideways move* tanpa perbaikan. *Stochastic* hanya mencapai 290 setelah 1000 iterasi. Pada *seed* 11, *steepest* dan *stochastic* tidak memperbaiki nilai awal 330, sementara *sideways* mencapai 335 melalui gerakan setara. Semua varian mencapai 335 pada *seed* 13. Hasil ini menunjukkan bahwa gerakan tunggal *swap*, *move*, atau rotasi yang tidak boleh menurunkan nilai dapat membuat pencarian menetap pada optimum lokal.

Random restart adalah satu-satunya varian yang tidak tersangkut pada *seed* 12 (330) dan mencapai 335 pada dua *seed* lain sehingga simpangan bakunya turun dari sekitar 25–29 menjadi 2,89. Sepuluh pendakian tambahan dari *state* acak memberi peluang untuk mendarat di cekungan yang berbeda. Iterasi rata-ratanya kecil (1,67) karena sebagian besar pendakian segera berhenti. Durasinya (0,15 detik) berada di antara *steepest* dan *sideways*.



Gambar 4.1: Ringkasan *hill climbing* dengan `total_value`

Fungsi objektif `eta_weighted_value`

Varian	Rata-rata	Simpangan baku	Terbaik	Nilai total paket rata-rata	Durasi rata-rata (s)
Steepest	103,23	11,82	116,33	303,33	0,0064
Sideways	116,33	0	116,33	305,00	0,3379
Stochastic	110,89	9,43	116,33	310,00	0,1644
Random restart	114,89	1,50	116,33	298,33	0,1784

Tabel 4.4: Hasil *hill climbing* dengan `eta_weighted_value` (`base`, 3 *seed*)

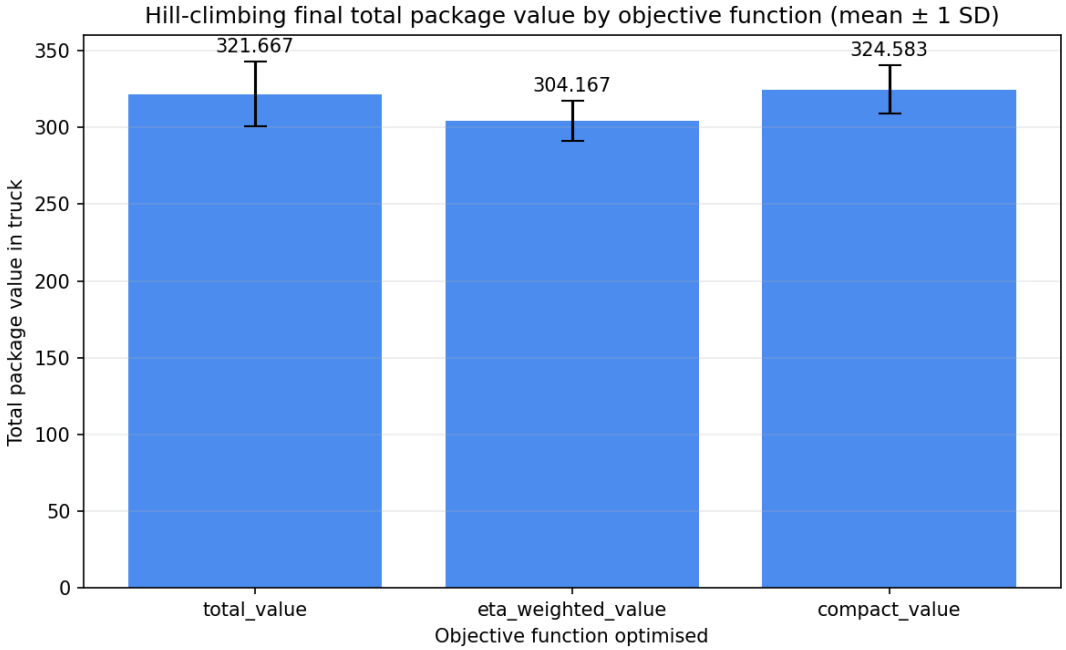
Pada `eta_weighted_value`, *sideways* mencapai nilai terbaik 116,33 pada ketiga *seed*. *Random restart* memperoleh 116,33, 115,00, dan 113,33. Ini satu-satunya kombinasi *instance* dan *objective* dalam eksperimen per algoritma ketika *random restart* bukan yang terbaik. Penyebabnya tidak diselidiki lebih lanjut. Dengan tiga seed, selisih 1,4 poin dapat berasal dari noise. Himpunan paket yang dipilih juga berbeda dari hasil `total_value`. Nilai total paket pada *state* akhir hanya 298–310, di bawah 335 yang dapat dicapai dan dibahas pada subbab fungsi objektif.

Fungsi objektif `compact_value`

Varian	Rata-rata	Simpangan baku	Terbaik	Iterasi rata-rata	Durasi rata-rata (s)
Steepest	316,91	27,55	335,254	0,33	0,0053
Sideways	328,76	7,58	335,380	103,67	0,3724
Stochastic	322,03	18,86	335,344	1000	0,1381
Random restart	331,98	2,84	335,254	18,33	0,1874

Tabel 4.5: Hasil *hill climbing* dengan `compact_value` (`base`, 3 *seed*)

Berbeda dari `total_value`, `compact_value` memberi gradien kecil pada dataran sehingga *sideways* naik dari 318,33 menjadi 328,76 dan *stochastic* dari 318,33 menjadi 322,03. Pada *seed* 12 keduanya meninggalkan *state* 285 karena ada gerakan yang menurunkan tumpukan (iterasi *random restart* juga naik dari 1,67 menjadi 18,33). *Steepest* tidak terbantu (316,91) karena hanya menerima perbaikan sedikit demi sedikit dari sampel 30 *neighbor*. *Random restart* tetap yang terbaik (331,98). Nilai terbaik 335,380 pada *sideways* melebihi 335 karena bonus kepadatan lebih besar pada penataan yang lebih rendah, bukan karena paket tambahan. Bonus dirancang selalu kurang dari satu satuan nilai.



Gambar 4.2: Perbandingan ketiga *objective function* pada *hill climbing*

Metrik penghentian per *seed*

Spesifikasi meminta metrik penghentian dan parameter khusus tiap varian. Tabel 4.6 merinci setiap *run* `total_value` pada `base`: nilai awal dan akhir, iterasi hingga berhenti, jumlah *sideways move*, dan durasi. Parameter yang dipakai sama untuk semua *run*: `neighbors_per_iteration` = 30, `max_iterations` = 1000, `max_sideways` = 100, dan `max_restarts` = 10. Untuk *random restart*, iterasi adalah jumlah dari seluruh pendakian dan nilai awal adalah nilai *state* awal yang sama dengan varian lain.

Varian	*Seed*	Nilai awal	Nilai akhir	Iterasi	*Sideways move*	Durasi (s)
Steepest	11	330	330	0	0	0,0045
	12	285	285	0	0	0,0022
	13	320	335	1	0	0,0092
Sideways	11	330	335	101	100	0,4937
	12	285	285	100	100	0,2418
	13	320	335	101	100	0,6583
Stochastic	11	330	330	1000	0	0,1802
	12	285	290	1000	0	0,0955
	13	320	335	1000	0	0,2350
Random restart	11	330	335	1	0	0,1993
	12	285	330	3	0	0,1333
	13	320	335	1	0	0,1280

Tabel 4.6: Metrik tiap *run hill climbing* dengan `total_value` pada `base`

Tiga pola terlihat pada Tabel 4.6. Pertama, *steepest* berhenti pada iterasi ke-0 pada *seed* 11 dan 12. Artinya *state* awal (330 dan 285) tidak memiliki *neighbor* yang lebih baik di antara 30 sampel. Pada *seed* 13 hanya satu perpindahan terjadi (320 menjadi 335). Kedua, *sideways* selalu menghabiskan seluruh jatah 100 *sideways move* (100 atau 101 iterasi). Inilah penyebab durasinya jauh lebih tinggi daripada *steepest*. Gerakan setara itu memperbaiki hasil pada *seed* 11 (330 menjadi 335) dan *seed* 13 (320 menjadi 335), tetapi tidak pada *seed* 12, yang tetap 285 setelah 100 gerakan setara; dalam batas 100 langkah tidak ditemukan *state* yang lebih baik. Ketiga, *stochastic* selalu menjalankan 1000 iterasi. Pada *seed* 12 nilainya hanya naik dari 285 ke 290, dan pada *seed* 11 tidak berubah.

Tabel 4.7 memerinci sebelas pendakian *random restart*. Pendakian pertama berasal dari *state* awal dan sepuluh lainnya dari *state* acak baru.

Seed	Iterasi tiap pendakian	Nilai akhir tiap pendakian
11	0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0	330, 305, 335, 285, 335, 320, 330, 275, 295, 270, 285
12	0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0	285, 300, 280, 330, 290, 300, 310, 320, 305, 330, 305
13	1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0	335, 330, 285, 295, 320, 295, 295, 320, 270, 290, 335

Tabel 4.7: Iterasi dan nilai akhir tiap pendakian *random restart* pada `base` (`total_value`)

Sebagian besar pendakian berhenti pada iterasi ke-0. Pada *seed* 11 dan 13, sepuluh dari sebelas pendakian tidak melakukan perpindahan, dan pada *seed* 12 delapan dari sebelas. *State* acak baru sering kali sudah menjadi optimum lokal menurut sampel 30 *neighbor*. Nilai 335 pada *seed* 11 berasal dari dua sumber: pendakian ke-3 yang naik dari 310, dan pendakian ke-5 yang *state* acaknya sudah bernilai 335. Pada *seed* 13, nilai 335 berasal dari pendakian pertama (dari *state* awal, 320 menjadi 335) dan pendakian terakhir yang *state* acaknya sudah 335. Pada *seed* 12, yang menahan varian lain di 285–290, nilai 330 dicapai oleh pendakian ke-4 dan ke-10 yang sama-sama naik dari 315. Pendakian dari *state* awal (285) tidak bergerak. Hasil ini menunjukkan bahwa keunggulan *random restart* berasal dari kombinasi kesempatan memulai dari *state* acak yang baik dan sebagian kecil pendakian yang berhasil naik.

Varian	*Seed*	Nilai awal	Nilai akhir	Iterasi	*Sideways move*	Durasi (s)
Steepest	11	155	155	0	0	0,0025
	12	155	165	1	0	0,0036
	13	160	160	0	0	0,0027
Sideways	11	155	165	101	100	0,1888
	12	155	165	101	100	0,1374
	13	160	165	101	100	0,1639
Stochastic	11	155	155	1000	0	0,0512
	12	155	165	1000	0	0,0344
	13	160	160	1000	0	0,0707
Random restart	11	155	175	3	0	0,1076
	12	155	175	4	0	0,0881
	13	160	175	0	0	0,1101

Tabel 4.8: Metrik tiap *run hill climbing* dengan `total_value` pada `tight-capacity`

Pada `tight-capacity` (Tabel 4.8), *steepest* dan *stochastic* menghasilkan nilai akhir yang sama pada ketiga *seed* (155, 165, dan 160), padahal *stochastic* memakai 1000 iterasi. *Sideways* mencapai 165 pada ketiga *seed* dengan 100 *sideways move* pada masing-masing *run*. *Random restart* mencapai 175 pada ketiga *seed*.

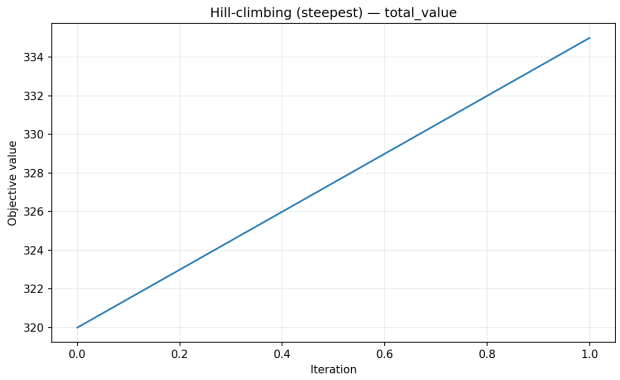
Seed	Iterasi tiap pendakian	Nilai akhir tiap pendakian
11	0, 0, 0, 0, 1, 0, 0, 0, 2, 0, 0	155, 160, 160, 170, 160, 175, 155, 160, 160, 135, 165
12	1, 0, 0, 0, 0, 0, 1, 0, 0, 2, 0	165, 155, 160, 160, 175, 155, 165, 160, 175, 170, 155
13	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0	160, 155, 175, 175, 160, 155, 160, 160, 155, 155, 165

Tabel 4.9: Iterasi dan nilai akhir tiap pendakian *random restart* pada `tight-capacity` (`total_value`)

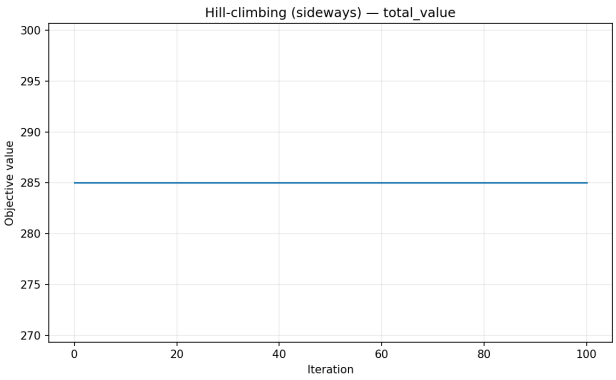
Tabel 4.9 menunjukkan bahwa nilai 175 pada *run* yang dicatat selalu berasal dari *state* acak yang sudah bernilai 175 dan tidak bergerak (iterasi 0), bukan dari pendakian yang naik. Pada *seed* 13 bahkan seluruh sebelas pendakian berhenti pada iterasi ke-0. Pada `tight-capacity`, keunggulan *random restart* atas varian lain karenanya lebih tepat dibaca sebagai hasil pengambilan sampel *state* acak sebanyak sebelas kali daripada hasil pendakian yang lebih baik. Kesimpulan ini terbatas pada tiga *seed* yang dijalankan.

Riwayat *objective function*

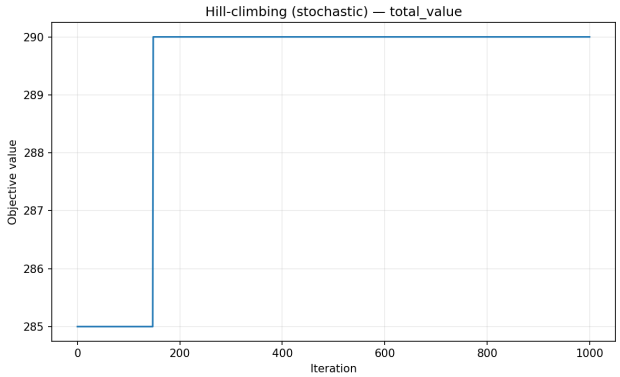
Gambar 4.3 menampilkan nilai *objective* terhadap iterasi untuk keempat varian pada `base`. Riwayat *steepest* pada *seed* 12 hanya terdiri atas satu titik karena berhenti pada iterasi ke-0 sehingga tidak ada garis yang tergambar, maka gambar *steepest* memakai *seed* 13 (320 menjadi 335 dalam satu iterasi). Pada *sideways seed* 12 nilainya datar di 285 selama seluruh *sideways move*. Pada *stochastic* nilai naik sekali ke 290 pada iterasi ke-148 dan tetap hingga iterasi ke-1000. Pada *random restart*, riwayat dari seluruh pendakian digabung dan garis merah putus-putus menandai batas antar-pendakian, sehingga kurva turun pada tiap awal pendakian dan sumbu x adalah indeks titik riwayat gabungan, bukan iterasi satu pendakian.



(a) *Steepest* (*seed* 13)



(b) *Sideways* (*seed* 12)



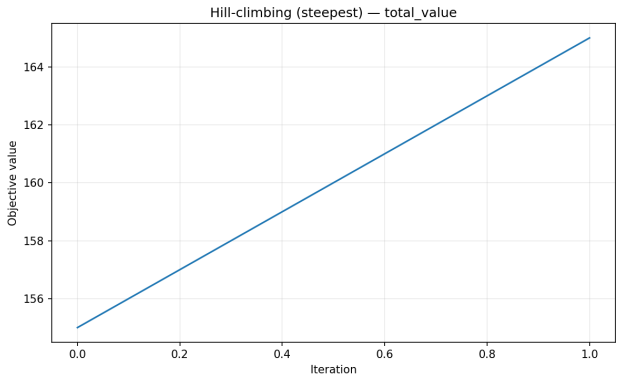
(c) *Stochastic* (*seed* 12)



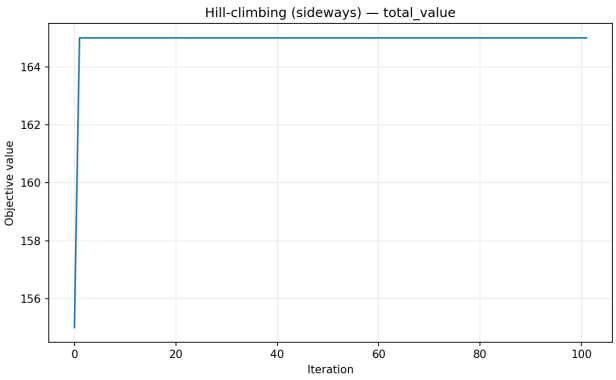
(d) *Random restart* (*seed* 12)

Gambar 4.3: Riwayat *objective* keempat varian *hill climbing* dengan `total_value` pada `base`

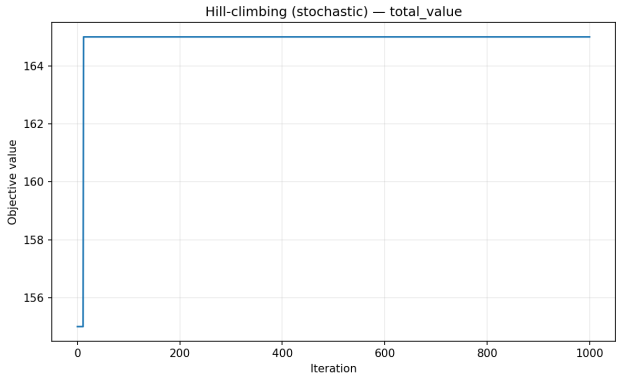
Gambar 4.4 menampilkan hal yang sama untuk `tight-capacity` pada *seed* 12.



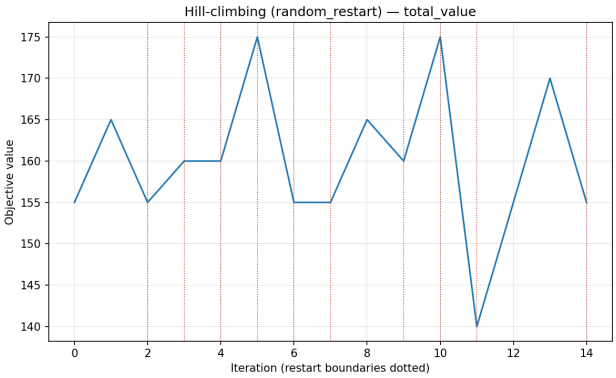
(a) *Steepest* (*seed* 12)



(b) *Sideways* (*seed* 12)



(c) *Stochastic* (*seed* 12)

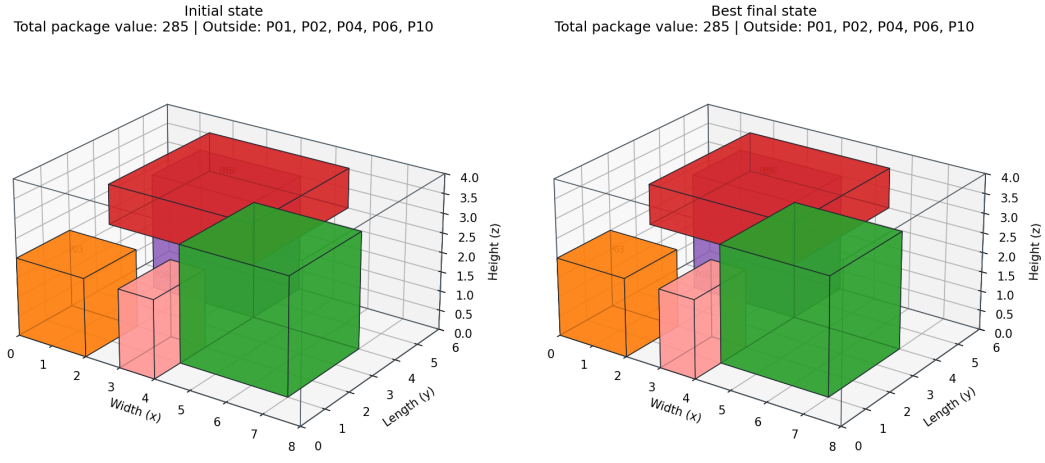


(d) *Random restart* (*seed* 12)

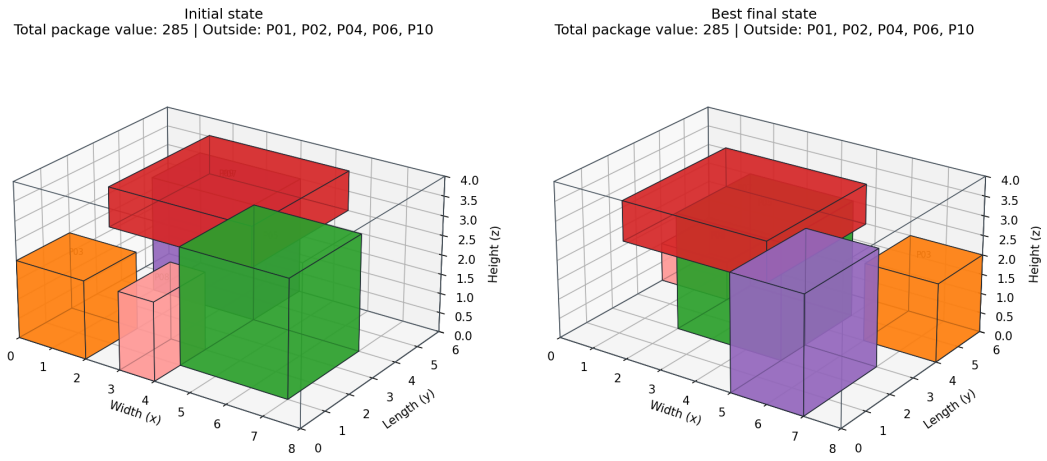
Gambar 4.4: Riwayat *objective* keempat varian *hill climbing* dengan `total_value` pada `tight-capacity`

State awal dan akhir

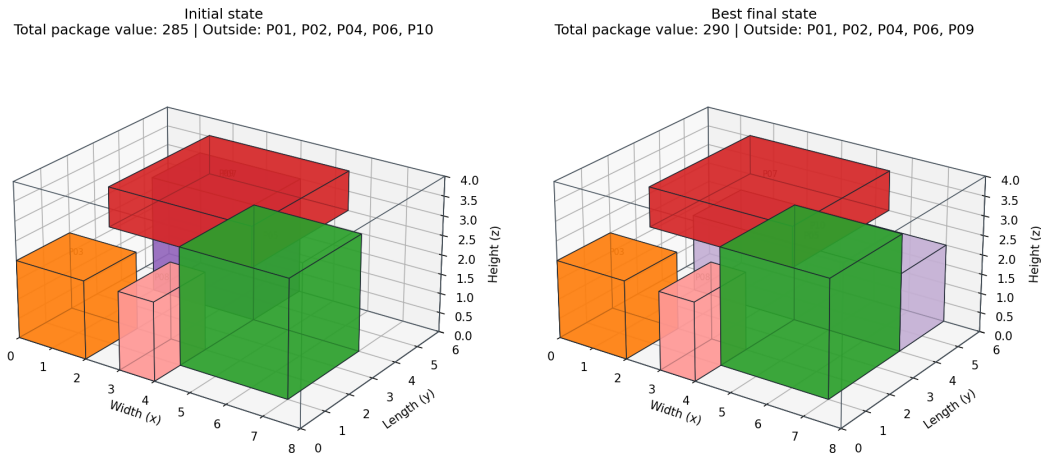
Gambar 4.5 sampai Gambar 4.8 memperlihatkan *state* awal dan akhir keempat varian pada `base` *seed* 12, yaitu kasus tersulit. *Steepest* dan *sideways* tidak mengubah *state* awal bernilai 285, *stochastic* mencapai 290, dan *random restart* mencapai 330.



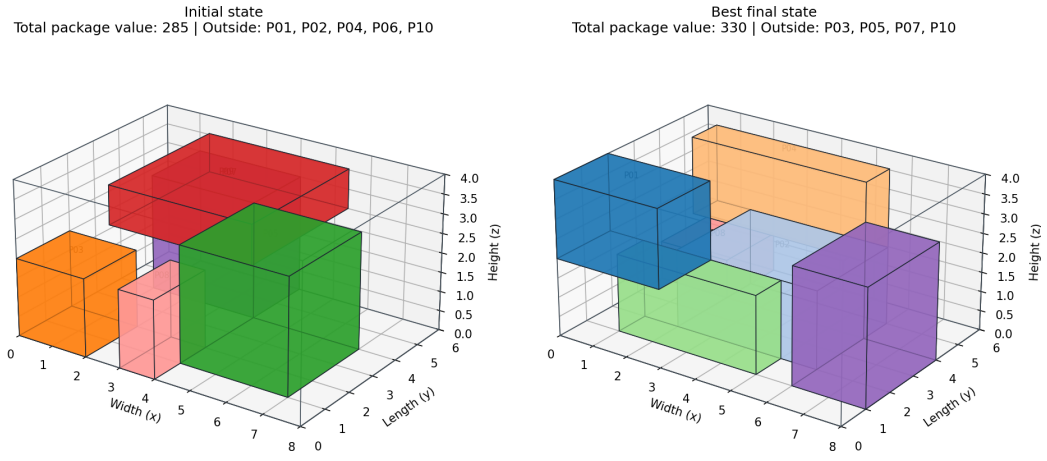
Gambar 4.5: *State* awal dan akhir *Steepest* pada `base` (*seed* 12). Nilai awal 285, nilai akhir 285



Gambar 4.6: *State* awal dan akhir *Sideways* pada `base` (*seed* 12). Nilai awal 285, nilai akhir 285

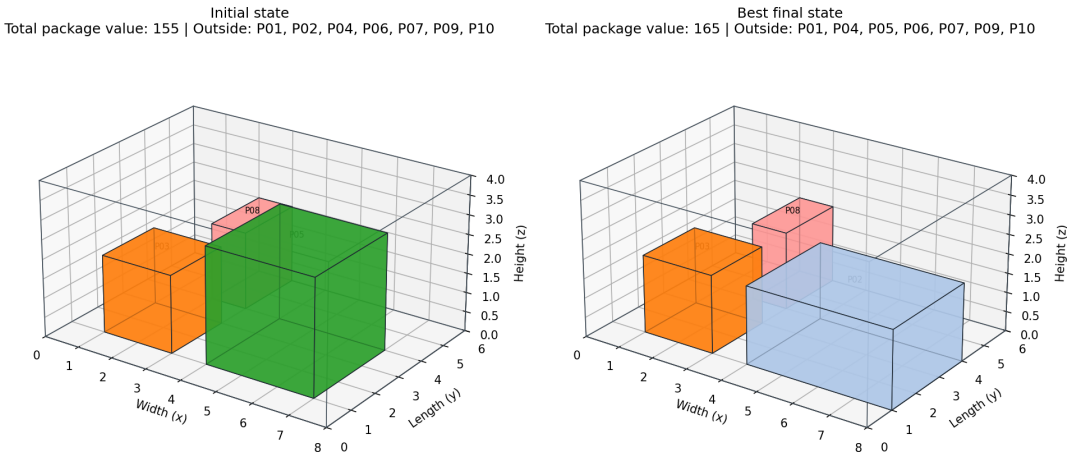


Gambar 4.7: *State* awal dan akhir *Stochastic* pada `base` (*seed* 12). Nilai awal 285, nilai akhir 290

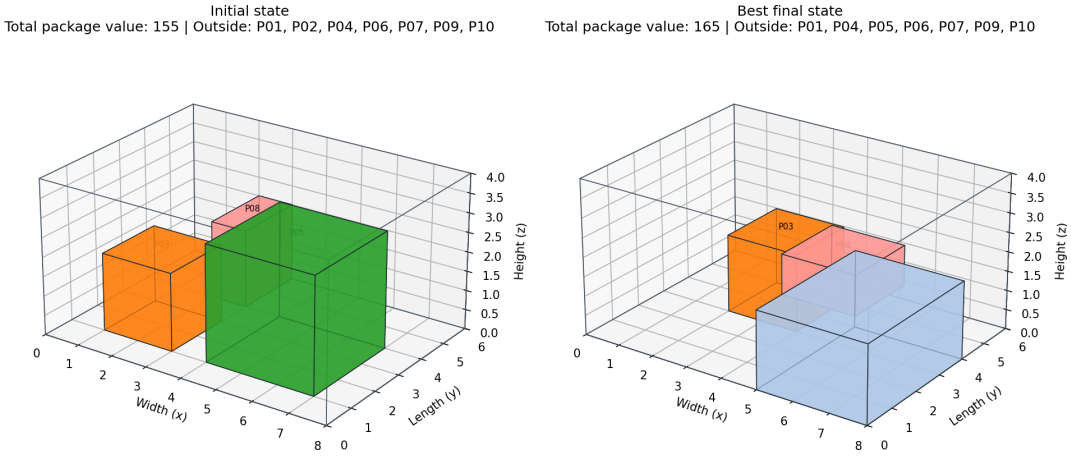


Gambar 4.8: *State* awal dan akhir *Random restart* pada `base` (*seed* 12). Nilai awal 285, nilai akhir 330

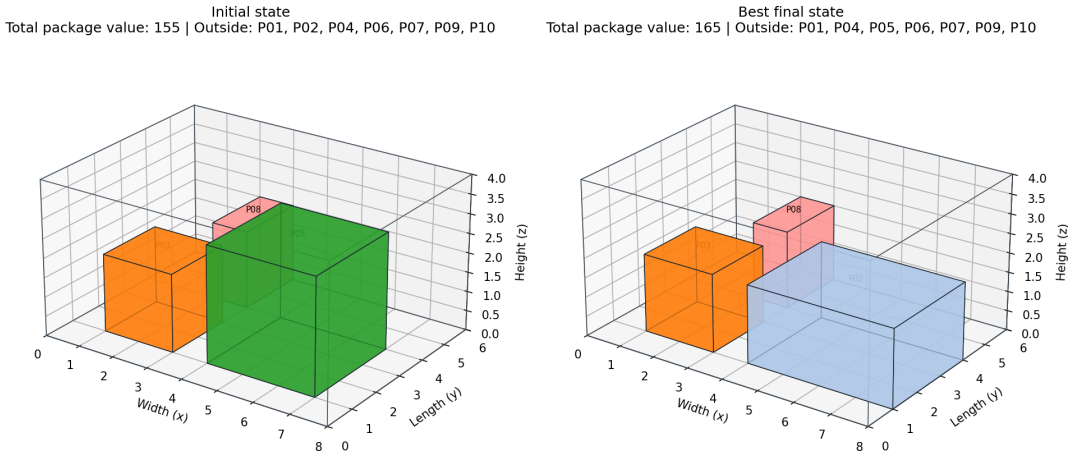
Gambar 4.9 sampai Gambar 4.12 menampilkan hal yang sama untuk `tight-capacity` *seed* 12. *Steepest*, *sideways*, dan *stochastic* sama-sama mencapai 165 dari nilai awal 155, sedangkan *random restart* mencapai 175.



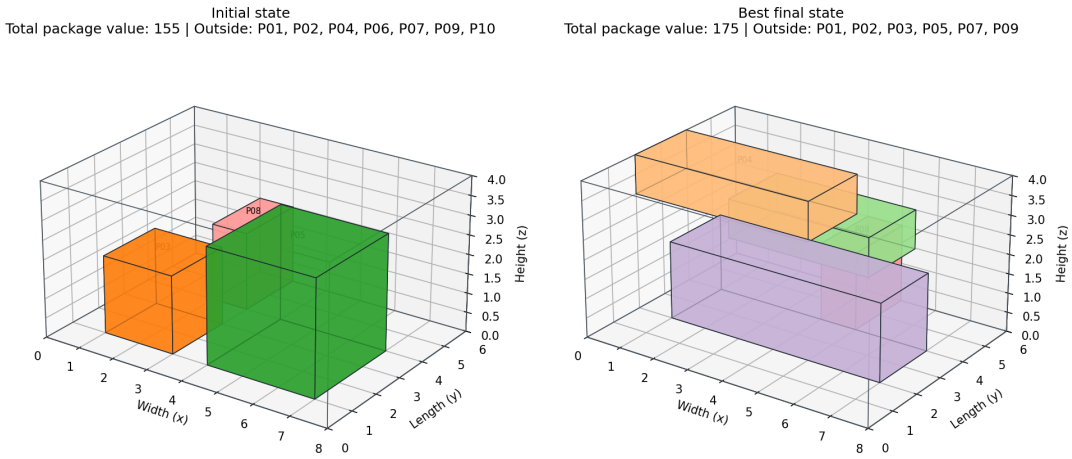
Gambar 4.9: *State* awal dan akhir *Steepest* pada `tight-capacity` (*seed* 12). Nilai awal 155, nilai akhir 165



Gambar 4.10: *State* awal dan akhir *Sideways* pada `tight-capacity` (*seed* 12). Nilai awal 155, nilai akhir 165



Gambar 4.11: *State* awal dan akhir *Stochastic* pada `tight-capacity` (*seed* 12). Nilai awal 155, nilai akhir 165



Gambar 4.12: *State* awal dan akhir *Random restart* pada `tight-capacity` (*seed* 12). Nilai awal 155, nilai akhir 175

4.2.2 Simulated Annealing

Fungsi objektif `total_value`

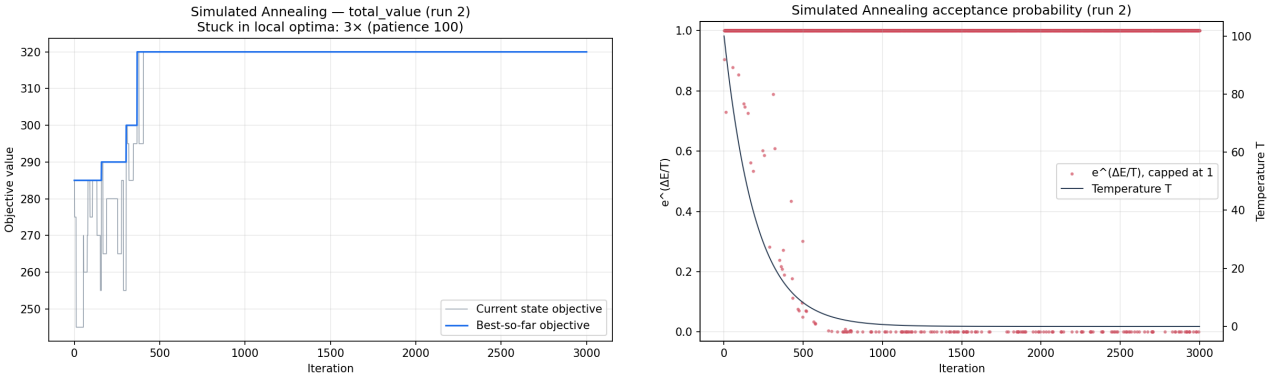
Objective	Rata-rata akhir	Simpangan baku	Nilai total paket	Durasi rata-rata (s)
`total_value`	330	8,66	330	$0{,}390 \pm 0{,}037$
`eta_weighted_value`	116,33	0	305	$0{,}375 \pm 0{,}027$
`compact_value`	332,04	2,79	331,67	$0{,}568 \pm 0{,}043$

Tabel 4.10: Hasil *simulated annealing* (`base`, 3 *seed*, 3000 iterasi)

Dengan `total_value`, SA mencapai 335 pada *seed* 11 dan 13 serta 320 pada *seed* 12, dengan rata-rata 330. *Seed* 12 berawal dari 285 dan menjadi kasus tersulit bagi hampir semua algoritma. SA keluar dari *local optimum* tersebut dan naik ke 320. Hasil ini lebih tinggi daripada *steepest* dan *sideways* yang berhenti pada 285, serta *stochastic* yang mencapai 290. Namun, SA belum mencapai 335. Riwayat nilai saat ini menunjukkan bahwa SA menerima solusi jauh lebih buruk saat suhu tinggi. Nilai minimum yang dikunjungi adalah 245 pada *seed*

12 dan 255 pada *seed* 11. Seiring pendinginan $0{,}995^{3000} \approx 3 \times 10^{-7}$, pencarian menyempit ke satu cekungan. Hasil akhir menyimpan solusi terbaik yang pernah dikunjungi, bukan solusi saat pencarian berhenti. Jumlah kejadian *stuck*, saat nilai terbaik tidak berubah selama 100 iterasi berturut-turut, rata-rata 2 per *run*. Nilainya 2, 3, dan 1 untuk *seed* 11, 12, dan 13.

Biaya utama SA adalah waktu pencarian. Algoritma selalu menjalankan 3000 iterasi tanpa berhenti dini, dengan durasi sekitar 0,4 detik per *run*. Durasi ini sekitar 2,5 kali durasi *random restart* dan 74 kali durasi *steepest*.



(a) Nilai *objective* (*seed* 12)

(b) $e^{\Delta E/T}$ terhadap iterasi

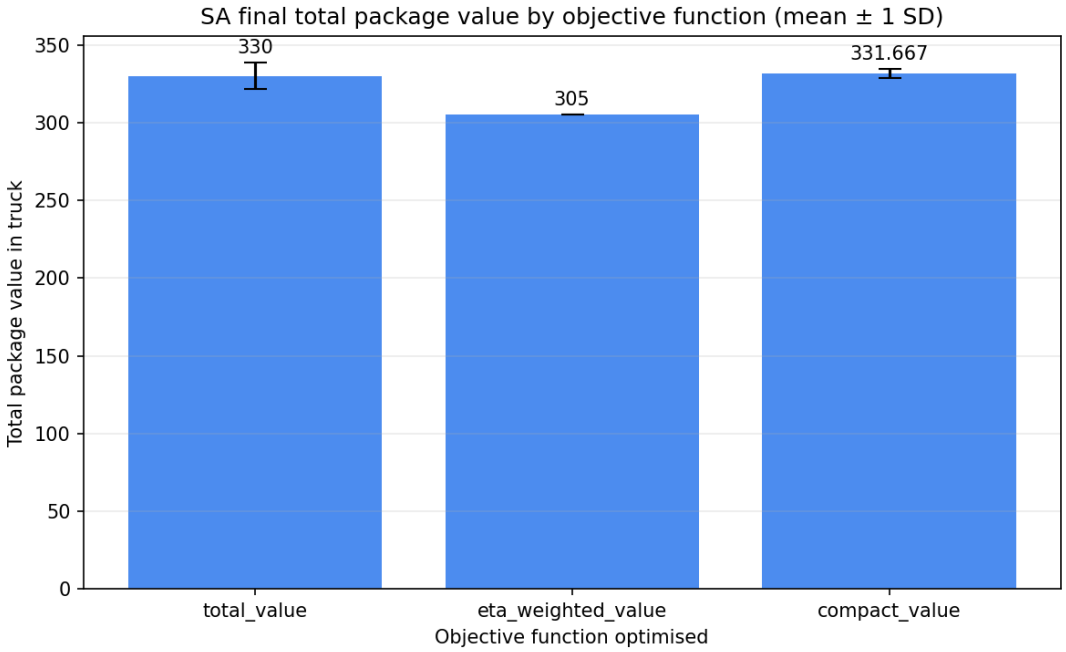
Gambar 4.13: *Simulated annealing* dengan `total_value` pada *seed* 12

Fungsi objektif `eta_weighted_value`

Ketiga *seed* menghasilkan 116,33 dengan simpangan baku 0, sama dengan nilai terbaik yang ditemukan *hill climbing*. Himpunan paket berbeda dari hasil `total_value`. Nilai total paketnya 305, bukan 335, tetapi skor ETA mencapai 116,33 dibandingkan 93,45 ketika state hasil `total_value` dinilai dengan fungsi ETA, atau naik sekitar 24%. Jumlah kejadian *stuck* rata-rata 4, dua kali lipat hasil `total_value`. Salah satu kemungkinan penyebabnya adalah skala objektif yang lebih kecil membuat suhu awal 100 relatif lebih tinggi. Pengaruh suhu belum diuji dengan variasi T_0. Durasi 0,375 detik setara dengan `total_value`.

Fungsi objektif `compact_value`

Nilai akhir pada tiga *seed* adalah 330,449, 330,410, dan 335,254. Rata-ratanya 332,04. Kepadatan rata-rata hasil SA naik dari 0,250 pada `total_value` menjadi 0,375 sehingga bonus kepadatan mendorong penataan ke tumpukan yang lebih rendah. Nilai total paket pada masing-masing *seed* adalah 330, 330, dan 335, dengan rata-rata 331,67. Sebagai pembanding, `total_value` menghasilkan 335, 320, dan 335. Selisih rata-rata 1,67 berbeda arah antar-seed dan dapat berasal dari *noise*. Durasi rata-rata mencapai 0,568 detik, sekitar 46% lebih lama daripada 0,390 detik pada `total_value`. Penyebab perbedaan durasi ini belum diselidiki.



Gambar 4.14: Perbandingan ketiga *objective function* pada *simulated annealing*

Protokol dan hasil baseline lintas kasus

Baseline SA dijalankan pada seluruh 11 *fixture* valid dengan `total_value`, $T_0 = 100$, laju pendinginan 0,995, 3000 iterasi, dan lima *seed* (11–15) sehingga menghasilkan 55 *run*. *State* awal diturunkan dengan $s_0 = s \oplus$ `0x5EED` sehingga setiap pasangan *fixture–seed* selalu berangkat dari *state* awal yang sama di seluruh rangkaian SA. Setiap *run* memakai 3001 evaluasi objektif (satu untuk *state* awal dan satu per iterasi). Seluruh *state* akhir lolos validasi (100%), dan kelima *run* pada `infeasible` mengembalikan *state* kosong bernilai 0.

Selain nilai akhir, tabel mencatat dua besaran yang khas SA, yaitu jumlah gerakan memburuk yang diterima (transisi ketika nilai *state* saat ini turun) dan iterasi saat *state* terbaik pertama kali ditemukan. Keduanya memperlihatkan seberapa sering mekanisme penerimaan probabilistik benar-benar dipakai dan seberapa lama pencarian masih berguna.

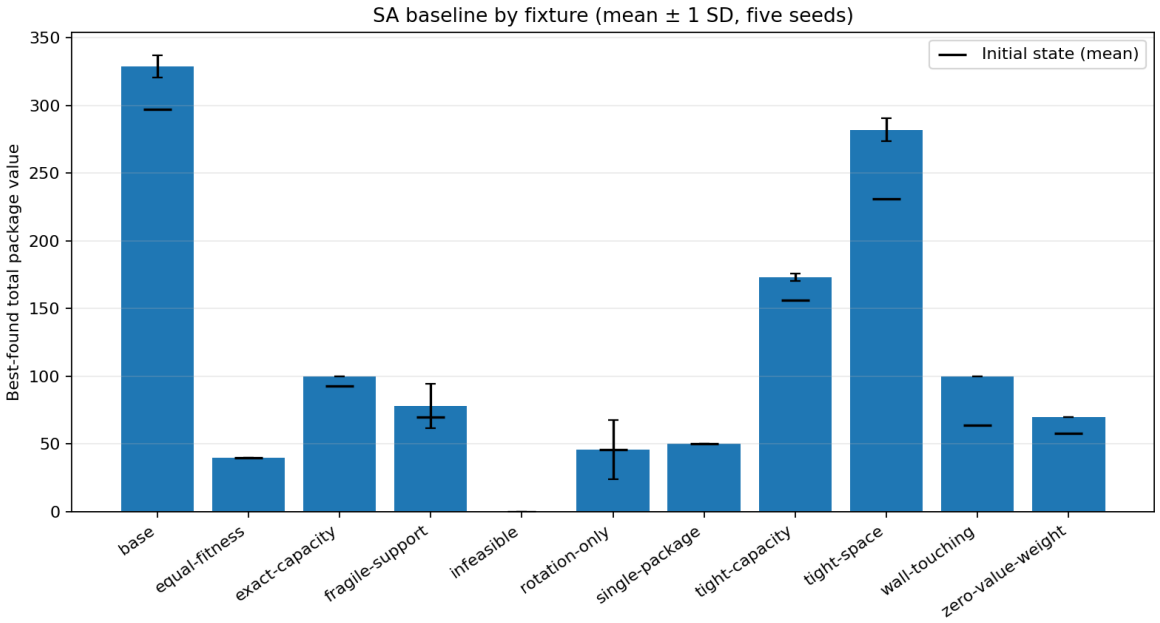
Tabel 4.11: Baseline SA pada seluruh fixture valid, dengan rata-rata dan simpangan baku sampel dari lima seed

Fixture	Nilai awal	Rata-rata ± SD	Terbaik	Paket	Memburuk diterima	Iterasi terbaik	Durasi (s)
`base`	297,0	329,00 ± 8,22	335	6,0	8,6	585	0,640
`tight-space`	231,0	282,00 ± 8,37	290	5,4	2,8	819	0,480
`tight-capacity`	156,0	173,00 ± 2,74	175	3,8	5,8	73	0,304
`infeasible`	0,0	0,00 ± 0,00	0	0,0	0,0	0	0,072
`single-package`	50,0	50,00 ± 0,00	50	1,0	0,0	0	0,045
`equal-fitness`	40,0	40,00 ± 0,00	40	4,0	0,0	0	0,127
`exact-capacity`	93,0	100,00 ± 0,00	100	3,0	41,2	18	0,099
`wall-touching`	64,0	100,00 ± 0,00	100	4,0	6,6	67	0,121
`fragile-support`	70,0	78,00 ± 16,43	90	2,0	9,4	1	0,154
`rotation-only`	46,0	46,00 ± 21,91	70	1,0	0,4	0	0,080
`zero-value-weight`	58,0	70,00 ± 0,00	70	3,0	2,8	11	0,145

SA memperbaiki nilai awal pada tiga *fixture* utama. Pada `base`, kenaikannya rata-rata 32 poin (297 ke 329). Pada `tight-space`, kenaikannya 51 poin (231 ke 282), sedangkan pada `tight-capacity` kenaikannya 17 poin (156 ke 173). Pada `base`, kelima *seed* berakhir pada 335, 320, 335, 320, dan 335. Dua *seed* yang berawal dari 285 dan 275 (12 dan 14) berhenti pada 320. Pada `tight-capacity`, tiga *seed* mencapai optimum 175 dan dua berakhir pada 170, termasuk *seed* 14 yang *state* awalnya (170) tidak pernah dilampaui. `tight-space` memiliki

sebaran yang serupa dengan `base` (simpangan baku 8,37) dan nilai terbaik 290, di bawah batas atas *knapsack* 335 sehingga kesulitannya terletak pada penataan geometris.

Pada kasus batas, SA mencapai optimum pada `exact-capacity`, `wall-touching`, dan `zero-value-weight` dari seluruh *state* awal, termasuk *state* `wall-touching` yang hanya bernilai 10 (*seed* 12). Sebaliknya, `fragile-support` berhenti pada 60 untuk *seed* 12 dan 15, sedangkan `rotation-only` berhenti pada 30 untuk *seed* 11, 13, dan 14. Pada kelima *run* tersebut, tidak ada satu pun dari 3000 *neighbor* yang diusulkan bernilai lebih rendah atau lebih tinggi daripada *state* saat ini. Semua kandidat berupa *move* atau rotasi yang tidak mengubah himpunan paket. Suhu tinggi tidak membantu karena tidak ada gerakan memburuk yang dapat diterima. Ini mengonfirmasi analisis pada `rotation-only` di subbab ketahanan. Keterbatasan berasal dari keterhubungan operator *neighbor*, bukan dari jadwal pendinginan.



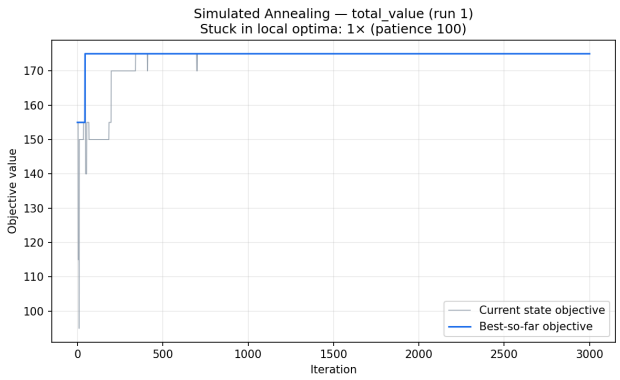
Gambar 4.15: Rata-rata nilai terbaik SA pada tiap fixture (batang, ±1 SD) dan rata-rata nilai *state* awal (garis hitam)

Dinamika penerimaan dan pendinginan

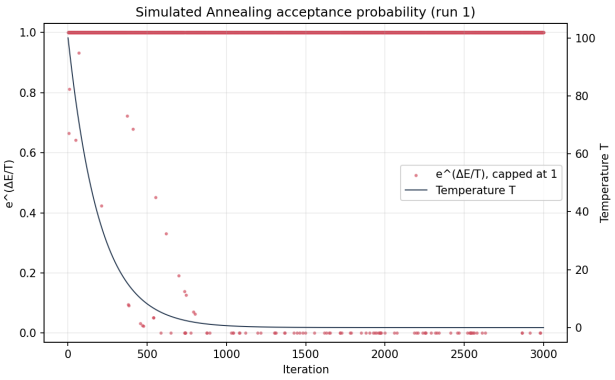
Dengan $T_0 = 100$ dan laju 0,995, suhu turun di bawah 1 setelah sekitar 919 iterasi ($100 \cdot 0{,}995^i < 1 \iff i > \ln 100 / -\ln 0{,}995 \approx 919$), dan pada iterasi ke-3000 tersisa 3×10^{-5}. Pada `base`, rata-rata hanya 196 dari 3000 *neighbor* (6,5%) bernilai lebih rendah daripada *state* saat ini, dan dari jumlah itu hanya 8,6 yang diterima. Gerakan memburuk terakhir diterima pada iterasi ke-346 hingga ke-685 sehingga setelah sekitar 700 iterasi SA praktis hanya menerima gerakan yang setara atau lebih baik. *State* terbaik ditemukan rata-rata pada iterasi ke-585 (paling lambat 1344 pada *seed* 11). Sisa iterasi tidak menghasilkan perbaikan apa pun.

Sebagian besar *neighbor* tidak mengubah nilai karena *move* dan rotasi hanya mengubah penataan, sedangkan hanya *swap* yang mengubah himpunan paket. Pada `tight-space`, rata-rata hanya 31 *neighbor* memburuk per *run* dan 2,8 gerakan diterima. Ruang truk yang sempit membuat *swap* yang layak jarang. Sebaliknya, `exact-capacity` memiliki lebih dari 540 usulan memburuk per *run* dan 41 gerakan diterima. Hal ini terjadi karena nilai keempat paketnya berdekatan (25–40) sehingga selisih nilai akibat *swap* kecil relatif terhadap suhu dan $e^{\Delta E/T}$ mendekati 1. Temuan ini menunjukkan bahwa makna $T_0 = 100$ bergantung pada skala selisih nilai di tiap *instance*.

Gambar 4.16 memperlihatkan pola ini pada `tight-capacity` dengan *seed* 11. Nilai *state* saat ini turun hingga 95 pada puluhan iterasi pertama. *State* terbaik bernilai 175 ditemukan pada iterasi ke-45, sementara beberapa penurunan singkat ke 170 masih diterima hingga sekitar iterasi ke-700. Grafik $e^{\Delta E/T}$ menunjukkan probabilitas penerimaan gerakan memburuk yang meluruh menuju nol mengikuti pendinginan.



(a) Nilai *state* saat ini dan terbaik



(b) $e^{\Delta E/T}$ terhadap iterasi

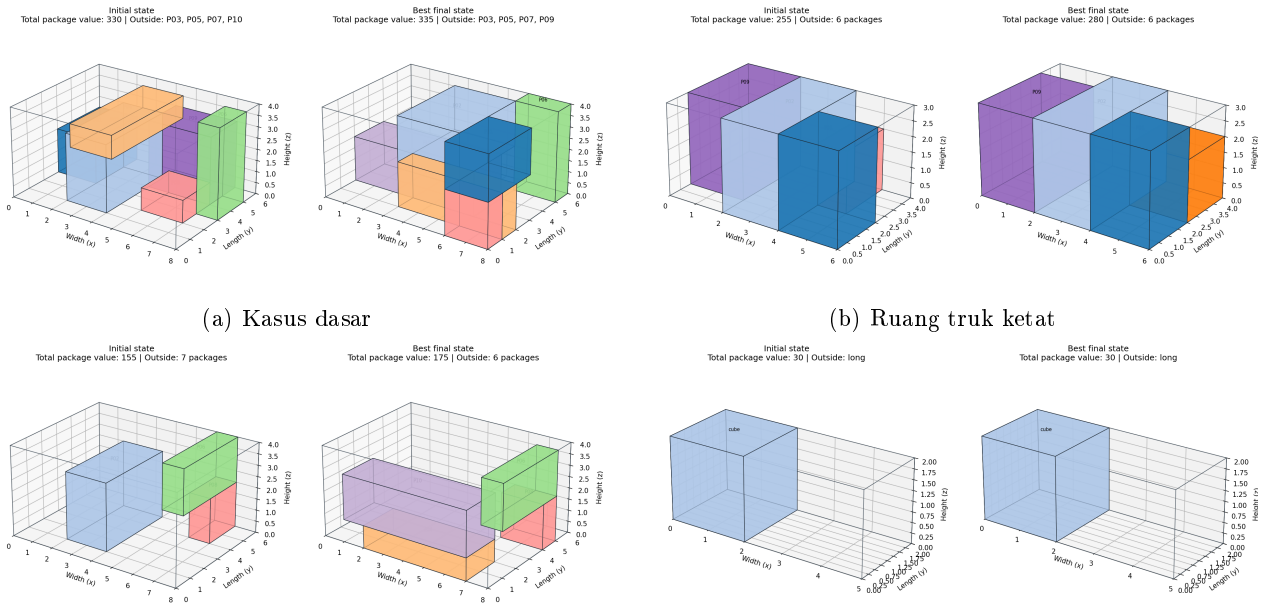
Gambar 4.16: Dinamika SA pada `tight-capacity` (*seed* 11)

Contoh keluaran dan visualisasi state

Tabel 4.12 memperlihatkan keluaran konkret *run* dengan *seed* 11, satu *run* representatif yang dipilih sebelum melihat hasil. Kolom paket menunjukkan isi *state* terbaik, bukan seluruh *state* yang dikunjungi. Pada `base`, SA naik dari 330 ke optimum 335 dengan enam paket, dan pada `tight-space` dari 255 ke 280 dengan empat paket. Pada `tight-capacity`, SA mencapai optimum 175 dengan empat paket. Pada `rotation-only`, SA tetap memuat paket kubus (30) karena tidak ada *neighbor* yang mengubah nilai, sedangkan `infeasible` tetap kosong.

Tabel 4.12: Keluaran aktual SA untuk seed 11 pada satu wakil tiap kategori dan kedua fixture kendala ketat

Kasus	Nilai awal → akhir	Paket	Valid	Paket termuat pada hasil
`base`	330 → 335	6/10	Ya	P01, P02, P04, P06, P08, P10
`tight-space`	255 → 280	4/10	Ya	P01, P02, P03, P09
`tight-capacity`	155 → 175	4/10	Ya	P04, P06, P08, P10
`infeasible`	0 → 0	0/10	Ya	–
`rotation-only`	30 → 30	1/2	Ya	`cube`



(a) Kasus dasar

(b) Ruang truk ketat

(c) Kapasitas truk ketat

(d) Paket hanya muat setelah rotasi (tidak membaik)

Gambar 4.17: State awal dan state terbaik yang dikembalikan SA untuk seed 11

Pengaruh parameter SA

Suhu awal, laju pendinginan, dan jumlah iterasi diuji secara terpisah pada `base` dengan lima *seed* dan *state* awal yang sama. Parameter lain tetap pada baseline. Tabel 4.13 juga mencantumkan suhu akhir $T_0 \cdot r^N$ untuk membantu membaca seberapa "dingin" pencarian ketika berhenti.

Tabel 4.13: Sweep satu faktor SA pada fixture `base` (lima seed per nilai parameter)

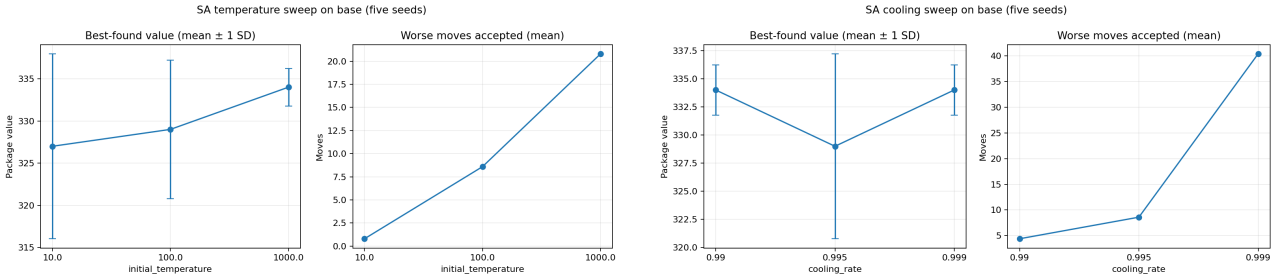
Parameter	Nilai	Rata-rata ± SD	Nilai per *seed* (11–15)	Memburuk diterima	Suhu akhir	Durasi (s)
T_0	10	327,00 ± 10,95	335, 315, 335, 315, 335	0,8	3×10^{-6}	0,429
	100	329,00 ± 8,22	335, 320, 335, 320, 335	8,6	3×10^{-5}	0,457
	1000	334,00 ± 2,24	335, 330, 335, 335, 335	20,8	3×10^{-4}	0,502
Cooling rate	0,99	334,00 ± 2,24	330, 335, 335, 335, 335	4,4	8×10^{-12}	0,494
	0,995	329,00 ± 8,22	335, 320, 335, 320, 335	8,6	3×10^{-5}	0,473
	0,999	334,00 ± 2,24	330, 335, 335, 335, 335	40,4	4,97	0,431
Iterasi	1000	328,00 ± 7,58	330, 320, 335, 320, 335	8,6	0,665	0,138
	3000	329,00 ± 8,22	335, 320, 335, 320, 335	8,6	3×10^{-5}	0,425
	6000	329,00 ± 8,22	335, 320, 335, 320, 335	8,6	9×10^{-12}	0,891

Suhu awal memberi tren yang paling jelas. Menaikkan T_0 dari 10 ke 1000 menaikkan jumlah gerakan memburuk yang diterima dari 0,8 menjadi 20,8 per *run*, dan dua *seed* tersulit (12 dan 14) naik dari 315 ke 330 dan 335. Dengan $T_0 = 10$, SA hampir tidak pernah menerima gerakan memburuk (empat dari lima *run* menerima paling banyak satu) sehingga hampir seluruh pencarian hanya menerima gerakan setara atau lebih baik. Simpangan baku turun dari 10,95 ke 2,24, dan durasi hanya naik sekitar 17%.

Laju pendinginan tidak memberi tren monoton. Laju 0,99 dan 0,999 sama-sama menghasilkan rata-rata 334, lebih tinggi daripada baseline 0,995 (329), dengan pola per *seed* yang identik. Seed 12 dan 14 mencapai 335, tetapi seed 11 berhenti pada 330. Mekanismenya berbeda. Dengan 0,999, suhu akhirnya masih 4,97 sehingga gerakan memburuk diterima hingga iterasi ke-2550–2970 (40,4 per *run*) dan pencarian tetap menjelajah hingga akhir. Dengan 0,99, suhu turun di bawah 1 setelah sekitar 460 iterasi dan hanya 4,4 gerakan memburuk diterima. Namun, lintasan acak yang berbeda pada awal pencarian kebetulan mengarah ke cekungan yang lebih baik. Karena hasil per *seed* berubah tanpa pola yang konsisten, keunggulan 0,99 dan 0,999 atas 0,995 pada lima *seed*

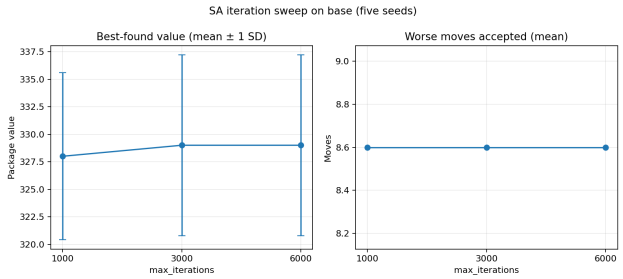
ini lebih tepat dibaca sebagai kepekaan SA terhadap lintasan acak daripada sebagai laju pendinginan optimum.

Jumlah iterasi hanya memengaruhi *seed* 11. Karena generator acak dan jadwal suhu sama, *run* dengan iterasi lebih sedikit merupakan awalan persis dari *run* yang lebih panjang. Dengan 1000 iterasi, seed 11 berhenti pada 330 karena *state* 335 baru ditemukan pada iterasi ke-1344. Dengan 6000 iterasi, hasilnya identik dengan 3000 karena tidak ada perbaikan setelah iterasi ke-1344 pada seed mana pun. Durasi justru berlipat dua, dari 0,425 menjadi 0,891 detik. Pada konfigurasi ini, iterasi setelah suhu mendekati nol hanya menambah biaya sehingga kondisi berhenti dini berdasarkan `stuck_patience` atau suhu minimum akan memangkas durasi tanpa mengubah hasil.



(a) Suhu awal

(b) Laju pendinginan



Gambar 4.18: Pengaruh parameter SA terhadap nilai terbaik dengan batang galat satu simpangan baku serta jumlah gerakan memburuk yang diterima

Sweep hanya dilakukan pada satu *fixture* dengan lima *seed* sehingga hasilnya belum menunjukkan konfigurasi terbaik untuk kategori lain. Durasi diukur pada lingkungan lokal yang sama, tetapi konfigurasi yang identik dapat berbeda hingga sekitar 50% antar-rangkaian (baseline `base` 0,640 detik pada Tabel 4.11 dibandingkan 0,425–0,473 detik pada *sweep*) sehingga selisih durasi kecil antarnilai parameter tidak bermakna. Seluruh 100 keluaran per-*run*, parameter, riwayat *objective*, riwayat suhu, serta *state* awal/akhir disimpan di `output/sa-analysis/`. Eksperimen dapat direproduksi dengan menjalankan `python -m src.experiments.sa.results_analysis` dari root repositori.

4.2.3 Genetic Algorithm

Protokol dan hasil baseline lintas kasus

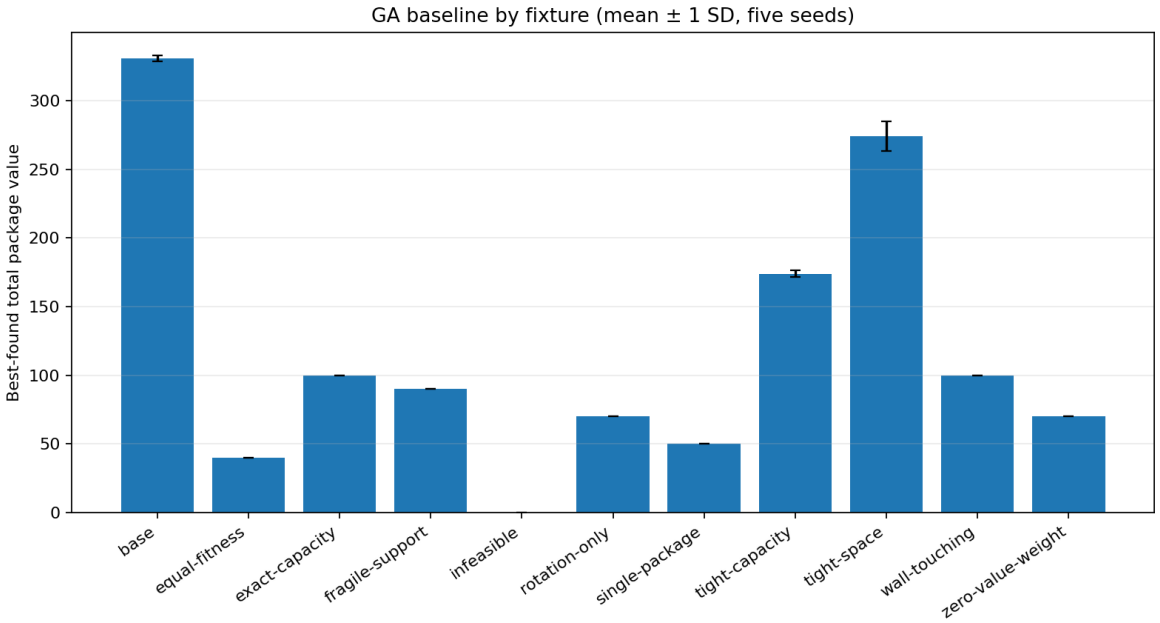
Baseline GA dijalankan pada seluruh 11 *fixture* valid dengan `total_value`, populasi 20, 40 generasi, *mutation rate* 0,05, dan lima *seed* (11–15) sehingga menghasilkan 55 *run*. Untuk setiap pasangan fixture dan seed, state awal dibuat dari seed turunan yang sama, $s_0 = s \oplus \texttt{0x5EED}$. Setiap run melakukan 820 evaluasi objektif, sesuai $(40 + 1) \times 20$. Seluruh state akhir lolos validasi. Pada fixture `infeasible`, kelima run mengembalikan state kosong yang valid dengan nilai 0 dan tanpa paket termuat.

Tabel 4.14 merangkum nilai akhir, variasi antarseed, paket termuat, serta waktu pencarian. Nilai terbaik pada tabel adalah maksimum hasil run yang benar-benar diamati, bukan klaim optimum global. Kasus tanpa pemuatan layak dilaporkan sebagai pemeriksaan validitas, bukan sebagai perbandingan kualitas dengan kasus yang dapat dimuat.

Tabel 4.14: Baseline GA pada seluruh fixture valid. Rata-rata dan simpangan baku sampel dari lima seed

Fixture	Rata-rata nilai ± SD	Terbaik	Paket termuat	Durasi (s)
`base`	331,00 ± 2,24	335	6,0	0,681
`tight-space`	274,00 ± 10,84	290	5,0	0,543
`tight-capacity`	174,00 ± 2,24	175	3,4	0,523
`infeasible`	0,00 ± 0,00	0	0,0	0,205
`single-package`	50,00 ± 0,00	50	1,0	0,041
`equal-fitness`	40,00 ± 0,00	40	4,0	0,339
`exact-capacity`	100,00 ± 0,00	100	3,0	0,170
`wall-touching`	100,00 ± 0,00	100	4,0	0,233
`fragile-support`	90,00 ± 0,00	90	2,0	0,177
`rotation-only`	70,00 ± 0,00	70	1,0	0,108
`zero-value-weight`	70,00 ± 0,00	70	3,0	0,163

Perbedaan terhadap kasus dasar terlihat pada ruang truk yang dipersempit. Rata-rata `tight-space` turun dari 331 pada `base` menjadi 274, dan simpangan bakunya 10,84 merupakan yang terbesar di antara 11 fixture. Nilai terbaik yang ditemukan pada lima seed adalah 290. Sebaliknya, nilai pada `tight-capacity` mendekati batas nilai berbasis kapasitas 175, tetapi tidak semua seed mencapainya. Fixture batas berukuran kecil mencapai nilai yang sama pada kelima seed. Konsistensi tersebut menunjukkan bahwa kasus-kasus sederhana ini terjangkau oleh konfigurasi GA, bukan bahwa algoritma selalu menemukan optimum pada instance lain.



Gambar 4.19: Rata-rata nilai terbaik yang ditemukan pada tiap fixture. Batang galat menunjukkan satu simpangan baku sampel

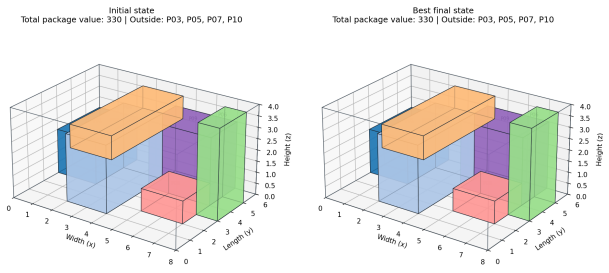
Contoh keluaran dan visualisasi state

Tabel 4.15 memperlihatkan hasil konkret dari run dengan seed 11, satu run representatif yang dipilih konsisten sebelum melihat perbandingan antarparameter. Kolom paket menunjukkan assignment pada state akhir, bukan seluruh kandidat yang diperiksa GA. Pada `base`, nilai awal sudah 330 sehingga run tersebut tidak meningkat. Pada `tight-space` dan `tight-capacity`, pencarian memperbaiki nilai awal. Kasus `rotation-only` memuat paket `long` setelah rotasi, sementara kasus `infeasible` tetap kosong.

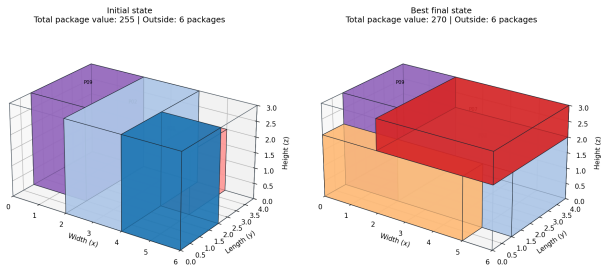
Tabel 4.15: Keluaran aktual GA untuk seed 11 pada satu wakil tiap kategori dan kedua fixture kendala ketat

Kasus	Nilai awal → akhir	Paket	Valid	Paket termuat pada hasil
`base`	330 → 330	6/10	Ya	P01, P02, P04, P06, P08, P09
`tight-space`	255 → 270	4/10	Ya	P02, P04, P07, P09
`tight-capacity`	155 → 175	3/10	Ya	P02, P04, P06
`infeasible`	0 → 0	0/10	Ya	–
`rotation-only`	30 → 70	1/2	Ya	`long`

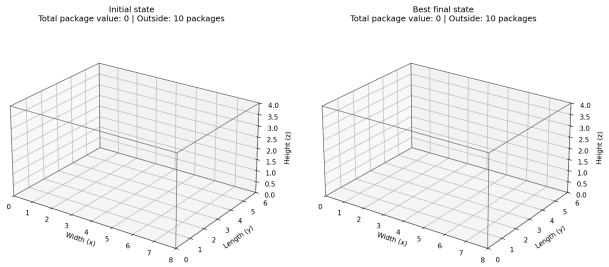
Gambar 4.20 memperlihatkan perubahan penempatan sebelum dan sesudah pencarian pada empat kategori. Gambar 4.21 menambahkan visualisasi `tight-capacity` sehingga kedua bentuk kendala ketat dapat dibandingkan. Gambar state untuk semua fixture dan seed pertama tersedia di `output/ga-analysis/figures/`.



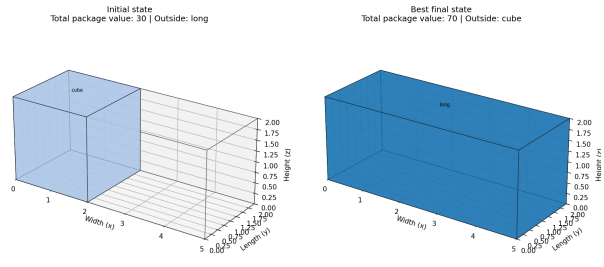
(a) Kasus dasar



(b) Ruang truk ketat

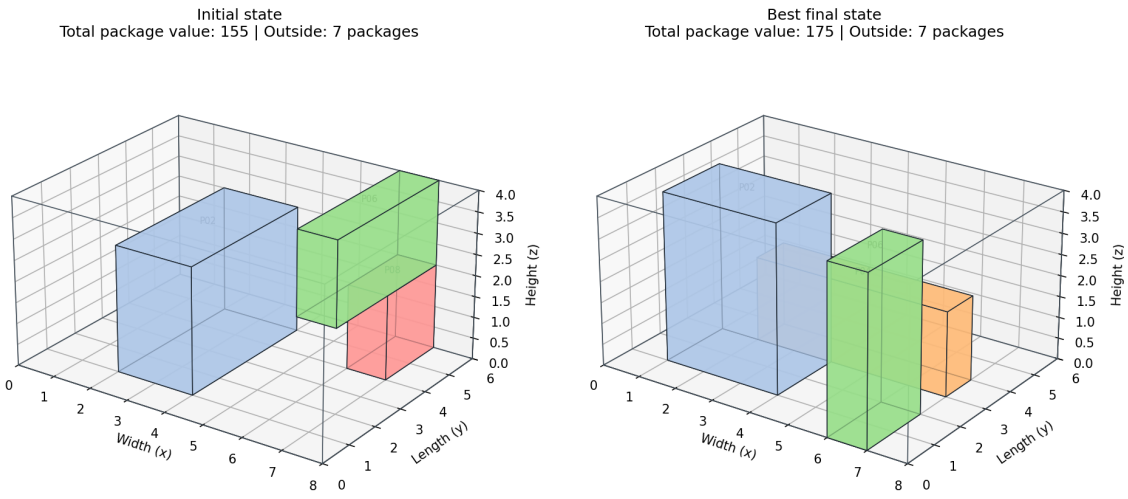


(c) Tidak ada pemuatan layak



(d) Paket hanya muat setelah rotasi

Gambar 4.20: State awal dan state terbaik yang dikembalikan GA untuk seed 11



Gambar 4.21: State awal dan state akhir pada kasus kapasitas truk ketat (seed 11)

Pengaruh parameter GA

Pengaruh ukuran populasi, jumlah generasi, dan laju mutasi diuji secara terpisah pada `base`. Setiap nilai parameter dijalankan dengan lima seed yang sama dan derivasi state awal yang sama. Parameter lain dipertahankan pada baseline. Tabel 4.16 melaporkan rata-rata nilai terbaik yang ditemukan, simpangan baku sampel, maksimum hasil run, durasi, dan jumlah evaluasi.

Tabel 4.16: Sweep satu faktor GA pada fixture `base` (lima seed per nilai parameter)

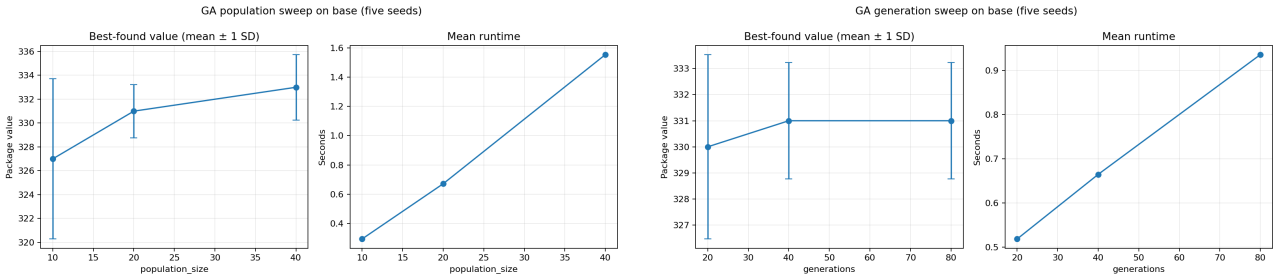
Parameter	Nilai	Rata-rata ± SD	Terbaik	Durasi (s)	Evaluasi
Generasi	20	330,00 ± 3,54	335	0,519	420
	40	331,00 ± 2,24	335	0,665	820
	80	331,00 ± 2,24	335	0,935	1.620
Populasi	10	327,00 ± 6,71	335	0,295	410
	20	331,00 ± 2,24	335	0,672	820
	40	333,00 ± 2,74	335	1,553	1.640
Mutation rate	0,00	330,00 ± 0,00	330	0,623	820
	0,05	331,00 ± 2,24	335	0,659	820
	0,20	328,00 ± 4,47	330	0,809	820

Ukuran populasi memberi tren kualitas yang paling jelas pada rentang yang diuji. Rata-rata naik dari 327 menjadi 333 saat populasi bertambah dari 10 ke 40, sementara durasi rata-rata meningkat sekitar lima kali (0,295 menjadi 1,553 detik). Namun, nilai terbaik yang pernah ditemukan tetap 335 pada ketiga ukuran populasi sehingga keuntungan yang terlihat adalah konsistensi rata-rata, bukan batas solusi baru. Karena hanya lima seed yang diuji, tren ini bersifat deskriptif dan tidak membuktikan perbedaan statistik.

Menambah generasi dari 20 menjadi 40 menaikkan rata-rata hanya satu poin (330 ke 331), sedangkan dari 40 ke 80 tidak menambah rata-rata maupun nilai terbaik. Durasi naik dari 0,665 menjadi 0,935 detik. Ini menunjukkan stagnasi hasil pada seed dan rentang generasi ini, bukan bahwa tambahan generasi tidak pernah berguna. Jumlah evaluasi bertambah proporsional dengan ukuran populasi dan banyak generasi karena setiap generasi mengevaluasi seluruh populasi.

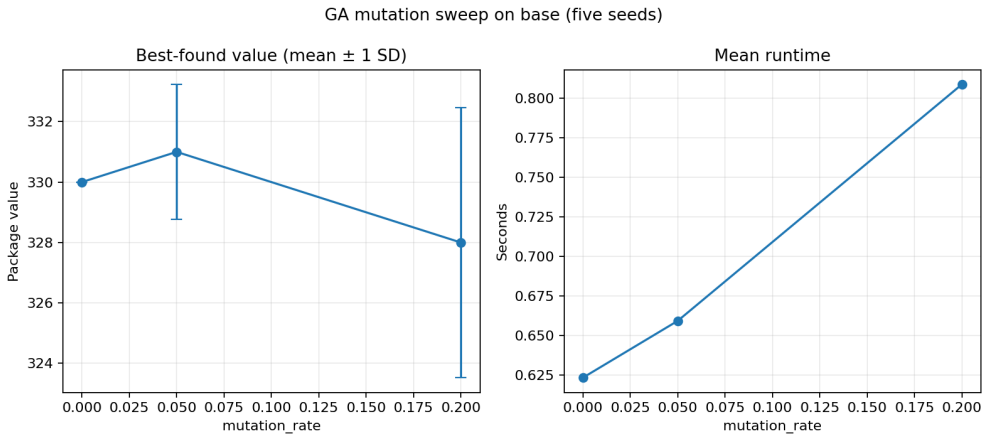
Laju mutasi 0,05 memberi rata-rata tertinggi (331) dan satu run mencapai 335. Tanpa mutasi seluruh lima run berakhir pada 330. Laju 0,20 menurunkan rata-rata menjadi 328 dan menghasilkan sebaran lebih lebar (SD 4,47). Pada instance ini, hasil konsisten dengan mutasi moderat membantu mempertahankan variasi yang berguna, sedangkan laju lebih tinggi dapat mengganggu kombinasi yang baik. Lima seed belum cukup untuk menyimpulkan hubungan umum.

Gambar 4.22 memperlihatkan hubungan kualitas dan biaya untuk ketiga sweep. Riwayat pencarian per run disimpan terpisah pada berkas JSON. Grafik di sini menekankan nilai akhir dan durasi agar dampak parameter mudah dibandingkan.



(a) Ukuran populasi

(b) Jumlah generasi



Gambar 4.22: Pengaruh parameter terhadap nilai terbaik dan durasi rata-rata. Batang galat menunjukkan ±1 SD

Eksperimen ini menggunakan satu fixture untuk sweep parameter dan lima seed sehingga hasilnya belum menunjukkan konfigurasi terbaik untuk semua kategori. Waktu yang dilaporkan adalah durasi algoritma pada lingkungan lokal yang sama, tetapi dapat berubah akibat beban mesin. Seluruh 100 keluaran per-run, parameter, riwayat objective, dan state awal/akhir disimpan di `output/ga-analysis/`. Eksperimen dapat direproduksi dengan menjalankan `python -m src.experiments.ga.results_analysis` dari root repositori.

4.3 Analisis Keseluruhan

4.3.1 Perbandingan Antar-Algoritma pada Seluruh Kategori

Seluruh 330 *run benchmark* menghasilkan *state* akhir yang dinyatakan valid oleh validator. Proporsi valid mencapai 100%. Ketiga puluh *run* pada `infeasible` berakhir dengan state kosong yang valid, tanpa paket termuat, dan bernilai 0. Perbandingan kualitas hanya mencakup *instance* yang dapat dimuat. Tabel 4.17 merangkum kasus dasar dan kendala ketat. Tabel 4.18 merangkum kasus batas.

Instance	Terbaik	*Steepest*	*Sideways*	*Stochastic*	*Restart*	SA	GA
`base`	335	303,0 (27,5)	317,0 (20,5)	305,0 (26,2)	331,0 (6,5)	329,0 (8,2)	331,0 (2,2)
`tight-space`	300	242,0 (26,8)	255,0 (35,0)	246,0 (34,4)	280,0 (16,2)	282,0 (8,4)	274,0 (10,8)
`tight-capacity`	175	158,0 (11,5)	166,0 (2,2)	162,0 (5,7)	175,0 (0)	173,0 (2,7)	174,0 (2,2)

Tabel 4.17: Rata-rata (simpangan baku) nilai akhir `total_value`, lima *seed*

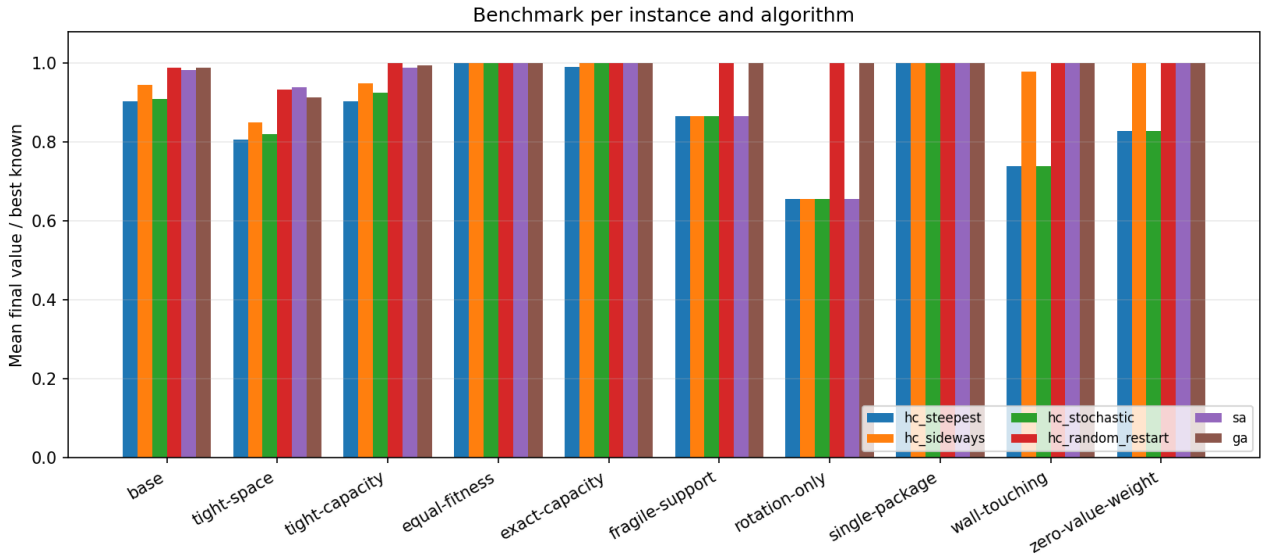
Frekuensi mencapai nilai terbaik (dari lima *seed*) pada `base` adalah 1, 2, 1, 3, 3, dan 1 untuk *steepest*, *sideways*, *stochastic*, *random restart*, SA, dan GA. Pada `tight-space` adalah 0, 1, 1, 1, 0, dan 0. Dan pada `tight-capacity` adalah 0, 0, 0, 5, 3, dan 4.

Instance	Terbaik	*Steepest*	*Sideways*	*Stochastic*	*Restart*	SA	GA
`single-package`	50	50	50	50	50	50	50
`equal-fitness`	40	40	40	40	40	40	40
`exact-capacity`	100	99	100	100	100	100	100
`wall-touching`	100	74	98	74	100	100	100
`zero-value-weight`	70	58	70	58	70	70	70
`fragile-support`	90	78	78	78	90	78	90
`rotation-only`	70	46	46	46	70	46	70

Tabel 4.18: Rata-rata nilai akhir `total_value` pada kasus batas, lima *seed*

	Steepest	*Sideways*	*Stochastic*	*Restart*	SA	GA
Evaluasi objektif (rata-rata)	43	3085	1001	383	3001	820
Durasi (s)	0,0056	0,4699	0,1432	0,1543	0,5654	0,4746
Durasi per evaluasi (ms)	0,13	0,15	0,14	0,40	0,19	0,58

Tabel 4.19: Biaya pencarian pada `base` (lima *seed*)



Gambar 4.23: Rata-rata nilai akhir terhadap nilai terbaik pada tiap *instance* dan algoritma

Beberapa pola konsisten muncul.

1. **Kualitas.** Pada `base` dan `tight-capacity`, rata-rata *random restart*, SA, dan GA berbeda sekitar 1%. Rata-rata *steepest*, *sideways*, dan *stochastic* tertinggal 5–10%. Pada `tight-space`, SA memperoleh 282, sedikit di atas *random restart* dengan 280 dan GA dengan 274. Tidak ada metode yang mencapai rata-rata 300. *Steepest*, SA, dan GA juga tidak pernah mencapai 300 dalam lima *seed*. Kesulitan pada ruang kecil tampaknya berasal dari penataan geometris, bukan pemilihan paket, karena batas *knapsack* sebesar 335 jauh di atas 300. Selisih rata-rata antara ketiga metode tersebut lebih kecil daripada simpangan bakunya. Lima *seed* belum cukup untuk menyatakan perbedaan yang nyata.

2. **Konsistensi.** GA memiliki simpangan baku terkecil pada `base` (2,2) dan SA pada `tight-space` (8,4). Tiga varian pertama *hill climbing* memiliki simpangan baku terbesar pada `base` dan `tight-space` (20–35). *Random restart* menurunkan simpangan baku varian dasarnya (27,5 menjadi 6,5 pada `base`, dan 11,5 menjadi 0 pada `tight-capacity`). Pada `tight-capacity`, *random restart* mencapai optimum 175 pada kelima *seed*.

3. **Biaya.** *Steepest* paling murah dengan 43 evaluasi dan durasi 0,006 detik. *Random restart* memakai 383 evaluasi, sekitar seperdelapan dari SA yang memakai 3001 dan setengah dari GA yang memakai 820, dengan kualitas yang sebanding. Durasi per evaluasi GA sekitar empat kali durasi *steepest* karena juga mencakup pembentukan populasi, *crossover*, perbaikan, dan validasi. Perbandingan biaya perlu melihat durasi bersama jumlah evaluasi.

4. **Ketahanan terhadap *local optimum*.** Pada `base`, perbedaan hasil muncul dari cara algoritma menangani state awal yang buruk. Varian HC yang hanya menerima gerakan setara atau lebih baik dapat tersangkut. SA dapat keluar dengan menerima gerakan yang memburuk saat suhu tinggi. *Random restart* dan GA dapat memulai pencarian dari state acak lain.

4.3.2 Ketahanan pada Kasus Batas dan Masukan Tidak Valid

Tanpa pemuatan layak. Seluruh 30 *run* pada `infeasible` mengembalikan *state* valid tanpa paket termuat dan nilai 0 sehingga representasi yang mengizinkan semua paket berada di luar truk bekerja seperti yang dirancang. Karena nilai 0 saja ambigu bila ada paket bernilai nol, jumlah paket termuat dicatat bersamaan (0 pada seluruh *run*).

Kasus batas. Kasus `single-package` dan `equal-fitness` tidak membedakan algoritma karena *state* awal acak sudah optimal (50 dan 40). Keduanya hanya menunjukkan bahwa seleksi *roulette-wheel* dengan fitness yang sama dan satu paket tidak menimbulkan galat. Kasus lain membedakan algoritma dengan jelas.

- `rotation-only`. *Random restart* dan GA mencapai 70 pada kelima *seed*. Empat metode lain hanya mencapai 70 pada dua *seed* yang state awalnya sudah memuat paket panjang. Pada tiga state awal lainnya, rata-ratanya tetap 46. Operator *move* tidak mengeluarkan paket dari truk. Operator *swap* memberi orientasi awal $(0, 1, 2)$ kepada paket dari luar sehingga paket $2 \times 2 \times 5$ tidak muat. Paket panjang tidak dapat dimasukkan selama paket kubus berada di dalam. Karena itu, dari state tersebut tidak ada jalur menuju state yang lebih baik dengan operator yang tersedia. Ini merupakan batasan operator *neighbor*. *Random restart* dan GA dapat menghindarinya dengan membangkitkan state awal baru.
- `fragile-support`. *Random restart* dan GA mencapai 90 pada seluruh *seed* dengan menempatkan paket rapuh di atas paket yang tidak rapuh. Empat metode lain mendapat nilai 60 pada dua dari lima *seed* dan rata-rata 78. Ketiganya mencapai 90 pada tiga *seed* lainnya.
- `wall-touching`. *Random restart*, SA, dan GA mengisi truk hingga tepat menyentuh dinding dan mendapat nilai 100 pada seluruh *seed*. Nilai rata-rata *sideways* adalah 98, sedangkan *steepest* dan *stochastic* memperoleh 74. Validator menerima penempatan yang menyentuh dinding pada seluruh *run*.
- `exact-capacity`. Seluruh algoritma mencapai 100 (berat tepat sama dengan kapasitas) kecuali satu *run* *steepest* pada 95 sehingga batas "$\leq$" pada kapasitas ditangani benar.
- `zero-value-weight`. *Steepest* dan *stochastic* berakhir pada nilai 10 pada satu *seed*, yaitu seed 13. Paket berbobot nol termuat tanpa paket `regular`. Algoritma lain selalu memperoleh nilai 70. Paket bernilai nol atau berbobot nol tidak menyebabkan galat maupun state tidak valid.

Masukan tidak valid. Seluruh delapan *fixture* pada `invalid/` ditolak oleh *loader* dengan `ValueError` dan pesan yang menyebut penyebabnya (Tabel 4.20). Tidak ada yang lolos ke pencarian.

Fixture	Pesan galat
`package-dimension-nonpositive`	*package dimensions must contain three positive integers*
`package-dimension-length`	*packages[0].dimensions must be a list of three integers*
`package-dimension-noninteger`	*packages[0].dimensions must be a list of three integers*
`truck-dimension-nonpositive`	*truck dimensions must contain three positive integers*
`truck-capacity-negative`	*truck capacity must be a non-negative integer*
`duplicate-ids`	*package IDs must be unique*
`empty-packages`	*problem must contain at least one package*
`missing-field`	*packages[0] is missing 'eta'*

Tabel 4.20: Penolakan masukan tidak valid oleh *loader*

4.3.3 Pengaruh Variasi *Objective Function*

Instance	Algoritma	Nilai total paket rata-rata			Durasi (s)	
		`total`	`eta`	`compact`	`total`	`compact`
`base`	*Hill climbing*	321,67	304,17	324,58	0,198	0,176
	SA	330	305	331,67	0,390	0,568
	GA	331,39	296,94	331,39	0,447	0,441
`tight-capacity`	*Hill climbing*	165	153,75	165,83	0,080	0,084
	SA	173,33	170	171,67	0,285	0,314
	GA	174,17	166,94	174,17	0,397	0,316

Tabel 4.21: Nilai total paket pada *state* akhir menurut *objective* yang dioptimalkan (`total` = `total_value`, `eta` = `eta_weighted_value`, `compact` = `compact_value`). Durasi untuk `total_value` dan `compact_value`

Nilai total paket pada Tabel 4.21 dihitung ulang dengan `total_value` untuk seluruh *state* akhir sehingga ketiga *objective* dapat dibandingkan pada satu skala. `eta_weighted_value` jelas mengubah himpunan paket. Pada kedua *instance* dan ketiga algoritma nilai total turun (misalnya GA pada `base` dari 331,39 menjadi 296,94, dan SA dari 330 menjadi 305), sedangkan skor ETA yang dioptimalkan naik (SA pada `base` dari 93,45 menjadi 116,33). `compact_value` tidak mengorbankan nilai total dan nilai akhirnya sama dengan atau sedikit di atas `total_value`. Selisih kecil pada HC dan SA lebih mungkin berasal dari gradien kepadatan yang membantu pencarian melewati dataran (Tabel 4.5) daripada dari perubahan himpunan paket. Pada GA `compact_value` memberi nilai total yang identik dengan `total_value` (331,39 dan 174,17).

Efek `compact_value` pada penataan terukur pada SA. Kepadatan rata-rata naik dari 0,250 menjadi 0,375 pada `base` dan dari 0,305 menjadi 0,614 pada `tight-capacity`. Biaya komputasi ketiga *objective* sebanding pada HC dan GA (selisih durasi rata-rata hingga sekitar 25% pada kedua *instance*), tetapi SA dengan `compact_value` lebih lambat pada `base` (0,568 dibanding 0,390 detik), perbedaan yang tidak terlihat pada `tight-capacity` (0,314 dibanding 0,285 detik) dan belum ditelusuri penyebabnya.

4.3.4 Keterbatasan

Seluruh *instance* bersifat sintetis dan kecil (1–10 paket), dan konfigurasi per algoritma hanya diulang tiga kali (lima kali pada *benchmark* dan ablasi). Akibatnya selisih yang kecil, seperti antara *random restart*, SA, dan GA pada `base`, tidak dapat dibedakan dari *noise*. Parameter tiap algoritma (suhu awal, laju pendinginan, ukuran populasi, *mutation rate*, jumlah tetangga per iterasi) tidak disetel, hanya satu konfigurasi yang dipakai pada *benchmark* sehingga urutan algoritma berlaku untuk konfigurasi tersebut. Optimum sebenarnya hanya diketahui untuk `base` dan `tight-capacity` (lewat batas atas *knapsack*). Untuk `tight-space` nilai "terbaik" hanyalah yang tertinggi yang ditemukan. Hasil pada data paket kelompok yang sesungguhnya, jika ada dan berbeda, perlu ditambahkan pada bagian ini.

4.4 Analisis Implementasi Spesifikasi Bonus

4.4.1 *Extra* Objective Function

Dua fungsi objektif tambahan, `eta_weighted_value` dan `compact_value`, diuji dengan algoritma, *seed*, dan *state* awal yang sama seperti `total_value` (Tabel 4.21).

1. `eta_weighted_value` **mengubah pilihan paket saat nilai dan urgensi berbeda.** Pada `base`, nilai total paket turun dari 321,67 menjadi 304,17 untuk HC, dari 330 menjadi 305 untuk SA, dan dari 331,39 menjadi 296,94 untuk GA. Skor ETA SA naik dari 93,45 menjadi 116,33. Pada `tight-capacity`, nilai HC turun dari 165 menjadi 153,75, SA dari 173,33 menjadi 170, dan GA dari 174,17 menjadi 166,94. Skor objektif ini lebih kecil, dengan maksimum 116,33 pada `base` dan 75,83 pada `tight-capacity`. Parameter yang bergantung pada skala, seperti suhu awal SA, mungkin perlu disesuaikan. Jumlah kejadian *stuck* SA meningkat dari 2 menjadi 4 pada `base`, tetapi pengaruh T_0 belum diuji.
2. `compact_value` **terutama mengubah penataan.** Nilai total paket sama dengan atau sedikit di atas

`total_value`, kepadatan SA naik (0,250 menjadi 0,375 pada `base`, 0,305 menjadi 0,614 pada `tight-capacity`), dan dinamika pencarian berubah. Pada HC, *sideways* naik dari 318,33 menjadi 328,76 dan *stochastic* dari 318,33 menjadi 322,03 karena bonus memberi gradien pada dataran.

3. **Biaya komputasi umumnya tidak berubah.** Durasi ketiga fungsi objektif berbeda hingga sekitar 25% pada HC dan GA di kedua *instance*, tanpa arah yang konsisten. Pengecualiannya adalah SA dengan `compact_value` pada `base`, yang berjalan sekitar 46% lebih lama.

4.4.2 All-variant Hill Climbing Algorithm

Empat varian *hill climbing* dibandingkan pada *seed* dan *state* awal yang sama. Tabel 4.22 merangkum hasil `total_value` pada dua *instance*. Hasil dua *objective* lain untuk `base` ada pada Tabel 4.4 dan Tabel 4.5.

	`base`			`tight-capacity`	
Varian	Rata-rata	Iterasi	Durasi (s)	Rata-rata	Iterasi
Steepest	316,67	0,33	0,0053	160	0,33
Sideways	318,33	100,67	0,4646	165	101
Stochastic	318,33	1000	0,1703	160	1000
Random restart	333,33	1,67	0,1535	175	2,33

Tabel 4.22: Perbandingan empat varian *hill climbing* dengan `total_value` (rata-rata 3 *seed*)

1. ***Random restart* memberi hasil tertinggi pada lima dari enam kombinasi *instance* dan *objective*.** Rata-ratanya 333,33 dan 331,98 pada `base` untuk `total_value` dan `compact_value`. Pada `tight-capacity`, nilainya 175, 75,83, dan 175,49 untuk ketiga fungsi objektif. Pengecualiannya adalah `eta_weighted_value` pada `base`, ketika *sideways* memperoleh 116,33 dan *random restart* memperoleh 114,89. Sepuluh state acak tambahan memberi peluang bagi *random restart* untuk memulai dari cekungan lain. Pada `tight-capacity`, metode ini mencapai optimum 175 pada ketiga *seed*.

2. ***Sideways move* membantu bila ada dataran yang dapat dilalui, dengan biaya besar.** Pada `total_value`, hasilnya hampir sama dengan *steepest* di `base` (318,33 dibanding 316,67), tetapi lebih baik di `tight-capacity` (165 dibanding 160), `eta_weighted_value` (116,33 dibanding 103,23 pada `base`), dan `compact_value` (328,76 dibanding 316,91 pada `base`). Durasinya 56–88 kali durasi *steepest* karena selalu menghabiskan 100 *sideways move*.

3. ***Stochastic* setara *steepest* dengan biaya lebih tinggi.** Karena hanya mencoba satu *neighbor* per iterasi dan selalu menjalankan 1000 iterasi, varian ini sekitar 32 kali lebih lambat daripada *steepest* pada `base`, tetapi hanya sedikit lebih baik (318,33 dibanding 316,67) atau sama (160 pada `tight-capacity`), dan lebih lemah daripada *sideways* pada `eta_weighted_value` (110,89 dibanding 116,33).

4. ***Steepest* termurah** (0,0053 detik pada `base`) dan cukup bila *state* awal berada di cekungan yang baik, tetapi tidak melindungi dari *state* awal yang buruk (nilai akhir pada *seed* 12 tetap 285).

Steepest dan *sideways* pada implementasi ini memakai sampel 30 *neighbor* acak per iterasi, bukan seluruh ruang *neighbor* sehingga hasil di atas hanya berlaku untuk pengaturan tersebut.

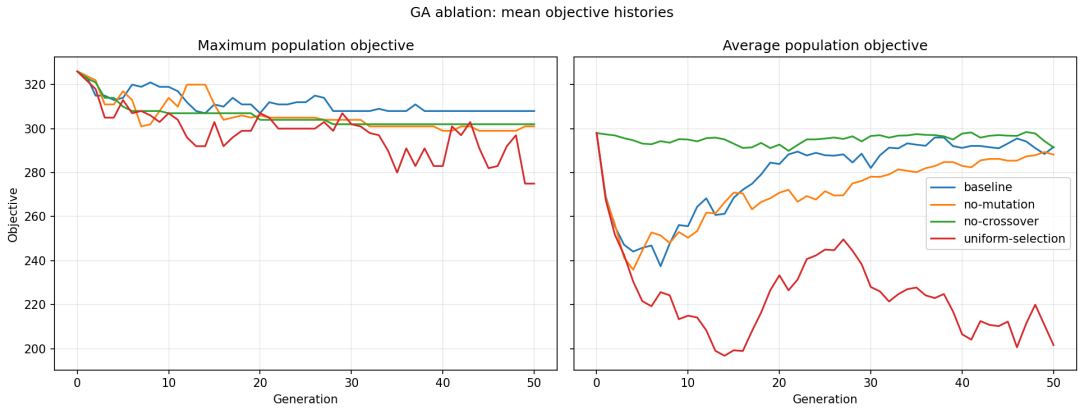
Perbandingan perlu mempertimbangkan jumlah pendakian. *Random restart* menjalankan 11 pendakian, yaitu satu dari state awal dan sepuluh dari state acak. Varian lain hanya menjalankan satu pendakian. Karena itu, sebagian keunggulannya dapat berasal dari anggaran pencarian yang lebih besar. Durasinya tetap rendah, yaitu 0,1535 detik pada `base`, karena sebagian besar pendakian segera berhenti. Waktu *sideways* yang tinggi sebagian dapat disebabkan oleh gerakan yang tidak mengubah state. Fungsi `random_neighbor` mengembalikan state asal bila tidak menemukan kandidat valid, kandidat tersebut bernilai sama, dan ia memakai jatah *sideways* tanpa menjelajahi state baru. Seberapa sering hal ini terjadi belum diukur, sehingga manfaat *sideways* pada Tabel 4.22 dapat lebih kecil daripada yang ditunjukkan oleh batas 100 langkah.

4.4.3 Studi Ablasi Operator GA

Ablasi dilakukan pada `base` dengan `total_value`, lima *seed* (11–15), populasi 20, dan 50 generasi. Setiap varian hanya mengubah satu komponen terhadap *baseline* (*roulette-wheel*, *single-point crossover*, mutasi 0,05). Tanpa mutasi, tanpa *crossover*, dan seleksi seragam sebagai pembanding *roulette-wheel*.

Varian	Rata-rata	Simpangan baku	Nilai per *seed* (11–15)	Durasi (s)
Baseline	331	2,24	330, 335, 330, 330, 330	0,377
Tanpa mutasi	330	0	330, 330, 330, 330, 330	0,393
Tanpa *crossover*	326	5,48	330, 320, 330, 330, 320	0,196
Seleksi seragam	327	6,71	335, 320, 330, 330, 320	0,345

Tabel 4.23: Hasil ablasi operator GA pada `base`



Gambar 4.24: Ablasi operator GA. Nilai maksimum dan rata-rata populasi per generasi

1. **Dampak mutasi kecil pada percobaan ini.** Tanpa mutasi, rata-rata turun 1 poin dari 331 menjadi 330 dan seluruh *run* berakhir pada 330. Baseline mencapai 335 pada seed 12. Lima seed belum cukup untuk memastikan bahwa mutasi menyebabkan perbedaan tersebut.
2. ***Crossover* dan seleksi *roulette-wheel* tampak memberi manfaat kecil.** Tanpa *crossover* atau dengan seleksi seragam, rata-rata turun 4–5 poin dan dua dari lima *run* berakhir pada 320, sementara *baseline* tidak pernah di bawah 330. Namun dengan lima *seed* dan simpangan baku 5,5–6,7, selisih ini belum memenuhi syarat untuk disebut signifikan.
3. **Biaya.** Menghilangkan *crossover* memangkas durasi hampir separuh (0,196 dibanding 0,377 detik), yang menunjukkan bahwa *crossover* beserta perbaikan (*repair*) konflik adalah bagian termahal dari satu generasi.

4.4.4 Multi-Truck

Fixture Armada dan Protokol

Eksperimen memakai 11 fixture sintetis yang mengikuti kategori pengujian sebelumnya. Setiap fixture mempertahankan paket dan spesifikasi truk sumber, lalu menyediakan dua truk identik. Perbedaan hasil terutama mencerminkan pembagian paket ke lebih dari satu truk, bukan perubahan ukuran atau kapasitas truk. Fixture mencakup kasus dasar, ruang dan kapasitas ketat, kasus tanpa pemuatan layak, serta tujuh kasus batas.

Setiap kombinasi fixture dan seed dijalankan dengan empat varian HC, SA, dan GA. Eksperimen memakai lima seed, yaitu 11 sampai 15 sehingga menghasilkan $11 \times 5 \times 6 = 330$ run. Untuk tiap fixture dan seed, seluruh metode menerima state awal yang sama. Seed pembentuk state awal diperoleh dengan XOR seed run dengan konstanta tetap `0x5EED`. Objective yang digunakan adalah `total_value`.

Parameter mengikuti baseline runner armada. HC memakai 30 neighbor per iterasi, paling banyak 1000 iterasi, 100 sideways move, dan 10 restart. SA memakai 2000 iterasi, temperatur awal 100, laju pendinginan 0,995, dan patience 100 iterasi. GA memakai populasi 20, 40 generasi, dan laju mutasi 0,05. Anggaran komputasi ketiga algoritma tidak setara. Perbandingan ini menggambarkan perilaku konfigurasi yang diuji dan tidak menyimpulkan keunggulan algoritma pada biaya komputasi yang sama.

State Awal sebagai Baseline

State awal dianalisis sebelum hasil akhir dibandingkan. Untuk setiap fixture, ringkasan memakai satu state per seed, bukan satu salinan untuk setiap algoritma. Analisis mencatat validitas, nilai paket yang telah dimuat, jumlah paket termuat dan belum termuat, serta jumlah dan nilai paket pada masing-masing truk. Perubahan nilai dan jumlah paket pada akhir pencarian kemudian dihitung terhadap baseline ini.

Tabel 4.24: Karakteristik state awal per fixture, rata-rata dan simpangan baku sampel dari lima seed

Fixture	Nilai awal ± SD	Paket termuat	Truk 1 paket/nilai	Truk 2 paket/nilai
`base`	568,0 ± 43,8	9,6	4,4 / 277,0	5,2 / 291,0
`tight-space`	355,0 ± 60,3	6,8	3,4 / 175,0	3,4 / 180,0
`tight-capacity`	300,0 ± 0,0	6,0	3,2 / 148,0	2,8 / 152,0
`infeasible`	0,0 ± 0,0	0,0	0,0 / 0,0	0,0 / 0,0
`equal-fitness`	60,0 ± 0,0	6,0	3,0 / 30,0	3,0 / 30,0
`exact-capacity`	125,0 ± 0,0	4,0	1,8 / 55,0	2,2 / 70,0
`fragile-support`	100,0 ± 0,0	3,0	1,4 / 36,0	1,6 / 64,0
`rotation-only`	100,0 ± 0,0	2,0	1,0 / 46,0	1,0 / 54,0
`single-package`	50,0 ± 0,0	1,0	0,4 / 20,0	0,6 / 30,0
`wall-touching`	100,0 ± 0,0	4,0	2,0 / 56,0	2,0 / 44,0
`zero-value-weight`	70,0 ± 0,0	4,0	1,8 / 28,0	2,2 / 42,0

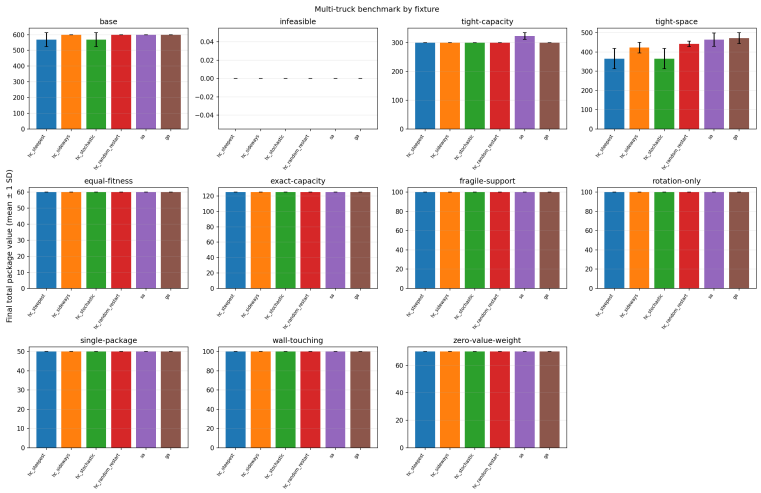
Pada `base`, state awal sudah memuat rata-rata 9,6 dari 10 paket dengan nilai 568. Dua seed yang nilainya 520 belum memuat paket P10. State awal `tight-space` lebih bervariasi, dengan nilai rata-rata 355 dan simpangan baku 60,3. Pada `tight-capacity`, seluruh seed mulai dari nilai 300 dan enam paket termuat. Pembagian muatan awal di dua truk relatif seimbang pada ketiga fixture tersebut. Karena state awal identik untuk semua metode pada seed yang sama, perbedaan hasil akhir bukan akibat perbedaan inisialisasi antaralgoritma.

Hasil Lintas Fixture

Tabel 4.25 membandingkan rata-rata dan simpangan baku nilai akhir untuk setiap metode. Nilai awal pada Tabel 4.24 menjadi acuan untuk membaca perbaikan. Kasus `infeasible` hanya digunakan untuk memeriksa validitas keluaran, bukan untuk menentukan metode terbaik. Nilai maksimum yang muncul pada eksperimen merupakan hasil terbaik yang teramati dan tidak otomatis menjadi optimum global.

Tabel 4.25: Nilai akhir rata-rata dan simpangan baku sampel dari lima seed, jumlah paket rata-rata dalam tanda kurung

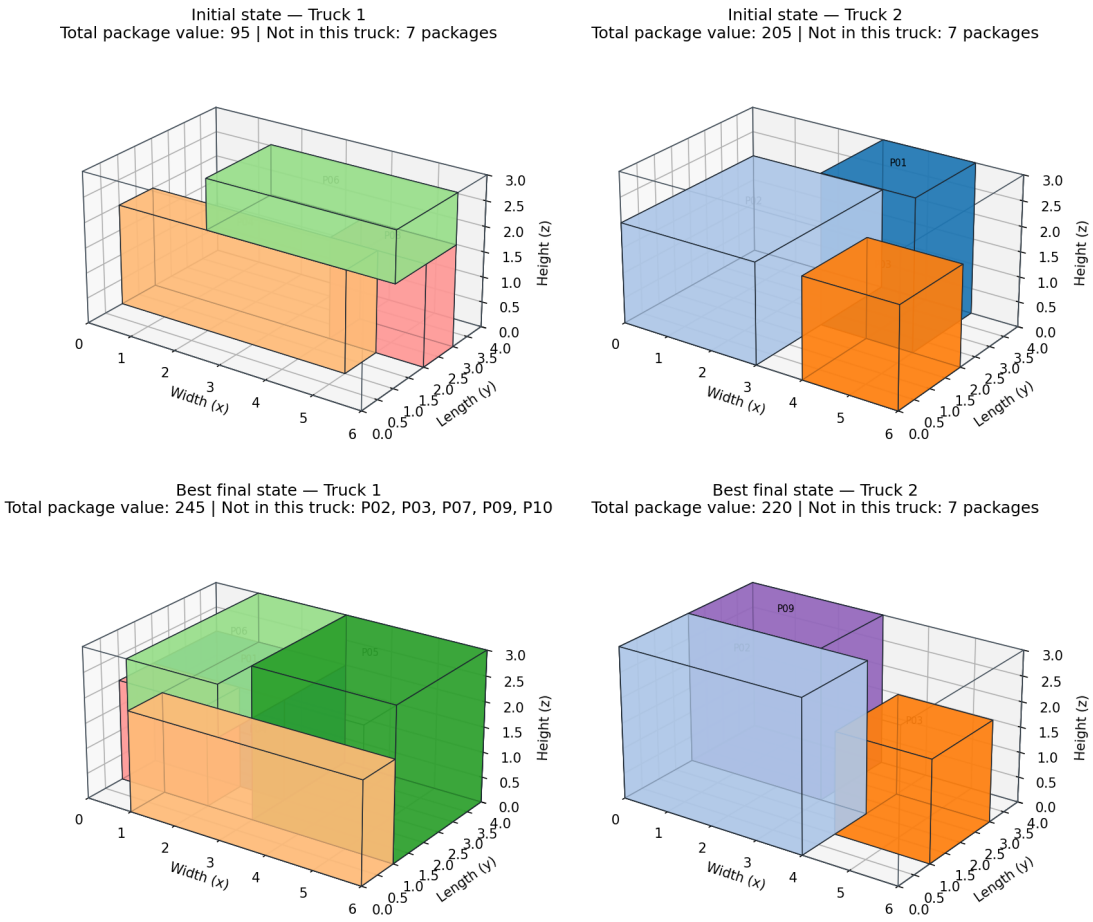
Fixture	HC steepest	HC sideways	HC stochastic	HC random restart	SA	GA
`base`	568,0 ± 43,8 (9,6)	600,0 ± 0,0 (10,0)	568,0 ± 43,8 (9,6)	600,0 ± 0,0 (10,0)	600,0 ± 0,0 (10,0)	600,0 ± 0,0 (10,0)
`tight-space`	366,0 ± 52,2 (7,0)	423,0 ± 27,5 (7,8)	366,0 ± 52,2 (7,0)	442,0 ± 14,0 (8,0)	464,0 ± 34,5 (8,2)	472,0 ± 28,2 (8,2)
`tight-capacity`	300,0 ± 0,0 (6,0)	300,0 ± 0,0 (6,0)	300,0 ± 0,0 (6,0)	300,0 ± 0,0 (6,0)	323,0 ± 11,5 (6,0)	300,0 ± 0,0 (6,0)
`infeasible`	0,0 ± 0,0 (0,0)	0,0 ± 0,0 (0,0)	0,0 ± 0,0 (0,0)	0,0 ± 0,0 (0,0)	0,0 ± 0,0 (0,0)	0,0 ± 0,0 (0,0)
`equal-fitness`	60,0 ± 0,0 (6,0)	60,0 ± 0,0 (6,0)	60,0 ± 0,0 (6,0)	60,0 ± 0,0 (6,0)	60,0 ± 0,0 (6,0)	60,0 ± 0,0 (6,0)
`exact-capacity`	125,0 ± 0,0 (4,0)	125,0 ± 0,0 (4,0)	125,0 ± 0,0 (4,0)	125,0 ± 0,0 (4,0)	125,0 ± 0,0 (4,0)	125,0 ± 0,0 (4,0)
`fragile-support`	100,0 ± 0,0 (3,0)	100,0 ± 0,0 (3,0)	100,0 ± 0,0 (3,0)	100,0 ± 0,0 (3,0)	100,0 ± 0,0 (3,0)	100,0 ± 0,0 (3,0)
`rotation-only`	100,0 ± 0,0 (2,0)	100,0 ± 0,0 (2,0)	100,0 ± 0,0 (2,0)	100,0 ± 0,0 (2,0)	100,0 ± 0,0 (2,0)	100,0 ± 0,0 (2,0)
`single-package`	50,0 ± 0,0 (1,0)	50,0 ± 0,0 (1,0)	50,0 ± 0,0 (1,0)	50,0 ± 0,0 (1,0)	50,0 ± 0,0 (1,0)	50,0 ± 0,0 (1,0)
`wall-touching`	100,0 ± 0,0 (4,0)	100,0 ± 0,0 (4,0)	100,0 ± 0,0 (4,0)	100,0 ± 0,0 (4,0)	100,0 ± 0,0 (4,0)	100,0 ± 0,0 (4,0)
`zero-value-weight`	70,0 ± 0,0 (4,0)	70,0 ± 0,0 (3,8)	70,0 ± 0,0 (4,0)	70,0 ± 0,0 (4,0)	70,0 ± 0,0 (4,0)	70,0 ± 0,0 (4,0)



Gambar 4.25: Nilai akhir rata-rata per fixture dan metode dengan batang galat satu simpangan baku sampel

Perbedaan antarmetode tampak terutama pada tiga fixture utama. Pada `base`, empat metode mencapai nilai 600 pada seluruh seed kecuali HC steepest dan HC stochastic, yang tidak memperbaiki dua state awal bernilai 520. Karena total nilai seluruh paket adalah 600 dan seluruh paket termuat pada hasil bernilai 600, nilai tersebut merupakan batas atas yang diketahui untuk fixture ini. HC sideways dan random restart, SA, serta GA mencapai batas itu pada kelima seed.

Pada `tight-space`, GA memiliki rata-rata tertinggi sebesar 472, disusul SA sebesar 464 dan HC random restart sebesar 442. GA dan SA sama-sama mencapai nilai terbaik teramati 520, tetapi sebaran hasil SA lebih besar. HC steepest dan stochastic menghasilkan rata-rata 366, sama dengan nilai awal rata-ratanya. Random restart memberi hasil lebih baik daripada tiga varian HC lain pada fixture ini, sementara sideways move membantu HC sideways memperbaiki sebagian seed.



Gambar 4.26: State awal dan state akhir GA pada fixture `tight-space` untuk seed 11

Pada `tight-capacity`, nilai awal semua seed adalah 300. Empat varian HC dan GA tetap pada nilai itu, sedangkan SA mencapai rata-rata 323 dengan simpangan baku 11,5. Jumlah paket yang dimuat tetap enam pada seluruh hasil. Kenaikan nilai SA berasal dari perubahan paket yang dipilih, bukan dari bertambahnya jumlah paket. Pola ini sesuai dengan kemampuan SA menerima perpindahan yang sementara menurunkan nilai saat temperatur masih tinggi.

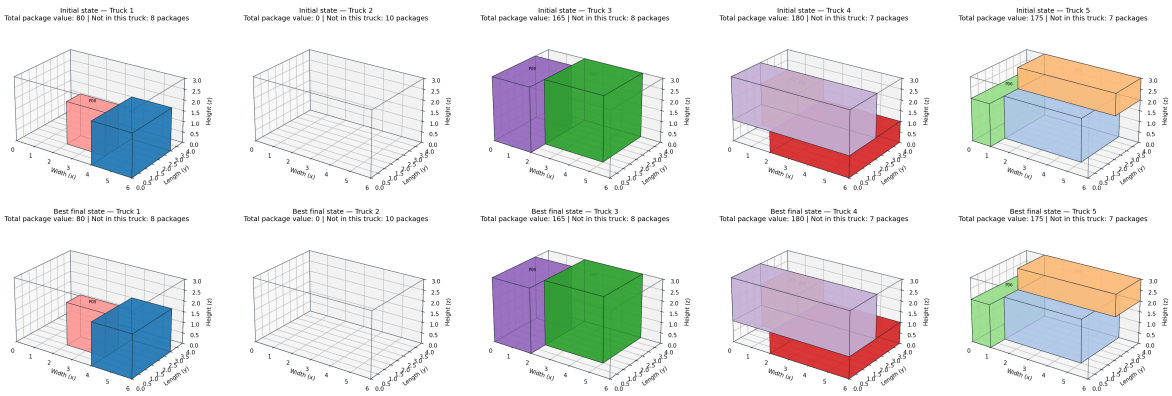
Eksperimen Jumlah Truk

Untuk melihat dampak penambahan armada, spesifikasi truk identik diulang menjadi tiga, empat, dan lima truk pada fixture `base`, `tight-space`, dan `tight-capacity`. Keenam metode memakai state awal yang sama dalam setiap pasangan fixture dan jumlah truk. Eksperimen ini memakai satu seed (11) sehingga tabel berikut berisi nilai observasi tunggal, bukan rata-rata atau ukuran variasi.

Tabel 4.26: Nilai dan jumlah paket pada state awal serta rentang nilai akhir dan paket termuat dari enam metode dengan seed 11

Fixture	Jumlah truk	State awal (nilai/paket)	Akhir enam metode (nilai dan paket)
`base`	3	600 / 10	600 dan 10
	4	600 / 10	600 dan 10
	5	600 / 10	600 dan 10
`tight-space`	3	520 / 9	520–600 dan 9–10
	4	600 / 10	600 dan 10
	5	600 / 10	600 dan 10
`tight-capacity`	3	445 / 8	445–470 dan 8
	4	445 / 8	445–600 dan 8–10
	5	520 / 9	520–600 dan 9–10

Pada `base`, seluruh metode mencapai nilai 600 untuk ketiga jumlah armada. Pada `tight-space`, hasil seed ini mencapai 600 untuk semua metode mulai empat truk. Dengan tiga truk, nilai akhir berada pada 520–600. Pada `tight-capacity`, rentang nilai akhir melebar saat empat truk (445–600), lalu menjadi 520–600 saat lima truk. Hasil tersebut menunjukkan bahwa kapasitas tambahan dapat membantu pada kasus ini, tetapi tidak selalu memperbaiki hasil setiap metode. Karena state awal juga berubah antarjumlah truk dan hanya satu seed yang digunakan, pola ini bersifat deskriptif dan tidak mengisolasi pengaruh jumlah truk secara statistik.



Gambar 4.27: State awal dan state akhir GA pada `tight-space`, lima truk, seed 11

Visualisasi state awal dan akhir untuk keenam metode pada setiap jumlah truk tersedia di `output/fleet-count-analysis/representative-states/`. Seluruh 54 run menghasilkan state valid. Benchmark armada dua truk tetap tersimpan terpisah sebagai pembanding.

Tujuh fixture batas menghasilkan nilai akhir yang sama untuk seluruh metode dan seed. Kesamaan ini memperlihatkan bahwa kasus batas tersebut sudah terjangkau dari state awal yang dibangkitkan. Seluruh 55 state awal dan 330 state akhir lolos validasi. Pada `infeasible`, seluruh 30 run mengembalikan state kosong yang valid. Kasus tersebut memeriksa penanganan paket yang tidak dapat dimuat dan tidak masuk peringkat kualitas.

Biaya Pencarian dan Karakteristik Algoritma

Biaya pencarian berbeda walaupun semua metode memakai state awal berpasangan. Pada `base`, HC steepest menggunakan rata-rata 31 evaluasi objektif dan selesai dalam 0,031 detik. HC stochastic menggunakan 1001 evaluasi dan 1,211 detik. HC sideways memakai 3043 evaluasi dan 3,103 detik, sedangkan random restart menggunakan 359 evaluasi dan 0,535 detik. SA menggunakan 2001 evaluasi dengan durasi 2,225 detik. GA memakai 820 evaluasi, tetapi durasi 2,436 detik. Jumlah evaluasi saja tidak menggambarkan biaya operasi setiap algoritma sehingga durasi dilaporkan bersamanya.

HC steepest dan stochastic berakhir pada nilai awal pada `base` untuk semua seed. Dua state awal bernilai 520 tidak membaik, sementara sideways dan random restart mencapai nilai 600 pada kedua seed tersebut. Sideways

menghabiskan lebih banyak evaluasi untuk menerima gerakan setara. Random restart menemukan hasil yang sama dengan biaya evaluasi dan waktu lebih rendah pada konfigurasi ini. Pola ini tidak membuktikan bahwa random restart selalu lebih efisien pada instance lain.

SA memberi hasil tertinggi pada `tight-capacity`, dengan rata-rata 323 dibanding 300 untuk metode lain. Jumlah paketnya tetap enam. Pada kelima seed, paket yang dipilih SA berbeda dari state awal. Riwayat SA mencatat nilai state saat ini turun di bawah 300 pada semua seed, hingga serendah 115, sebelum menghasilkan state terbaik dengan nilai 310 sampai 335. Pada `tight-space`, SA mencapai rata-rata 464 dan GA 472. Kedua algoritma pernah menghasilkan nilai 520, sedangkan simpangan baku GA lebih kecil. Lima seed belum cukup untuk menyebut selisih tersebut signifikan.

GA mencapai rata-rata tertinggi pada `tight-space` dan 600 pada seluruh seed `base`. Namun, pada `tight-capacity`, nilai akhirnya tetap 300. Pada konfigurasi ini, populasi dan crossover belum memberi keuntungan pada semua struktur kasus. Biaya GA juga tidak berbanding lurus dengan banyak evaluasi. Pembentukan populasi, crossover, perbaikan konflik penempatan, dan validasi armada menambah pekerjaan di luar pemanggilan fungsi objektif.

Eksperimen armada memakai paket sintetis dan truk identik. Studi dua truk menggunakan lima seed, sedangkan perluasan tiga sampai lima truk hanya memakai satu seed pada tiga fixture utama. Hasil perluasan belum mengukur variasi antar-seed dan tidak mendukung generalisasi. Belum ada pengujian truk heterogen. Parameter hanya memakai satu konfigurasi baseline per algoritma dan biaya komputasinya berbeda. Batas atas nilai 600 diketahui untuk `base` dari jumlah nilai seluruh paket. Nilai tertinggi pada fixture lain hanya disebut sebagai hasil terbaik yang ditemukan, bukan optimum global. Benchmark dua truk pada tiga fixture utama dapat diulang ke direktori terpisah dengan `python -m src.experiments.fleet.run_benchmark -truck-counts 2 -output output/fleet-two-truck-check`. Benchmark jumlah truk dapat diulang dengan `python -m src.experiments.fleet.run_benchmark`. Catatan run, ringkasan, dan visualisasi tersimpan di `output/fleet-analysis/` dan `output/fleet-count-analysis/`.

Bab 5

Kesimpulan dan Saran

5.1 Kesimpulan

Tugas besar ini mengimplementasikan empat varian *hill climbing*, *simulated annealing*, dan *genetic algorithm* untuk persoalan pemuatan paket tiga dimensi pada truk, di atas representasi *state*, validator *constraint*, dan fungsi *neighbor* yang sama. Dari eksperimen pada Bab 4, diperoleh kesimpulan berikut.

1. **Seluruh algoritma menghasilkan *state* yang valid.** Seluruh 330 *run benchmark* satu truk, 330 *run* armada dua truk, dan 54 *run* armada tiga sampai lima truk berakhir pada *state* yang lolos validator. Pada `infeasible`, setiap *run* mengembalikan *state* kosong dengan nilai 0, dan kedelapan masukan tidak valid ditolak *loader* sebelum pencarian dimulai. Representasi yang mengizinkan paket berada di luar truk membuat *state* kosong selalu layak sehingga pencarian tidak pernah kehilangan solusi yang valid.

2. ***Random restart*, SA, dan GA memberi kualitas yang setara dan lebih baik daripada varian *hill climbing* lain.** Pada `base` dan `tight-capacity`, rata-rata ketiganya berada dalam sekitar 2% dari optimum (335 dan 175) dan dalam sekitar 1% satu sama lain, sedangkan *steepest*, *sideways*, dan *stochastic* tertinggal 5–10% dengan simpangan baku yang jauh lebih besar. Dengan lima *seed*, selisih di antara ketiga algoritma terbaik lebih kecil daripada simpangan bakunya sehingga tidak ada satu algoritma yang dapat dinyatakan paling unggul.

3. **Dari sisi biaya, *random restart* paling efisien pada konfigurasi yang diuji.** *Random restart* mencapai kualitas yang sebanding dengan SA dan GA dengan 383 evaluasi objektif, dibandingkan 3001 pada SA dan 820 pada GA. *Steepest* paling murah (43 evaluasi), tetapi hasilnya sangat bergantung pada *state* awal. Durasi per evaluasi GA paling tinggi karena *crossover*, perbaikan konflik, dan validasi turut dihitung sehingga jumlah evaluasi saja bukan ukuran biaya yang adil antaralgoritma.

4. **Kemampuan keluar dari *local optimum* adalah pembeda utama.** *State* awal *seed* 12 pada `base` (bernilai 285) sudah merupakan *local optimum* bagi *steepest* dan *sideways*. SA keluar darinya dengan menerima gerakan yang memburuk pada suhu tinggi (naik ke 320), sedangkan *random restart* dan GA keluar dengan membangkitkan *state* baru. Kasus `rotation-only` menunjukkan bahwa sebagian *local optimum* berasal dari keterbatasan operator *neighbor*. Operator *move* tidak pernah mengeluarkan paket dari truk dan operator *swap* selalu memberi orientasi awal sehingga dari beberapa *state* tidak ada jalur ke *state* yang lebih baik.

5. **Kendala ruang lebih sulit daripada kendala kapasitas.** Pada `tight-space`, tidak ada algoritma yang rata-ratanya mencapai nilai terbaik teramati (300), padahal batas atas *knapsack* berbasis berat adalah 335. Kesulitannya terletak pada penataan geometris, bukan pada pemilihan paket. Sebaliknya, `tight-capacity` hampir selalu diselesaikan secara optimal oleh *random restart*, SA, dan GA.

6. **Perbedaan antarvarian *hill climbing* sesuai dengan mekanismenya.** *Sideways move* membantu bila ada dataran yang dapat dilalui (misalnya pada `compact_value` dan `tight-capacity`), tetapi 56–88 kali lebih lambat daripada *steepest*, dan sekitar 18–24% gerakannya tidak mengubah *state*. *Stochastic* hampir tidak lebih baik daripada *steepest* meskipun sekitar 32 kali lebih lambat. Sebagian keunggulan *random restart* berasal dari anggaran sebelas pendakian, bukan semata dari aturan penerimaannya.

7. **Pada GA, ukuran populasi paling berpengaruh dan operatornya memberi manfaat kecil.** Menaikkan populasi dari 10 ke 40 menaikkan rata-rata dari 327 ke 333 dengan durasi sekitar lima kali, sedangkan menambah generasi di atas 40 tidak mengubah hasil. Laju mutasi 0,05 memberi rata-rata tertinggi. Tanpa mutasi, seluruh *run* berhenti pada 330, sedangkan laju 0,20 memperlebar sebaran. Ablasi menunjukkan bahwa menghilangkan *crossover* atau mengganti *roulette-wheel* dengan seleksi seragam menurunkan rata-rata 4–5 poin. Operator *crossover* beserta *repair* menjadi bagian termahal dari satu generasi.

8. **Pilihan *objective function* menentukan solusi yang dicari.** `eta_weighted_value` memilih himpunan paket yang lebih mendesak dengan mengorbankan nilai total. Sebagai contoh, SA pada `base` menurunkan nilai total dari 330 ke 305 dan menaikkan skor ETA sekitar 24%. `compact_value` terutama mengubah penataan menjadi tumpukan yang lebih rendah tanpa mengurangi nilai total. Gradien kecilnya membantu varian *hill climbing* melewati dataran.

9. **Pada armada dua truk, pola utamanya bertahan dengan perbedaan pada kasus ketat.** Pada `base` armada, semua metode kecuali *steepest* dan *stochastic* mencapai batas atas 600 pada kelima *seed*. GA memberi rata-rata tertinggi pada `tight-space` (472), sedangkan SA adalah satu-satunya metode yang memperbaiki `tight-capacity` (rata-rata 323 dari nilai awal 300) dengan menukar paket yang dipilih.

10. **Pada eksperimen satu seed, penambahan truk berkaitan dengan hasil lebih baik di fixture ketat, tetapi tidak merata.** Semua metode mencapai nilai 600 pada `base` untuk tiga sampai lima truk. Pada `tight-space`, seluruh metode mencapai 600 mulai empat truk. Pada `tight-capacity`, hasil terbaik teramati menjadi 600 pada empat dan lima truk. Karena hanya seed 11 yang diuji dan state awal berbeda antarjumlah truk, temuan ini merupakan indikasi pada fixture yang diuji, bukan bukti keunggulan umum jumlah truk tertentu.

Secara keseluruhan, tidak ada satu algoritma yang unggul di semua kasus. *Random restart hill climbing* memberi keseimbangan kualitas dan biaya terbaik pada *instance* kecil yang diuji. SA paling mampu memperbaiki *state* yang terjebak melalui gerakan memburuk, sedangkan GA paling konsisten pada `base` dan paling kuat pada `tight-space` armada. Kesimpulan ini berlaku untuk *instance* sintetis berukuran 1–10 paket dan satu konfigurasi parameter per algoritma. Eksperimen utama memakai tiga sampai lima *seed*, sedangkan perluasan jumlah truk hanya memakai satu seed.

5.2 Saran

Berdasarkan hasil dan keterbatasan eksperimen, beberapa pengembangan berikut disarankan.

1. **Memperbaiki operator *neighbor*.** Operator *move* sebaiknya dapat mengeluarkan paket dari truk, dan *swap* sebaiknya mencoba seluruh orientasi paket yang masuk, bukan hanya orientasi awal. Perubahan ini menghilangkan *local optimum* buatan seperti pada `rotation-only`. Selain itu, `random_neighbor` sebaiknya tidak mengembalikan *state* asal agar jatah *sideways move* tidak terbuang untuk gerakan *no-op*.

2. **Menyamakan anggaran pencarian.** Perbandingan berikutnya sebaiknya memberi setiap algoritma jumlah evaluasi objektif atau batas waktu yang sama, termasuk menyamakan jumlah pendakian *random restart* dengan anggaran algoritma lain. Dengan begitu, keunggulan suatu algoritma dapat dipisahkan dari besarnya anggaran yang diterimanya.

3. **Menambah pengulangan dan uji statistik.** Jumlah *seed* perlu ditambah, misalnya menjadi 30, dan perbandingan berpasangan diuji dengan uji nonparametrik seperti Wilcoxon *signed-rank*. Dengan lima *seed*, selisih antara *random restart*, SA, dan GA belum dapat dibedakan dari *noise*.

4. **Menyetel parameter secara sistematis.** Suhu awal SA sebaiknya disesuaikan dengan skala *objective*, misalnya dari rata-rata $|\Delta E|$ pada beberapa gerakan awal, karena skala `eta_weighted_value` jauh lebih kecil daripada `total_value`. Sweep GA dapat diperluas ke kombinasi populasi dan laju mutasi serta dijalankan pada lebih dari satu *instance*, dan SA dapat diberi kondisi berhenti dini agar tidak selalu menjalankan seluruh iterasi.

5. **Memakai *instance* yang lebih besar dan optimum yang diketahui.** *Instance* dengan puluhan hingga ratusan paket diperlukan untuk menguji skalabilitas. Untuk kasus yang dibatasi ruang seperti

`tight-space`, optimum sebenarnya dapat dihitung dengan solver eksak, misalnya formulasi *integer programming*, agar kualitas solusi dapat diukur terhadap acuan, bukan hanya terhadap nilai terbaik yang ditemukan.

6. **Mencoba pendekatan hibrida.** Hasil menunjukkan bahwa algoritma yang membangkitkan *state* baru dan algoritma yang menerima gerakan memburuk sama-sama efektif keluar dari *local optimum*. Kombinasi keduanya, seperti GA yang menjalankan *hill climbing* singkat pada anak (*memetic algorithm*) atau SA dengan *restart*, layak diuji.

7. **Memperluas eksperimen armada heterogen.** Eksperimen armada saat ini memakai truk identik. Pengujian berikutnya dapat memakai truk dengan ukuran dan kapasitas berbeda serta menambahkan operator yang memindahkan paket antartruk secara langsung.

8. **Menelusuri temuan yang belum dijelaskan.** Dua temuan belum diselidiki penyebabnya. SA dengan `compact_value` sekitar 46% lebih lambat pada `base`, sementara *random restart* kalah dari *sideways* pada `eta_weighted_value`. Profil waktu dan eksperimen dengan lebih banyak *seed* dapat memastikan apakah keduanya nyata atau hanya *noise*.

Bab 6

Lampiran

6.1 Pembagian Tugas

Pembagian tugas mengikuti pembagian yang tercatat pada `README.md` repositori.

Nama	NIM	Kontribusi
Muhammad Aufar Rizqi Kusuma	13524061	Kontrak dan fondasi bersama (model, validator, *neighbor*, *SearchResult*), *genetic algorithm*, serta bonus multi-truk
Arina Azka	13524049	*Hill climbing* wajib beserta seluruh variannya (*steepest*, *sideways*, *stochastic*, *random restart*) sebagai bonus
Michael James Liman	13524106	*Simulated annealing* wajib dan *objective function* alternatif (`eta_weighted_value` dan `compact_value`) sebagai bonus, serta *visualizer* berbasis *web server*

Tabel 6.1: Pembagian tugas anggota kelompok Three Agents

6.2 Repositori GitHub

Kode sumber, data eksperimen, visualisasi, dan sumber laporan ini tersedia pada repositori berikut.

Repositori GitHub Three Agents

DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM	: Muhammad Aufar Rizqi Kusuma, Arina Azka, Michael James Liman
Fakultas/Sekolah	: Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah	: IF3170
Nama Mata Kuliah	: Inteligensi Artifisial
Judul Tugas/Proposal	: Penerapan Algoritma Local Search untuk Optimasi Pemilihan dan Penataan Paket dalam Peti Kemas

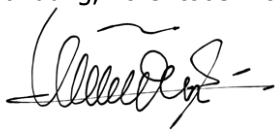
Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak menggunakan~~* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya/~~Tidak~~*	2- AI-assisted idea generation and structuring
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/~~Tidak~~*	2- AI-assisted idea generation and structuring
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/~~Tidak~~*	3 - AI-assisted workflow
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya/~~Tidak~~*	
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Muhammad Aufar Rizqi Kusuma (13524061)

Bibliografi

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[2] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[3] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.

[4] D. L. Poole and A. K. Mackworth, *Artificial Intelligence: Foundations of Computational Agents*, 2nd ed. Cambridge University Press, 2017. [Online]. Available: https://artint.info/2e/html2e/ArtInt2e.Ch4.S7.SS1.html